TECHNISCHE
UNIVERSITÄT
WIEN
Vienna | Austria

Packet Sampling for Lightweight Machine Learning-based Anomaly Detection

BACHELOR'S THESIS

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Telecommunications

by

Patrick Resch
Registration Number e1027075

to the Faculty of Electrical Engineering and Information Technology

at the TU Wien

Advisor: Univ.Prof Dipl.Ing. Dr.-Ing. Tanja Zseby
Univ.Ass. Dipl.-Ing. Fares Meghdouri

Vienna, 29th October, 2021

Declaration of Authorship

Patrick Resch

I hereby declare that this thesis is in accordance with the Code of Conduct rules for good scientific practice (in the current version of the respective newsletter of the TU Wien). In particular it was made without the unauthorized assistance of third parties and without the use of other than the specified aids. Data and concepts directly or indirectly acquired from other sources are marked with the source. The work has not been submitted in the same or in a similar form to any other academic institutions.

Vienna, 29th October, 2021

Contents

Abstract

This research quantifies the effect of various packet and flow sampling techniques on machine learning-based anomaly detection in network traffic. These effects are not limited to classification results, but also take into account necessary resource usage, especially when using low-end hardware devices for the classification process. For this purpose, two different representations and a Random Forest classifier algorithm were chosen for further investigation. In order to identify bottlenecks initially a Raspberry Pi Zero W, and later on a virtualisation with similar specifications, was used for the classification chain. The results show how different network traffic representations and sampling techniques affect resource usage, but more importantly, classification accuracy. In this context, one cause of increased resource usage is the effect of different traffic representations on the complexity of the chosen classifier algorithms model. The findings open up room for more lightweight machine learning-based anomaly detection in network traffic, particularly on high-bandwith links.

Zusammenfassung

Das Ziel dieser Studie ist es, die Auswirkungen verschiedener Sampling-Methoden auf die Machine Learning-basierte Anomalieerkennung in Netzwerkverkehr zu untersuchen. Die Betrachtungen beschränken sich jedoch nicht auf die Ergebnisse der Anomalieerkennung, sondern berücksichtigen auch die unterschiedlichen Anforderungen an die benötigten Systemressourcen, insbesondere beim Einsatz leistungsschwächerer Hardware zur Klassifizierung. Um die Variablen zu identifizieren, die einen Einfluss auf den Ressourcenverbrauch haben, wurde zuerst ein Raspberry Pi Zero W für die Klassifizierung verwendet, danach eine Virtualisierung mit ähnlichen Spezifikationen. Die Resultate zeigen den Einfluss unterschiedlicher Arten der Darstellung des Netzwerktraffic und der verwendeten Samplingverfahren auf die benötigten Systemressourcen sowie auf die erzielten Klassifizierungsergebnisse. Eine der Gründe für die steigenden Anforderungen an die Systemspezifikationen ist in diesem Zusammenhang der Einfluss unterschiedlicher Darstellungsarten des Netzwerkverkehrs auf die Komplexität des generierten Modells des Klassifizierungsalgorithmus. Die gewonnen Erkenntnisse öffnen Raum für weitere Fragestellungen zur Anwendung ressourcenschonender Anomalieerkennung in Netzwerkverkehr, insbesondere für Verbindungen mit hohen Bandbreiten.

CHAPTER 1

Introduction

Due to continuous advancements in global Internet technologies, not only is the number of online users steadily increasing, but so is the amount of traffic generated. When combined with the desire to secure private data, this leads to an increase in the generation of encrypted network traffic, making anomaly detection based on payload-based classification difficult.

For any analyis of encrypted traffic payloads it is necessary to decrypt, inspect, re-encrypt and forward every single packet passing an observation point. While this can be done within company networks, in general such a process entails tremendous privacy and legal concerns for many services, precluding the access to payload data completely for many cases. Therefore, traffic classification utilizing machine learning techniques have gained attention for identifying and classifying network traffic without violating customers' privacy since this approach proves a promising alternative based on independent statistical features that can be gathered without inspecting encrypted payloads.

Much of the existing published research focuses on different machine learning algorithms' achievable accuracy (classification accuracy) for anomaly detection problems. However, there is little information available about the hardware resources used or necessary to perform such a task. The motivation of this study is to provide such information, considering various packet sampling techniques for low-end hardware usage.

This research focuses on the hardware requirements needed to perform the classifications and the impact of different sampling techniques on computational resource requirements, utilizing two different feature-vectors for feature selection, as mentioned in chapter 4.3, to perform classification on the publicly available Intrusion Detection Evaluation Dataset *CIC-IDS-2017* with the decision tree-based Random Forest classifier.

Instead of focusing solely on the algorithms' achievable accuracy, all applied sampling techniques are compared to each other, using a metric that takes classification accuracy and computational performance into account. This leads to questions pertaining to the

influence of packet sampling, resource demands and classification performance in machine learning-based anomaly detection in network traffic.

The *CIC-IDS-2017* dataset's network captures used in this study are unlabelled; a script[1] available at the CN-TU github repository is used to set the ground truth labels necessary for classification based on the information the authors published in the dataset's documentation[2].

To obtain the necessary TCP features for both investigated feature-vectors, all TCP header information must be available. As a result, this study focuses on TLS encrypted network traffic; IPsec encrypted traffic is not suitable for classification due to the lack of TCP flags.

The following paragraphs provide a brief overview of how the experiments in this study are carried out. The framework consists of several modules whose purpose is to execute various tasks and to collect results and data for subsequent analysis. All modules used in this study are described in detail in chapter 5.3.

The framework executes various processing steps on:

- **Mid-range hardware, server:** Is used to apply various sampling techniques as described in chapter 3.2 including all feature-calculations required to obtain the desired feature-vector. This machine is also used to fit the machine learning-based classifiers model described in chapter 4.2.
- **Low-end hardware:** To aquire the results for this study, the preprocessing chain is executed on a low-end hardware device, including all necessary preprocessing steps such as scaling and the use of Principal Component Analysis (PCA) before importing an already fitted model to create predictions.

The sampling chain starts with the selection of the appropriate module to apply the specified sampling technique. Flow collection is done using the feature-vector of choice, and various preprocessing and postprocessing steps are applied. The order of these steps is determined by the sampling technique used:

- **Flow collection:** To collect flows with *go-flows*, a specific configuration file based on the chosen feature-vector and sampling technique is used.
- **Preprocessing:** Several steps must be completed before sampling can be applied. These steps may differ depending on the sampling technique used.
- **Sampling:** Packet-based sampling is applied to the raw capture file while flow-based sampling is applied to packet values within previously collected flows.
- **Postprocessing:** This can include necessary calculations to finalize the chosen feature-vector as well as setting the ground truth labels for the resulting flows.

Once the sampling has been applied and the resulting flows have been generated, another module handles all of the classification steps. The feature representation generated by the sampling chain is imported, and the following steps must be taken to generate predictions:

[1] https://github.com/CN-TU/Datasets-preprocessing/tree/master/CIC-IDS-2017/labeling
[2] https://www.unb.ca/cic/datasets/ids-2017.html

- **Processing:** Various datatypes are conveted and files are split into smaller portions on-the-fly to reduce the overall memory usage.
- **Scaling:** Scaling is applied to all features to improve results.
- **PCA:** Afterwards Principal Component Analysis (PCA) is applied, as described in chapter 4.4.2, to further reduce the feature space.
- **Classification:** Finally an already existing model is either imported to create predictions, or a model is fitted and exported for later usage.

All steps during classification are executed on a low-end hardware device to gather predictions with the selected classifier algorithm. Throughout this process various system resources are monitored and logged to evaluate the strain put on the hardware.

The rest of this paper is structured as follows:

Chapter 2 provides background information on related work. This includes papers about the extraction of information from network traffic captures, machine learning and anomaly detection in general, as well as work that is loosely related to basic resource requirements and sampling related topics.

Chapter 3 defines the terminology used throughout this paper by describing common sampling techniques and their associated modes used in this research.

Chapter 4 gives a brief introduction into machine learning-based anomaly detection, the classifier algorithm used in this study, feature representation in form of feature-vectors and and the utilized technique to reduce the feature space.

Chapter 5 goes over all of the framework's modules in detail explaining various code excerpts. This section also describes all tools used to obtain the results, the utilized configurations to collect flows, as well as some basic hardware specifications.

Chapter 6 lists all experiment configurations, defines metrics, and describes the dataset used to generate the results of this work.

Chapter 7 displays the gathered data and presents the research findings.

Chapter 8 summarizes this research, mentioning issues that appeared during this research, presenting conclusions based on the obtained results, and finally provides an outlook on possible improvements and future research avenues to pursue in order to continue the work on this topic.

CHAPTER 2

Related Work

The extraction of information from raw network traffic captures into a usable format is the first important topic in this section:

The paper by Ndatinya et al.[1] is a good starting point for this topic. This work includes a detailed description of how to use *Wireshark* to obtain data using various analysis techniques.

Alothman describes preprocessing steps in [2] for obtaining and converting raw network traffic data to plain text. He mentions several critical tasks that should be considered for analysis, such as data labeling, one-hot encoding, missing value imputation, and other helpful steps.

Sikos' survey on network forensics analysis [3] describes various formats for capturing and storing network packets, as well as possibilities for inspecting and analyzing captures. This article also discusses network packet analyzers, such as *Wireshark* and *tcpdump*.

Hofstede et al.[4] provide a tutorial on all stages for a flow monitoring setup. This includes descriptions on how to approach packet observation, flow metering, flow export, data collection and data analysis.

The article by Vormayr et al.[5] contains a detailed description of network flow collection, including the IPFIX network flow definition, a description of flow formats, and a comparison of nine different flow exporters, including *go-flows*, the flow exporter used in this study.

Quittek et al.[6] provide specifications for the IP flow information export (IPFIX) protocol to exchange flow information. This includes useful definitions of terminology used in this study.

D'Antonio et al.[7] present various flow selection techniques as well as motivations for using a flow selection process. This includes a definition for the flow sampling terminology mentioned in chapter 3.2.1.

Zseby et al.[8] describes sampling and filtering techniques for IP packet selection, including an overview for various random and systematic packet sampling techniques.

Iglesias et al.[9] introduce the AGM (AGgregation & Mode) feature-vector to represent network traffic. The intended purpose of this feature-vector is to characterize IP hosts by extracting agregated and mode values of IP header fields without inspecting payloads. This results in a lightweight and tailored traffic representation suitable to analyze large volumes of network traffic.

For large traffic sources, Jedwab et al.[10] investigate packet sampling with limited storage. The authors describe a packet sampling system for estimating network traffic attributes by describing a method for generating representative random samples and estimating packet counts based on those samples. A proposed truncation algorithm is used to limit packet counts of various types, and a method for binding the probability of miscounts or misranked packet types is provided.

The problem addressed in Duffields' paper[11] is how to distribute a sampling budget (limited capacity due to finite storage capacity) across subpopulations of flow records, while meeting accuracy goals and fairly sharing the budget. This paper proposes a method for sampling flow records derived from rate heterogeneous traffic subpopulations by allocating a fair sampling budget rather than using a fixed reservoir partitioning.

Machine learning and anomaly detection in general, and machine learning-based anomaly detection in particular, are the second topics for related work:

Shafiq et al.[12] discuss various traffic classification techniques for identifying network applications or protocols, as well as machine learning techniques for creating classification models, including supervised and unsupervised learning.

The article by Chandola et al.[13] provides a good overview of machine learning-based algorithms and describes various types of anomalies, algorithm outputs, applications, and classification-based anomaly detection techniques, including rule-based techniques such as decision trees.

Agrawal et al.[14] discuss anomaly detection using machine learning-based techniques like decision trees, support vector machines, neural networks and other approaches to gain basic insights for anomaly detection.

Bhuyan et al.[15] provide a detailed comprehensive overview of various network anomaly detection possibilities, including a comparison of different detection methods and systems. The paper also includes descriptions of tools that can be used in various stages of network traffic anomaly detection.

The book by Bhattacharyya and Kalita [16] provides a comprehensive overview of network anomaly detection in the context of machine learning, covering a wide range of topics.

Lim et al.[17] analysed different sources of discriminative power in classifying single-directional application traffic. Their work clearly shows that classification accuracy decreases when the first few packets of a flow are missed.

Velan et al.[18] investigate methods for encrypted traffic classification and analysis. Their work in particular shows that payload-based classification relies on knowledge of payload formats for every application.

Meghdouri et al.[19] analysed lightweight feature-vectors for attack detection in network traffic on the UNSW-NB15 dataset, including the CAIA and AGM feature-vectors used in this study. The investigated classification algorithms also include the Random Forest classifier.

The work of Van Essen et al.[20] shows that compact Random Forest models composed of many, small trees rather than fewer, deep trees does not necessarily drop prediction accuracies.

The third important topic for this study is the impact of sampling on resource usage as well as the classifiers performance. Finding relevant research on this topic proved to be difficult.

Vargaftik et al.[21] investigate the increasing memory footprints of decision tree-based ensemble classification methods for anomaly detection in general, but their research is unrelated to network traffic or the use of sampling to reduce resource requirements. Instead, the authors' work looks into ways to reduce the Random Forest classifier's memory and performance drawbacks by presenting a method to reduce model sizes and, as a result, the resources required for classification.

The relationship between the sizes of tree-based machine learning algorithm models and runtime performance is investigated in the paper by Asadi et al.[22], which demonstrates that removing branches of the model or operating on multiple feature vectors at once can optimize runtime performance.

Williams and Zander [23] investigated various metric comparisons such as model build times or classification speed, as well as the importance of speed for near real-time classification on large numbers of concurrent network flows.

Arndt and Zincir-Heywood [24] compare three different machine learning techniques for analyzing encrypted network traffic. This work also investigates sampling (randomly and continuously) for the training set when traffic is presented as flow-based statistics, and it presents training and testing times for all algorithms considered in their work.

Erman et al.[25] already discussed runtime analysis in their work on Internet traffic identification and concluded that the size of the training set is ultimately limited by the amount of memory available on a machine.

Other papers concentrate on hardware design for machine learning, with no consideration for low-end hardware usage. Many papers concentrate on the design of system-on-chips or FPGAs, for example, to suit a specific usecase or on a specific machine learning-based classifier type, such as decision trees.

Sze et al.[26] evaluate hardware opportunities in machine learning architectures in general, focusing on CPU and GPU, sparsity, compression, and accelerators, as well as ways to

speed up specific operations.

Taghavi and Shoaran [27] investigate system-on-chips hardware and performance comparisons for deep neural networks and decision tree ensembles for real-time classification, comparing hardware and performance while taking energy efficiency, hardware utilization, and storage requirements into account.

The majority of published papers on machine learning-based network traffic anomaly detection provide little information, neither about the hardware used nor about detailed resource usage in general. Some papers on machine learning-based anomaly detection, on the other hand, are concerned with performance-related issues. This research is most closely related to papers investigating the impact of machine learning classification on resource usage. Vargaftik et al.[21], in particular, looked into resource-efficiency for supervised anomaly detection using decision tree-based ensemble classification methods. The authors present a framework for enhancing decision tree-based classifiers in order to reduce model complexity while achieving good anomaly detection results. Their research focuses on classifier models in order to reduce memory size, training time, and classification latency. This study, on the other hand, focuses on the impact of various traffic representations combined with various sampling techniques on both resource usage and classification performance without augmenting model generation. As the following chapters will demonstrate, reducing the overall memory footprint for classification is an important consideration, particularly on low-end devices.

CHAPTER 3

Background on Packet Sampling

In the field of communication networks, the term packet sampling refers to various types of packet extraction techniques. The purpose of using sampling techniques in general is to avoid collecting all packets received by a router or a similar device in order to reduce hardware requirements for subsequent processing. Because of the ever-increasing transmission speed and total amount of network traffic generated, sampling techniques for characterizing and classifying network traffic are expected to become even more important in the future.

In this study, the use of various sampling techniques on network traffic aims to reduce the amount of hardware resources required for classification while also investigating the impact on classification accuracy. Reduced hardware requirements are especially desirable when considering low-end hardware devices for machine learning-based anomaly detection.

3.1 Communication Networks

Today's communication networks make physical distances meaningless while continuously becoming smarter and more complex. Computer network security is a critical issue since various types of attacks through the internet are constantly inflicted and therefore a huge amount effort is put into the development of mechanisms to defend against intrusions. This forces network engineers to analyse network traffic for better understanding and prevention of network-related attacks.

In the Internet all network communication can be reduced to sending and receiving packets. Current high-speed links may pass millions of packets per second and hundreds of thousands of active flows. Captured packets can reveal the signature of attacks and enable prevention from damages caused by attacks. Packet analysis can be a useful tool to diagnose a network, opening the possibility to enforce network security counter-measures. Therefore it seems reasonable to utilize methods to reduce the amount of examined packets for anomaly detection which reduces both time and resources significantly.

All network packets can be observed directly on the wire on any network interface while routed to their destination like e.g. router ports or a set of (physical or logical) network interfaces of a router. Those packets can provide precious information in the header, beside their actual payload, which enables the creation of various statistics based on counters for packet properties.

3.2 Common Sampling Techniques

Considering the focus on low-end hardware devices in this study it may become important to reduce the amount of packets that need to be analysed while maintaining high classification accuracy-scores. In terms of computational performance and resource usage for preprocessing, it seems intuitive that the more packets that can be dropped, the better.

But this idea may not apply for the pure classification itself, for example any flow-based sampling technique implemented in this study is not going to reduce the number of flows to classify at all. Though the reduction of packets (and information) within a flow due to the application of different kind of sampling modes often results in some kind of degraded classification accuracy-scores. In this study, various sampling techniques and modes are applied to network captures to investigate their impact on classification accuracy-scores and resource requirements.

As already described by the *Sampling and Filtering Techniques for IP Packet Selection*[1] working group within IETF[8], sampling modes can be divided into two basic classes: *systematic* sampling and *random* sampling. While *random* packet sampling relies on a probability distribution function to randomly select packets, *systematic* techniques select

[1] https://tools.ietf.org/html/rfc5475

packets deterministically based on a chosen criterion. All modes utilize the possibility to drop packets from given network traffic, but the approaches are vastly different.

Two different sampling techniques are used in this work, which are referred to as *packet-based* and *flow-based* sampling. *Packet-based* sampling selects packets using a deterministic or non-deterministic method and exports flows based on each selected packet, whereas *flow-based* sampling, as it is implemented in this study, first exports all observed packets into flows and then selects packets within each flow using a deterministic method. For both sampling techniques, the percentage of actual sampled packets is recorded and listed in table 6.1 for all investigated experiment configurations.

The following sections describe the sampling techniques used in this study.

3.2.1 Flow-Based

To begin with, a basic understanding of flow-based sampling techniques, a definition of what determines a network flow needs to be described. Several definitions of IP flows can be found. This work follows the definition of IP flows as described by the *Requirements for IP Flow Information Export (IPFIX)*[2] working group within IETF[6]:

"A flow is defined as a set of IP packets passing an observation point in the network during a certain time interval. All packets belonging to a particular flow have a set of common properties."

Within the IPFIX terminology, a specific set of common properties is called flow key. The flow key determines what packets belong to a certain flow. In other words a flow can be described as a set of packets transmitted from a source to a destination, sharing several common properties in their transport or application header fields:

$$\begin{gathered} F_i = \{H_i, P_i\} \\ H_i = \{IP_{src}, IP_{dest}, Port_{src}, Port_{dst}, Protocol\} \\ P_i = \{p_j \in P \mid h_j == H_i\} \\ i = 1, \ldots, N \; flows \\ j = 1, \ldots, M \; packets \end{gathered} \tag{3.2.1}$$

A flow F_i consists of a set of packets $P_i = \{p_{j,1},\ldots,p_{j,m}\}$ as subset of all packets P passing through the observation point, where all packets headers within one flow are matching the flow-key properties $h_j == H_i$, in this example it is the popular 5-tuple used in this study to obtain the CAIA feature vector.

$$F_{fb} = F_f \cup F_b \tag{3.2.2}$$

A bi-directional flow F_{fb} can be gathered by matching specific elements of the flow-key, in general switching IP_{src} and IP_{dest} and the associated port numbers. This results in

[2] https://tools.ietf.org/html/rfc3917

merging packets tied to the forward-directed flow F_f and the backward-directed flow F_b into the bi-directional flow F_{fb}.

To limit the number of packets collected in a single flow, two timeouts are defined: active and idle timeouts. When the active timeout expires, the flow is always terminated, and packets matching the same key observed after the expired active timeout are collected in a new flow. The idle timeout lasts less time than the active timeout. A flow is terminated if no new packets matching the flow key are observed for the time period of the idle timeout. Afterwards packets matching the same key are collected in a new flow.

It is mentionworthy that the implementation for *flow-based* sampling in this study differs significantly from the terminology used in *Flow Selection Techniques*[3] within IETF[7]:

"Flow sampling usually aims at the selection of a representative subset of all Flows in order to estimate characteristics of the whole set (e.g., mean Flow size in the network)."

The main difference between both techniques is illustrated in figure 3.1. While the IETF definition selects a subset of flows from all flows observed, *flow-based* sampling implemented in this study samples packets within each flow but considers all flows for further classification.

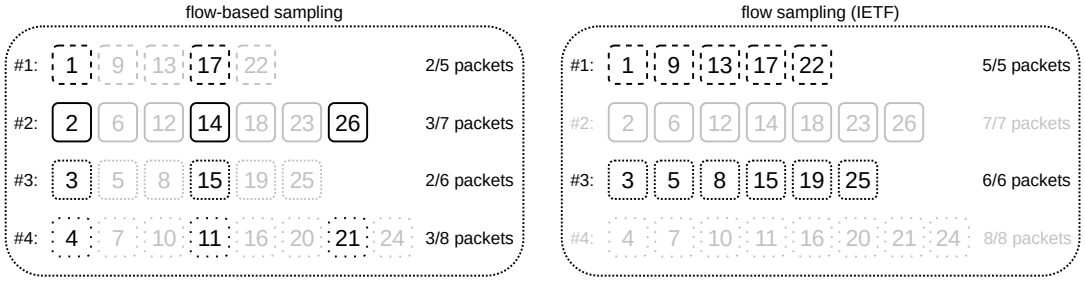

Figure 3.1: Flow-based sampling and flow sampling as defined by IETF

The upper part of figures 3.2 shows how packets passing through an observation point are collected and exported into flows. In order to generate these flows from the observed packets, *go-flows* is executed with a JSON configuration file containing the used flow key, timeout numbers and other various specifications. *go-flows* is a flow exporter written in go that can be utilized to create flows from a provided capture file, based on a given configuration. More information about the parameters used in the configuration file can befound in chapter 5.1.1.

A very detailed description on network flows in general and the flow exporter *go-flows* used in this study can be found in the paper from Vormayr et al[5].

It is easier to illustrate what flow-based sampling should accomplish after introducing the concept of exporting flows based on flow-keys. Only systematic methods for flow-based sampling were used in this work, and the following modes were investigated:

[3] https://tools.ietf.org/html/rfc7014

- every n-th packet
- sample first n packets

The lower part of figure 3.2 depicts all implemented flow-based sampling methods, a mentionable detail is that the first packet of a flow always gets sampled.

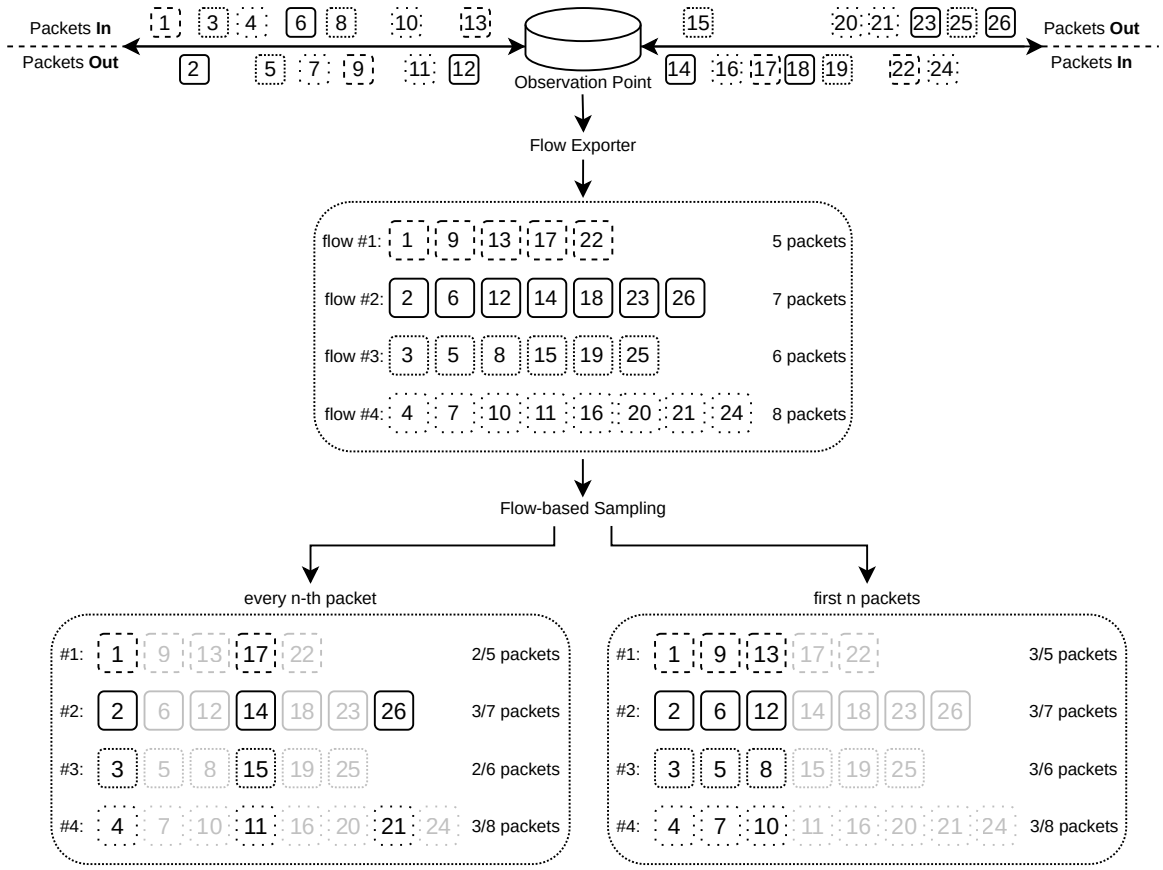

Figure 3.2: Flow-based sampling (n = 3)

3.2.2 Packet-Based

While the flow-based sampling methods used in this work are applied after the actual flow collection has occurred, the packet-based sampling is applied directly on the dataset prior to the generation of flows with *go-flows*. In practice, such an implementation would put less strain on flow generation resources because the number of packets that need to be processed by the flow exporter is reduced with this sampling method beforehand.

One disadvantage of packet-based sampling is that the number of packets sampled per flow can differ a great deal, depending on the total number of packets tied to a flow. This can result in completely missed flows, rendering them useless for later classification. The upper part of figure 3.2 depicts the flows collected from unsampled traffic for comparison with the results obtained using packet-based sampling, as illustrated in figures 3.3 and

3.4. Another disadvantage of this method is that it does not always sample the first packet(s) of a flow.

The first packets of a flow, according to Lim et al. [17], contribute the most to identifying application flows for UDP and TCP traffic. Hwang et al. [28] discovered that the first packets of a flow are sufficient for anomaly detection. Given these findings, packet-based sampling methods may not achieve the best possible results due to the nature of this sampling technique. It appears to be a reasonable assumption that this sampling technique might miss relevant packets and thus achieve suboptimal results, which can eventually lead to the loss of entire flows.

This paper investigates two packet-based sampling methods, one systematic and one random:

- every n-th packet
- random sampling

Figure 3.3 shows how, after using packet-based sampling, a flow does not appear at all. This can happen as a consequence of unfortunate sampling steps, which may lead to a loss of all packets from a single unique flow. As a result, a flow exporter used after sampling may not see any packets from a single flow and cannot collect the flow at all.

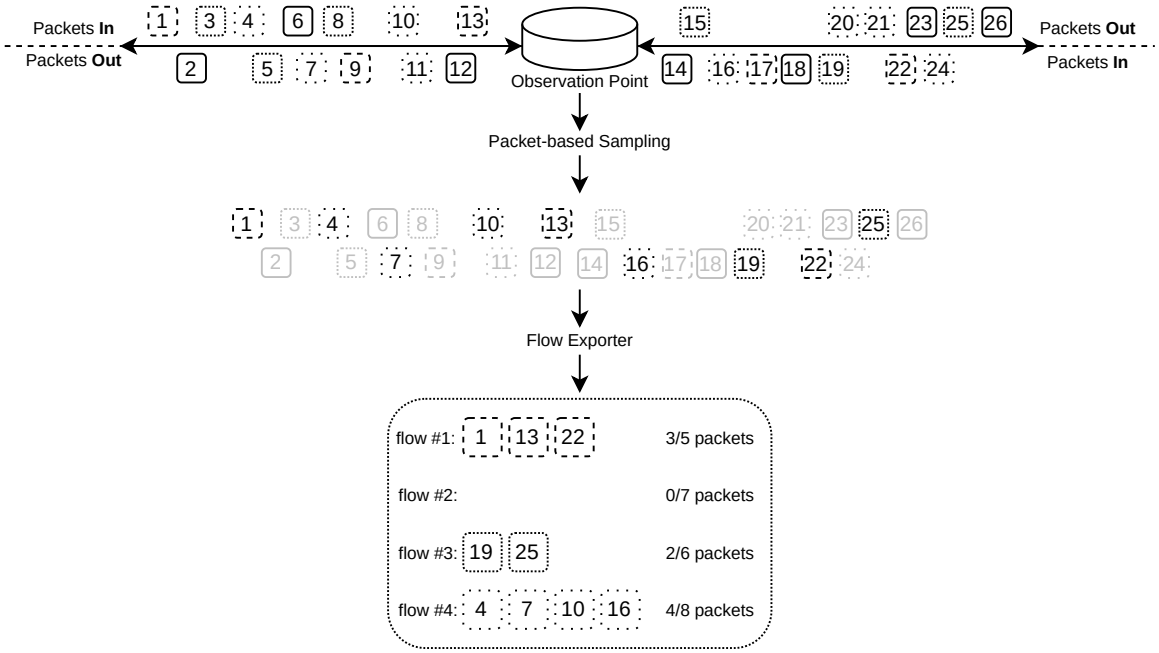

Figure 3.3: Packet-based sampling, every n-th packet (n=3)

The random sampling method is visualized in Figure 3.4. Packets are drawn from the network capture based on a given probability (p=1/n). Details about the random generator's implementation and the method for drawing packets are provided in chapter 5.3.1, specifically item ❾. After random sampling is applied all flows are exported based on the selected packets and the utilized flow key.

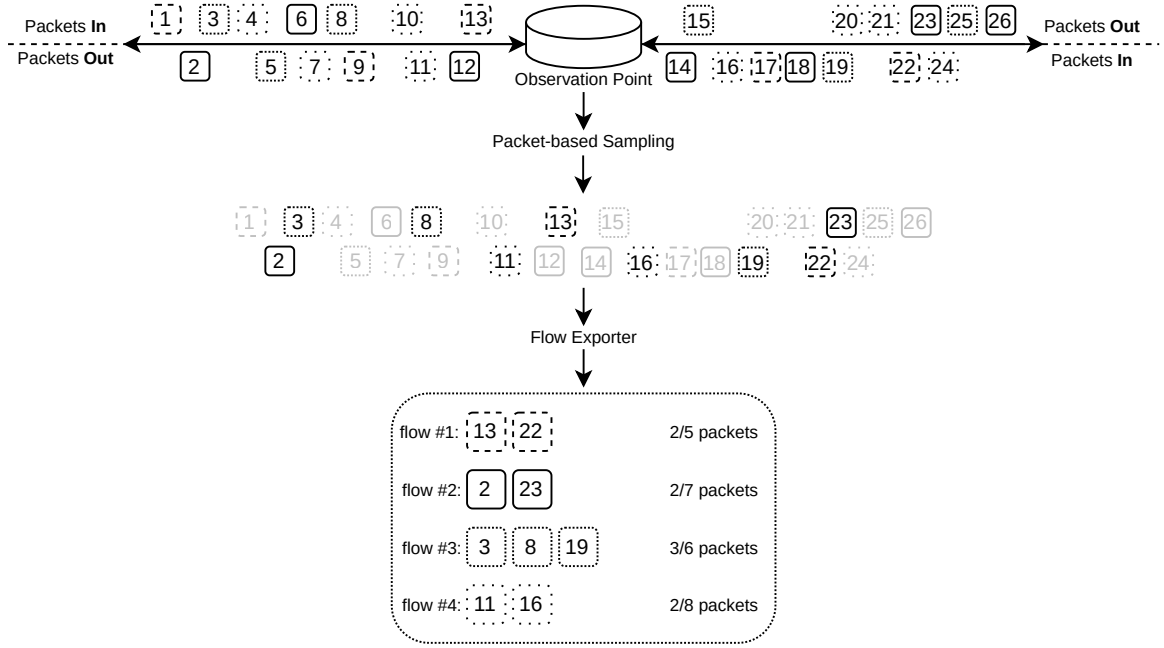

Figure 3.4: Packet-based sampling, random sampling (n=3, p=1/n)

CHAPTER 4

Machine Learning-based Anomaly Detection

For a long time, network traffic classification based on well-known TCP or UDP ports has been less accurate. There is an ever-increasing number of network applications that dynamically allocate ports as required. Moreover, users who intentionally use non-standard ports to hide their traffic or the use of network address port translation (NAT) makes this task difficult.

Along with this, there is a growing need and desire to protect transmitted data and users' privacy. Internet users have become more mindful of the importance of protecting their online privacy. As a result, the proportion of encrypted traffic is growing since many services use secured channels for traffic transmission. Traditional payload-based classification relies on knowledge of the payload formats for every application and may becomes infeasable when encryption is adopted. Velan et al. [18] treat this topic in their survey about encrypted traffic classification.

To address this problem, alternative methods for classifying network traffic have been researched for over a decade, utilizing various approaches based on unencrypted packet information rather than analyzing the payload.

4.1 Introduction to Machine Learning

In an article published in the IBM Journal of research and development in 1959, Arthur Samuel described machine learning as the ability of programming computers to learn from experience.[29] Machine learning focuses on the creation of computer programs that can access data and use it to learn for themselves, with the ability to automatically learn and benefit from experience without being explicitly programmed.

In other words, machine learning techniques are concerned with developing a system that enhances its efficiency based on previous findings and has the potential to modify behavior based on newly acquired knowledge.

Machine learning methods can be classified into different categories based on the nature of the learning system:

Supervised Learning: In supervised learning the learning algorithm approximates a function to predict output values based on an analysis of a labelled dataset. After sufficient training is done, such a system is capable of predicting outputs for any new input the system is given.

Unsupervised Learning: No labels are given to the learning algorithm, leaving it on its own to find structure in the given input. Unsupervised learning studies how systems can infer a function to describe a hidden structure from unlabelled data. Therefore the system does not figure out the correct output, it just explores the data and can draw inferences from datasets to describe hidden structures from unlabelled data.

Semi-Supervised Learning: As a combination of supervised and unsupervised learning methods, semi-supervised learning is located between the two. During training, semi-supervised models combine a small amount of labelled data with a larger amount of unlabelled data since incorporating both datatypes can result in a significant improvement in learning accuracy.

Reinforcement Learning: This type of machine learning uses an agent that learns to behave in an environment based on some kind of reward & punishment system. Trial and error search and delayed reward are the most relevant characteristics of reinforcement learning. This method allows to automatically determine the ideal behaviour within a specific context in order to maximize its performance.

It is important to understand what exactly needs to be predicted in order to understand what kind of machine learning can be applied to a given problem statement. The ability to detect anomalies in network traffic is of course the most important feature of anomaly detection. Anomalies or outliers are patterns that do not adhere to predefined normal behavior, and the cause of such events may be malicious activity, interference, or simply surprising yet harmless behavior. The process associated with the categorization of instances (e.g., packets or flows in network traffic) can be used to describe classification-based anomaly detection in general within this problem statement. Classification can be used to distinguish attacks from benign traffic, allowing for the detection of attack flows

in network traffic. The label of possible classes is a category, different instances may be assigned to one or more categories.

All described methods are capable to solve anomaly detection problems. In light of the problem statement, the supervised learning method was chosen to perform machine learning-based network anomaly detection.

Figure 4.1 depicts a typical methodology used to apply supervised learning algorithms:

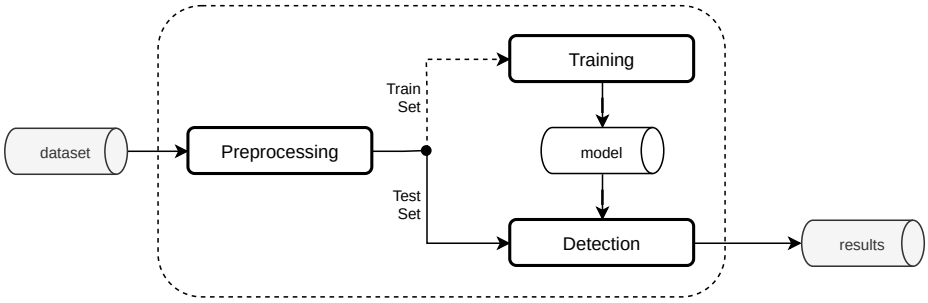

Figure 4.1: Supervised learning approach

Preprocessing: Network captures given as input file have to be pre-processed in order to provide the data in a pre-established format necessary to feed the chosen classifier.

Training: For training labelled data is necessary, allowing the classification algorithm to learn given attack patterns, and building a model based on the training data.

Detection: Once the model is built it can be used to classify given network traffic, being able to distinguish between benign and malicious traffic from given network traffic data.

4.2 Random Forest Classifier

The Random Forest classifier is a decision tree-based, supervised learning algorithm for classification and regression. The algorithm is considered highly accurate and robust due to the large number of decision trees ensembled to create a random forest. It predicts by combining the predictions of each contained component tree and usually outperforms a single decision tree in terms of accuracy. In general, the more trees there are in the forest, the more robust the algorithm becomes.

Decision tree learning is one of the most widely used and practical methods for inductive interference. Decision trees have lots of advantages, one of the most important ones is extremly quick classification. The applications of such algorithms are diverse, ranging from medical diagnosis to image recognition to stock market statistics and recommendation engines.[30]

A decision tree always starts with a single node and branches through different potential outcomes. Each of these potential outcomes results in the formation of new nodes, which

then branch into new instances. It takes on a tree-like form or structure, similar to a flowchart. One benefit of this structure is enhanced human readability since a tree of this kind can often be represented as a set of if-then laws.

One major disadvantage of decision trees is that they may become excessively adapted to the training data. When tree classifiers become too complex, they overfit the training data, which is a fundamental limitation of their complexity.

Ho[31] demonstrated in his 1995 paper that it is possible to resolve this problem, resulting in the ability to develop trees of arbitrary complexity while increasing accuracy on training and test sets. This is achieved by creating multiple trees in a way that they generalize independently. The method used to build such trees is to randomly select subspaces of the feature space and to use randomization to select feature vector components.

Breiman[32] proposed Random Forests in 2001 as a combination of tree predictors based on the values of a random vector sampled independently and with the same distribution for all trees in the forest. After a large number of trees is generated, they vote for the most popular class. In Breiman's paper, these procedures are referred to as Random Forests, with the following definition:

"A random forest is a classifier consisting of a collection of tree-structured classifiers $\{h(\mathbf{x}, \Theta_k), k = 1, ...\}$ *where the* $\{\Theta_k\}$ *are independent identically distributed random vectors and each tree casts a unit vote for the most popular class at input* $\mathbf{x}$*."*[32]

In general, designing a decision tree requires selecting an attribute selection method and a pruning method. There are several approaches to selecting attributes for decision tree induction, with the majority of approaches assigning a quality measure directly to the attribute. As an attribute selection measure, the Random Forest classifier utilizes the Gini Index, which determines the impurity of an attribute in relation to the classes.

Each time a tree is grown to the maximum depth, so these are not pruned. This is one of the most significant advantages of the Random Forest classifier over other decision tree methods, which grow a large tree and then use a pruning method to remove some portion of the tree or use some kind of stopping criterion instead of pruning.[33]

Breiman proposes in his analysis[32] that as the number of decision trees increases, the generalization error always converges even without pruning the tree, and overfitting is not a problem due to the Strong Law of Large Numbers.

Figure 4.2 shows a schematic overview of the Random Forest algorithm's working method. As the first step, an ensemble of decision trees must be constructed for each randomly selected portion of the training set. When all of the trees have been constructed, the algorithm is said to be trained and ready to make predictions. Each tree in the ensemble generates a prediction result for an instance, and then each of those predicted results is voted on. The prediction with the most votes is selected as the final result.

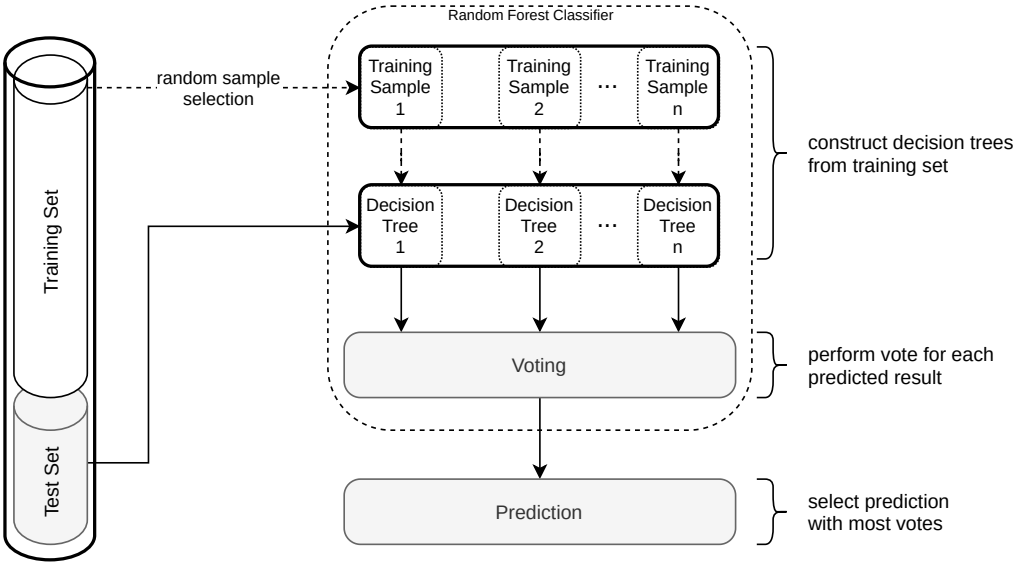

Figure 4.2: Random Forest: tree construction, voting and predicition

To summarize this section, the reason why the Random Forest classifier has been chosen for this study is that the classifier is considered highly accuraty and robust, overfitting is not an issue, and decision trees in general provide high execution speed [31]. Recent studies [19], [34] show that this classifier can achieve very good results for network anomaly detection, confirming its robustness and classifciation performance. Another useful insight that this classifier can provide for anomly detection in network traffic is the ability to recognize which features are important in the decision-making process.

4.3 Feature Representation

A feature vector, in the sense of this analysis, is a maximum m-dimensional subset of features that contains all of the attributes that must be extracted from a given network capture. This can include basic features from the provided data, as well as various statistical features that must be calculated before the classification happens. The feature vector is built by utilizing a JSON specification file to collect flows with *go-flows*, a tool to extract personalised features from traffic captures. The basic specification files used in this study to export flows with *go-flows* can be aquired from the CN-TU Git repository[1].

It should be noted that due to the inaccessibility of TCP header information when traffic is encrypted with the IPsec protocol suite, all of the TCP flag features included in the CAIA and AGM vectors introduced in the following subsections are unavailable for these feature representations if IPsec encryption is applied on network traffic.

[1] https://github.com/CN-TU/Datasets-preprocessing/blob/master/CIC-IDS-2017/flow_specifications

4.3.1 CAIA

The CAIA vector origins from the Centre of Advanced Internet Architecture at Swinburne University of Technology[2]. The authors propose the usefulness of differentiated algorithms based on computational performance rather than classification accuracy as only meaningful parameter in their paper [35].

In their study of lightweight feature vectors for attack detection in network traffic, Meghdouri et al [19] used the CAIA vector. Following the use in [17], the authors implemented eight TCP (Transmission Control Protocol) flags-related features. As a result, the final vector contains 30 features. Some changes were made to enable the calculation of min, mean, max, stdev, and total counts for various features in order to allow flow-based sampling methods. The differences of flow-based and packet-based *go-flows* configurations are shown in figure 5.2 and figure 5.3.

The CAIA vector is particularly useful for sampled network traffic because the effect of sampled packets has a minor impact on many features. Even if packets are skipped due to sampling, the minimum, maximum, and mean-values will maintain some precision, so this vector tends to be sufficient for this analysis. The resulting set of features used for the Machine Larning-based anomaly detection are shown in table 4.1.

The following features determine the flow key used by the CAIA vector, it is the popular 5-tuple:

$$(srcIP,\ srcPort,\ dstIP,\ dstPort,\ protocol) \tag{4.3.1}$$

The 5-tuple features are source network address, source port, destination network address, destination port and the transport protocol identifier.

This vector creates bidirectional flows, collecting the forward and backward directed packets within one flow as long as they match the forward and reversed flow key. In this case packets in the same flow need to have the same transport protocol identifier, and matching network addresses and transport ports for source and destination and vice versa.

The variance used for the standard deviation is calculated according to Welford's method, similar to how *go-flows* calculates the variance, in order to obtain similar results for flow-based and packet-based sampling. Welford's online algorithm is a single-pass or streaming algorithm that calculates the variance without having to keep all of the values during data collection by calculating the sum of squares of differences from the current mean value on a continuous basis. When memory access is limited, this can be extremly useful. It is a characteristic of flow collection in a real environment that the number of packets belonging to one flow cannot be predicted, therefore it is uncertain how much memory access is necessary. This algorithms' advantage is that it reads every input exactly once, making this method very suitable for calculating the variance for network flows on-the-fly. Implementation details are described in chapter 5.3.2, item 9.

[2] www.caia.swinburne.edu.au

Table 4.1: CAIA features

CAIA	
flow key:	
srcIP, dstIP,	internet protocol address
srcPort, dstPort,	port number
protocol	protocol number
features[3]**:**	
protocol,	
#Pkts_forward, #Pkts_backward,	number of incoming packets sent by source, destination
#Bytes_forward, #Bytes_backward,	number of bytes sent by source, destination
min_PktLength_forward, mean_PktLength_forward, max_PktLength_forward, stdev_PktLength_forward, min_PktLength_backward, mean_PktLength_backward, max_PktLength_backward, stdev_PktLength_backward,	metrics for packet-lengths
min_PktIAT_forward, mean_PktIAT_forward, max_PktIAT_forward, stdev_PktIAT_forward, min_PktIAT_backward, mean_PktIAT_backward, max_PktIAT_backward, stdev_PktIAT_backward	metrics for time differences between consecutive packets
flags:	
#TCPflag_forward:syn, #TCPflag_forward:ack, #TCPflag_forward:fin, #TCPflag_forward:cwr, #TCPflag_backward:syn, #TCPflag_backward:ack, #TCPflag_backward:fin, #TCPflag_backward:cwr	number of TCP flags occuring within a flow

4.3.2 AGM

The AGgregation & Mode (AGM) vector format was introduced by the authors of [9] to observe Internet Background Radiation in empty network address spaces. This vector characterizes host behavior by observing seven lightweight flow features and the total number of packets, as listed in table 4.2. This vector is suitable to analyze large traffic volumes, being able to capture long-term events like scanning as well as short-term targeted attacks and incidents.

The AGM flow key used in this study is significantly different from the CAIA flow key introduced in the previous section. This flow key collects unidirectional flows based on the packets source IP address and a 10 second observation window, ignoring different protocols, source and destination port numbers. This means that all packets connected to the same source IP address within a 10 second timeframe are grouped within the same

[3]source (src) features are forward directed, destination (dst) features are backward directed

flow. Packets matching the flow key outside of the observation window are collected and exported into a new flow. As a result, the AGM feature representation forms coarser-grained flows compared to CAIA. The numbers of collected flows for both feature representations investigated in this study are shown in table 6.3.

Table 4.2: AGM features

AGM	
flow key:	
srcIP	internet protocol address
timeWindow10s	observation period
features[4]**:**	
dstIP: distinct, mode, modeCount	internet protocol destination address
srcPort: distinct, mode, modeCount dstPort: distinct, mode, modeCount	port numbers
protocol: distinct, mode, modeCount	protocol number
ipTTL: distinct, mode, modeCount	Time To Live (TTL) contained in the packet header
octets: distinct, mode, modeCount	total number of octets in incoming packets
#packets	number of packets sent by srcIP during observed period
flags:	
tcpFlags: distinct, mode, modeCount	textual representation of all occuring TCP flags

The seven defined flow-features are further processed to obtain the number of unique values (*distinct*), the statistically most frequent occuring value (*mode*) and the number of packets corresponding to the statistical mode (*modeCount*). Thus, considering the total number of packets sent by the source IP address, the AGM vector consists of 22 features in total.

The source IP address and the statistical mode value for the destination IP are dropped before the feature vector gets passed to the Random Forest classifier. These features are removed due to their widely spread distribution and lack of information required for classification. In order to only pass numerical feature values to the Random Forest classifier the TCP flag mode feature is transformed into dummy variables, one feature per flag. The drawback of this method is the increase of dimensions. This results in an overall 29-feature vector used for classification.

[4]distinct: number of unqiue, mode: most frequent occuring value, modeCount: number of packets corresponding to the statistical mode

4.4 Feature Selection and Reduction

4.4.1 Selection

Feature selection is the process of selecting a subset of m features from the original n-dimensional feature space:

$$\begin{gathered} m \ll n, \\ \{f_1, ..., f_m\} \subset \{f_1, f_2, f_3, ..., f_n\} \end{gathered} \tag{4.4.1}$$

One of the primary reasons for feature selection is to improve an algorithms' output by removing redundant or irrelevant data and overall enhancing the quality of the processed data. Training a classifier using the maximum number of available features is not always the best choice, as irrelevant or redundant data may have a negative effect on the algorithm's performance. A selection of features can still achieve high classification accuracy while significantly improving computational efficiency and, as a result, lowering the necessary hardware requirements.

4.4.2 Principal Component Analysis

In this study Principal Component Analysis (PCA) is used to achieve further feature dimensionality-reduction without loosing significant information by computing new components $\{h_1, ..., h_p\}$ based on the given set of features $\{f_1, ..., f_m\}$.

The following expressions summarize the distinction between feature selection and reduction, features generated with PCA are denoted as h_p:

$$\begin{gathered} \{f_1, ..., f_m\} \subset \{f_1, f_2, f_3, ..., f_n\} \\ \{h_1, h_2, h_3, ..., h_p\} \cap \{f_1, f_2, f_3, ..., f_n\} = \emptyset \end{gathered} \tag{4.4.2}$$

Unlike feature selection, which selects a subset of attributes from a given set of features, feature reduction generates new features based on correlation. The aim is to further reduce the dimensionality of the processed data.

$$\begin{gathered} p \ll m \ll n, \\ \{f_1, f_2, f_3, ..., f_n\} \xrightarrow[\text{selection}]{\text{feature}} \{f_1, ..., f_m\} \xrightarrow[\text{reduction}]{\text{feature}} \{h_1, h_2, h_3, ..., h_p\} \end{gathered} \tag{4.4.3}$$

Only the most significant principal components are selected for further processing in order to achieve the optimal dimensionality-reduction. In this context, significance means that the principal component with the greatest possible variance in the data can be considered the most significant component. The next principal component is calculated in the same manner as the first, but it must be uncorrelated (perpendicular) to the first and account for the next largest variance.

While the total variance is the sum of the variances of all of the original dataset's components, the fraction of explained variance by one principal component is the ratio of

that component's variance to the total variance. PCA is designed to achieve maximized variances for the first and minimized variances for the last components, allowing for dimensionality reduction without significant information loss.

CHAPTER 5

Methodology

The main purpose of this work is to study the effect of packet sampling on the performance of a state-of-the-art Machine Learning (ML) algorithm for the classification of network traffic, in particular detection of anomalies, and the hardware-requirements necessary to apply different sampling patterns and techniques. Figure 5.1 depicts all of the basic steps required to obtain classification results from a given network traffic capture. Two different sampling-techniques are applied: packet-based and flow-based sampling, utilizing different modes, as described in chapter 3.

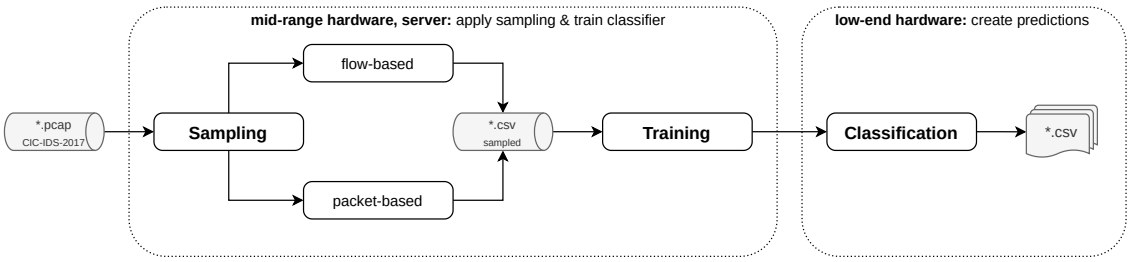

Figure 5.1: Superficial workflow

The following subsections details the tools used within the framework, including a description of the used arguments, configurations and possible modifications.

Beside basic information about the used dataset, this chapter also contains an in-detail description of the python framework used to gather all results. This includes every step within the sampling chain, necessary preprocessing and feature calculations, settings for the chosen Random Forest classifier and the possibility of automated script execution based on a pre-defined experiment configuration that can be set in the framework configuration.

5.1 Tools

5.1.1 Go-flows

go-flows is an open source[1], highly customizable flow exporter written in go by members of the Institute of Telecommunications' Communication Networks Group at the Technical University of Vienna. Vormayr et al.[5] argue for the importance of reproducibility in traffic analysis research and present flow concepts with descriptions, examples, and a comparison of various flow exporters, including the introduction of *go-flows*.

In this analysis, *go-flows* is used to export flows from provided capture files by selecting features based on the CAIA and the AGM specifications, as described in chapter 4.3. For the most part the feature vectors are obtained by utilizing a predefined JSON specification file with *go-flows*.

```
go-flows run features {json} export csv {outfile} source libpcap {infile}
```

This command is executed to collect flows using *go-flows*, passing a *json* configuration file containing the *flow-key* and the specifications required to obtain the desired feature-vector. The other arguments specify the *outfile* format as CSV and the *infile* filetype as libpcap.

JSON Specification

To collect flows with *go-flows*, a JSON specification file in the NTARC file format must be passed. More comprehensive documentation is provided by Godoc[2] and the nta-meta-analysis documentation[3]. The specification file contains the following parameters:

- **features**
  Is a formatted list that contains all of the features to export into a flow. It is possible to apply different functions to given features in order to create new features, collect per-packet values, or combine features and functions.
- **active_timeout**
  Sets the maximum allowed time duration for a collected flow in seconds. When this duration is exceeded, the flow is terminated.
- **idle_timeout**
  If no packets are observed within this time frame in seconds, the active flow expires.
- **bidirectional**
  If true, every collected flow contains packets from both directions.
- **key_features**
  Contains a list of flow-key features that describe a flow. Considering possible timeout values or observation windows, packets matching the flow-key belong to the same flow.

[1] https://github.com/CN-TU/go-flows
[2] https://pkg.go.dev/github.com/CN-TU/go-flows
[3] https://nta-meta-analysis.readthedocs.io/en/latest/

All of the features listed in tables 4.1 and 4.2 were used to classify flow-based and packet-based sampled traffic. For each sampling technique different JSON specification files are used. The main reason for this is how the flow-based, and packet-based sampling techniques are applied.

In a real-world environment, packet-based sampling would be applied directly using a packet capture technique such as port mirroring. This work implements packet-based sampling of capture files by utilizing *Python* and *editcap*. Once packet-based sampling is applied, *go-flows* can obtain the desired features specified in the corresponding configuration file. Figures 5.2 and 5.4 show the JSON specification files used for packet-based sampling.

Flow-based sampling in this work utilizes *go-flows* "accumulate" function to collect per-packet values of all packets grouped within the same flow. Afterwards *Python* is used to apply various sampling modes and, in order to gather similar feature vectors for flow- and packet-based sampling modes, various calculations are applied on the remaining per-packet values after sampling. Figure 5.3 and 5.5 show the used JSON specification files used for flow-based sampling.

Chapters 5.3.1 and 5.3.2 contain a detailed description of both sampling technique implementations.

```
{
  "version": "v2",
  "preprocessing": {
    "flows": [
      {
        "features": [
            "flowStartMilliseconds",

            "sourceIPAddress",
            "destinationIPAddress",
            "sourceTransportPort",
            "destinationTransportPort",
            "protocolIdentifier",

            {"apply": ["packetTotalCount", "forward"]},
            {"apply": ["octetTotalCount", "forward"]},
            {"apply": ["tcpSynTotalCount", "forward"]},
            {"apply": ["tcpAckTotalCount", "forward"]},
            {"apply": ["tcpFinTotalCount", "forward"]},
            {"apply": ["_tcpCwrTotalCount", "forward"]},
            {"apply": [{"min":["ipTotalLength"]}, "forward"]},
            {"apply": [{"mean":["ipTotalLength"]}, "forward"]},
            {"apply": [{"max":["ipTotalLength"]}, "forward"]},
            {"apply": [{"stdev":["ipTotalLength"]}, "forward"]},
            {"apply": [{"min":["_interPacketTimeSeconds"]}, "forward"]},
            {"apply": [{"mean":["_interPacketTimeSeconds"]}, "forward"]},
            {"apply": [{"max":["_interPacketTimeSeconds"]}, "forward"]},
            {"apply": [{"stdev":["_interPacketTimeSeconds"]}, "forward"]},

            {"apply": ["packetTotalCount", "backward"]},
            {"apply": ["octetTotalCount", "backward"]},
            {"apply": ["tcpSynTotalCount", "backward"]},
            {"apply": ["tcpAckTotalCount", "backward"]},
            {"apply": ["tcpFinTotalCount", "backward"]},
            {"apply": ["_tcpCwrTotalCount", "backward"]},
            {"apply": [{"min":["ipTotalLength"]}, "backward"]},
            {"apply": [{"mean":["ipTotalLength"]}, "backward"]},
            {"apply": [{"max":["ipTotalLength"]}, "backward"]},
            {"apply": [{"stdev":["ipTotalLength"]}, "backward"]},
            {"apply": [{"min":["_interPacketTimeSeconds"]}, "backward"]},
            {"apply": [{"mean":["_interPacketTimeSeconds"]}, "backward"]},
            {"apply": [{"max":["_interPacketTimeSeconds"]}, "backward"]},
            {"apply": [{"stdev":["_interPacketTimeSeconds"]}, "backward"]}


        ],
      "active_timeout": 1800,
      "idle_timeout": 300,
      "bidirectional": true,
      "key_features": [
          "sourceIPAddress",
          "destinationIPAddress",
          "protocolIdentifier",
          "sourceTransportPort",
          "destinationTransportPort"
          ]
      }
    ]
  }
}
```

Figure 5.2: CAIA go-flows configuration,
packet-based sampling

```
{
  "version": "v2",
  "preprocessing": {
    "flows": [
      {
        "features": [
            "flowStartMilliseconds",

            "sourceIPAddress",
            "destinationIPAddress",
            "sourceTransportPort",
            "destinationTransportPort",
            "protocolIdentifier",

            {"apply": [{"accumulate":["octetTotalCount"]}, "forward"]},
            {"apply": [{"accumulate":["tcpSynTotalCount"]}, "forward"]},
            {"apply": [{"accumulate":["tcpAckTotalCount"]}, "forward"]},
            {"apply": [{"accumulate":["tcpFinTotalCount"]}, "forward"]},
            {"apply": [{"accumulate":["_tcpCwrTotalCount"]}, "forward"]},
            {"apply": [{"accumulate":["ipTotalLength"]}, "forward"]},
            {"apply": [{"accumulate":["_interPacketTimeSeconds"]}, "forward"]},

            {"apply": [{"accumulate":["octetTotalCount"]}, "backward"]},
            {"apply": [{"accumulate":["tcpSynTotalCount"]}, "backward"]},
            {"apply": [{"accumulate":["tcpAckTotalCount"]}, "backward"]},
            {"apply": [{"accumulate":["tcpFinTotalCount"]}, "backward"]},
            {"apply": [{"accumulate":["_tcpCwrTotalCount"]}, "backward"]},
            {"apply": [{"accumulate":["ipTotalLength"]}, "backward"]},
            {"apply": [{"accumulate":["_interPacketTimeSeconds"]}, "backward"]}

        ],
      "active_timeout": 1800,
      "idle_timeout": 300,
      "bidirectional": true,
      "key_features": [
          "sourceIPAddress",
          "destinationIPAddress",
          "protocolIdentifier",
          "sourceTransportPort",
          "destinationTransportPort"
          ]
        }
      ]
  }
}
```

Figure 5.3: CAIA go-flows configuration,
flow-based sampling

```
{
        "features": [
            "flowStartMilliseconds",
            "sourceIPAddress",

            {"distinct":["destinationIPAddress"]},
            {"mode":["destinationIPAddress"]},
            {"modeCount":["destinationIPAddress"]},

            {"distinct":["sourceTransportPort"]},
            {"mode":["sourceTransportPort"]},
            {"modeCount":["sourceTransportPort"]},

            {"distinct":["destinationTransportPort"]},
            {"mode":["destinationTransportPort"]},
            {"modeCount":["destinationTransportPort"]},

            {"distinct":["protocolIdentifier"]},
            {"mode":["protocolIdentifier"]},
            {"modeCount":["protocolIdentifier"]},

            {"distinct":["ipTTL"]},
            {"mode":["ipTTL"]},
            {"modeCount":["ipTTL"]},

            {"distinct":["_tcpFlags"]},
            {"mode":["_tcpFlags"]},
            {"modeCount":["_tcpFlags"]},

            {"distinct":["octetTotalCount"]},
            {"mode":["octetTotalCount"]},
            {"modeCount":["octetTotalCount"]},

            "packetTotalCount"
        ],
    "active_timeout": 1800,
    "idle_timeout": 300,
    "bidirectional":false,
        "key_features": [
            "sourceIPAddress",
        "__timeWindow10s"
        ]
      }
```

Figure 5.4: AGM go-flows configuration, packet-based sampling

```
{
        "features": [
            "flowStartMilliseconds",
            "sourceIPAddress",

            {"apply": [{"accumulate":["destinationIPAddress"]}, "forward"]},
            {"apply": [{"accumulate":["sourceTransportPort"]}, "forward"]},
            {"apply": [{"accumulate":["destinationTransportPort"]}, "forward"]},
            {"apply": [{"accumulate":["protocolIdentifier"]}, "forward"]},
            {"apply": [{"accumulate":["ipTTL"]}, "forward"]},
            {"apply": [{"accumulate":["_tcpFlags"]}, "forward"]},
            {"apply": [{"accumulate":["octetTotalCount"]}, "forward"]}

        ],
    "active_timeout": 1800,
    "idle_timeout": 300,
    "bidirectional":false,
        "key_features": [
            "sourceIPAddress",
        "__timeWindow10s"
        ]
      }
```

Figure 5.5: AGM go-flows configuration, flow-based sampling

5.1.2 Wireshark Tools

The *Wireshark*[4] packet-analyzer contains a set of useful command-line utilities, which were used in this work to apply packet-based sampling on the capture files from the used *CIC-IDS-2017*[5] dataset.

Editcap

editcap reads packets from an specified *infile*, modifies them in various ways, and returns the modified packets in an *outfile*. The arguments used in this study are briefly described in the following paragraphs to give an idea of how the packet-based sampling with *editcap* works.

```
editcap -s {snaplen} {infile} {outfile}
```

The -s argument truncates each packet to the maximum *snaplen* of bytes of data. This step removes any payloads from packets in the capture files in order to reduce memory usage when processing the packets later.

```
editcap -c {packets} {infile} {outfile}
```

In another stage of the packet sampling logic, *editcap* is used with the -c argument to cut large capture files into several smaller files in order to speed up the sampling process. Each splitfile contains the maximum number of *packets*.

[4] https://www.wireshark.org/

[5] https://www.unb.ca/cic/datasets/ids-2017.html

```
editcap {infile} {outfile} [{packet #} [ - {packet #}] ...]
```

The most basic *editcap* feature accepts an *infile* and a list of packets to drop, and outputs the sampled captured *outfile*.

editcap provides useful packet manipulation capabilities, allowing for packet-based sampling on capture files without the need to implement a PCAP reader in *Python* with *Scapy* or similar programs.

The drawback of this simple and straightforward packet sampling implementation is a major *editcap* limitation. A maximum of 512 packets can be dropped in a single *editcap* execution, which creates another issue when working with large capture files. When *editcap* is executed, the tool reads the entire passed *infile*, drops the specified list of up to 512 packets, and writes the entire *outfile* containing the remaining packets. On files larger than a few GB, it is very likely that several hundred read/writes of the capture file must be processed, which may take a considerable amount of time significantly slowing down experiment executions.

To alleviate some of this frustration, splitting the files into smaller chunks greatly speeds up the entire operation. This implementation is only possible because the performance of the sampling process has no effect on the study's results. In practice, almost any case of packet-based sampling will be performed directly on the network interface of an observation point, with little effect on performance-related parameters or requirements to sample the packets.

Mergecap

mergecap can be used to concatenate or merge capture files. Because of the way packet-based sampling is implemented, all split files must be merged into a single file for further processing after the sampling has been applied.

```
mergecap -F pcap {infiles ...} -w {outfile}
```

The -F argument specifies the *outfile* file format as capture file, while the *infile* argument explicitly states the path to a folder containing all splitted files using a placeholder.

Capinfos

capinfos can display a variety of information about capture files. The tool is used to determine the total number of packets in the passed *infile* in order to correctly apply packet-based sampling before using *go-flows* to collect flows.

```
capinfos -M -c {infile} | grep packets
```

The -c argument prints the number of packets contained in the given *infile*, while the -M argument prints any numeric values machine readable. *grep* is used to only output the actual packet number, allowing to save the actual string of numbers into a *Python* variable and then process this string as a numerical value.

5.1.3 Dstat

dstat[6] is a very useful tool for generating statistics for Linux machine resources. It is possible to export logged statistics as *outfile* for further processing and analysis. In this work, *dstat* is used to track system resource usage when performing Machine Learning-based classification in order to compare various sampling techniques and patterns that take not only classification accuracy ratings but also the resources required into account.

```
dstat --epoch --cpu-adv --disk --mem-adv --swap --output {outfile}
```

This is the exact command used in this work to save various statistics for later analysis. These statistics include processor utilization, memory and swap usage. Epoch timestamps are saved to differentiate the segments of the classification process.

5.1.4 Rsync

For automated experiment execution, rsync[7] is used to transfer experiment-related files between the local machine and the remote device via SSH.

```
rsync -avz --progress {source} {user}@{host}:{destination}
```

The -avz argument enables archive mode (recursive, copies symlinks, preserves permissions, preserves modification times, preserves group, preserves owner, preserves special files and device files), increases verbosity and compresses file data during the transfer. The --progress argument outputs the progress during transfer.

5.1.5 Labeling

The labeling script[8] for the *CIC-IDS-2017* dataset, published by the Institute of Telecommunications' Communication Network Group at the Technical University Vienna, is used in this work.

```
python3 Labeling.py {infile} {mode}
```

This script takes an *infile* and adds two new features called "Attack" and "Label". The script is designed for general usage and supports various modes. However, depending on the desired feature representations flow-key, this study only uses "5tuple" and "AGM" (1tuple) as *mode* argument. The script was slightly modified to remove the feature "flowStartMilliseconds" before the script exits in order to save memory when loading the labelled data later for preprocessing and classification.

```
print('>> Dropping feature flowStartMilliseconds')
data.drop(axis=1, labels='flowStartMilliseconds', inplace=True)
```

Listing 1: Labeling script modification

[6] https://linux.die.net/man/1/dstat
[7] https://linux.die.net/man/1/rsync
[8] https://github.com/CN-TU/Datasets-preprocessing/tree/master/CIC-IDS-2017/labeling

5.2 Low-End Hardware

The purpose of this thesis is to look into the effects of various sampling techniques and patterns on resource usage and execution efficiency when using low-end hardware for Machine Learning-based anomaly detection. One reason for this research question is that there are few resources on the subject, and knowledge about low-end hardware requirements in published papers about Machine Learning-based anomaly detection is scarce.

In order to collect meaningful data, the experiments were first run on a Raspberry Pi Zero W[9], a computer with very low performance when compared to modern machines. With its specifications, this system seems ideal for this research. The following table lists the requirements of the low-end devices:

Table 5.1: Remote device hardware specifications

	CPU	RAM	Disk	OS
Raspberry Pi Zero W Rev 1.1	Broadcom BCM2835 ARM11 Single Core 1GHz armv6l	512MB LP-DDR2	8GB microSD	DietPi 7.0.2
LXC Container	Intel Atom D525 Single Core 1.80GHz x86	1024MB DDR3 800 MT/s	10GB SSD	Debian buster 10.9

The Raspbery Pi Zero W runs a headless installation of *DietPi*[10], a *Debian*-based Linux distribution designed for single-board computers. *DietPi* has the advantage of being extremely lightweight and highly optimized for limited CPU and RAM resource consumption.

Since this operating system is *Debian*-based, all packets required for this research can be easily installed on *DietPi* in the same way as the server or desktop machine might be set up. In comparison to other lightweight operating systems considered for this study, there is no need to look for substitutions for certain packets while retaining a very low base-resource consumption. This suits well enough to track resource-usage with *dstat* when performing classifications.

During this research, it was discovered that Random Forest models created on a x86_64 machine could not be imported onto the armv6l-based Raspberry Pi Zero W. The first assumption would be a 32bit/64bit incompatibility, but after configuring a 32bit *Debian* server to fit the model, this proved not to be the source of the problem. Since both systems use the same byte order this can also be excluded as possible root of this issue. Be careful when trying to fit and import models on different architectures, you may face such an issue.

[9] https://www.raspberrypi.org/products/raspberry-pi-zero-w/
[10] https://dietpi.com/

Because this incompability could not be resolved a LXC container was configured on Proxmox VE[11] to gather the results for this study. With this virtual machine the import of an already fitted model can be done without any trouble. A detailed description of the hardware specifications and the installed packets necessary for this ressarch can be found in chapter A.1 for both machines.

[11] https://www.proxmox.com/en/

5.3 Framework

The upcoming subsections provide flowcharts and descriptions for all modules used in this study, with most modules providing basic functionalities like the application of various sampling methods, preprocessing or classification.

The idea is to sample a given network traffic capture using a specified sampling technique based on the framework configuration. As described in chapter 3.2, these sampling techniques can be flow-based or packet-based, with both supporting multiple sampling modes. Following sampling, the preprocessing module loads the sampled data and either trains the Random Forest classifier to generate a model when run on mid-range hardware, or loads the already fitted model to generate predictions for later evaluation when run on low-end hardware. Multiple experiment configurations can be specified in the framework configuration file. All experiments can be carried out automatically, with all necessary actions taken to obtain the results.

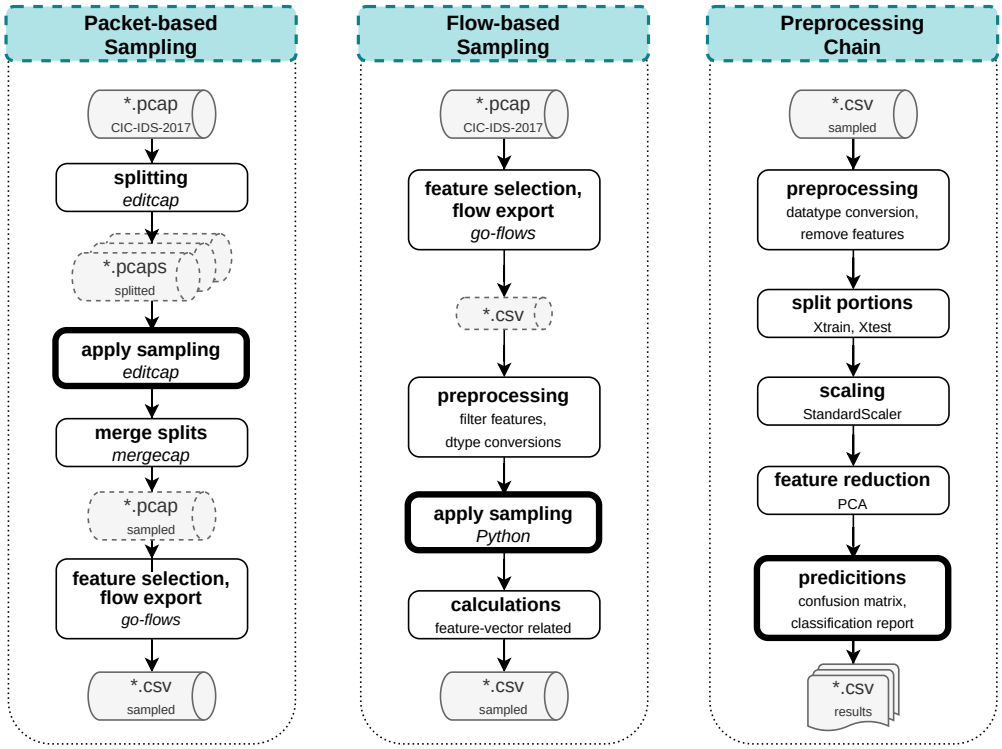

Figure 5.6: Superficial framework module overview, part I

Figure 5.6 depicts a high-level overview of the major framework modules required to generate the results. The overview illustrates the primary distinction between flow-based and packet-based sampling implementations. The packet-based implementation first samples the original capture files with *editcap* before extracting features and exporting flows with *go-flows*. The flow-based implementation extracts features and exports flows with *go-flows* in the first step of the processing chain, then samples the per-packet feature

values accumulated with *go-flows*.

Before classification results can be gathered, the preprocessing chain must complete several major steps. First, the imported, already sampled *.csv file is preprocessed, which includes datatype conversions and the removal of features that are not required by the Random Forest classifier. The data is then divided into two parts: a training portion used to train the classifier algorithm and a test portion used to generate the results. The scikit-learn StandardScaler is used for scaling, and Principal Component Analysis is used for further feature reduction, as described in chapter 4.4.2. As a final step, the Random Forest classifier is used to classify the test portion of the data, producing various results such as the confusion matrix and a classification report. The chapter 6.4 lists all of the results that were generated and exported into various files.

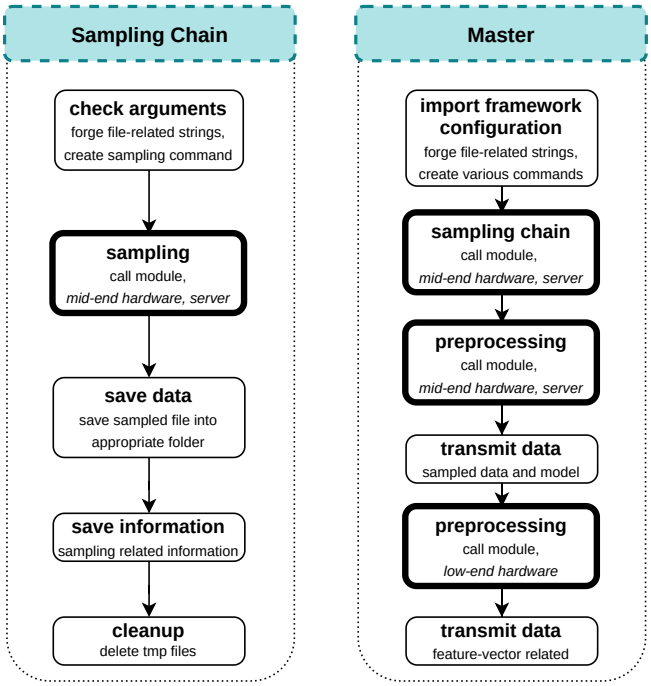

Figure 5.7: Superficial framework module overview, part II

Figure 5.7 illustrates the framework's remaining modules. Both are used to automate the execution of experiments based on the framework configuration. The framework configuration is imported by the master module, which iterates through each specified experiment. The module performs all necessary steps in this process, from sampling the raw data and fitting the classifier model on mid-range hardware to executing the preprocessing chain on low-end hardware to generate results and data for later evaluation. The superficial depicited sampling chain basically iterates through all workday capture files from the *CIC-IDS-2017* dataset, selects the correct sampling module, and then merges all sampled data into a single file for further processing. Each module will be described in detail in the sections that follow.

5.3.1 Packet-Based Sampling Logic

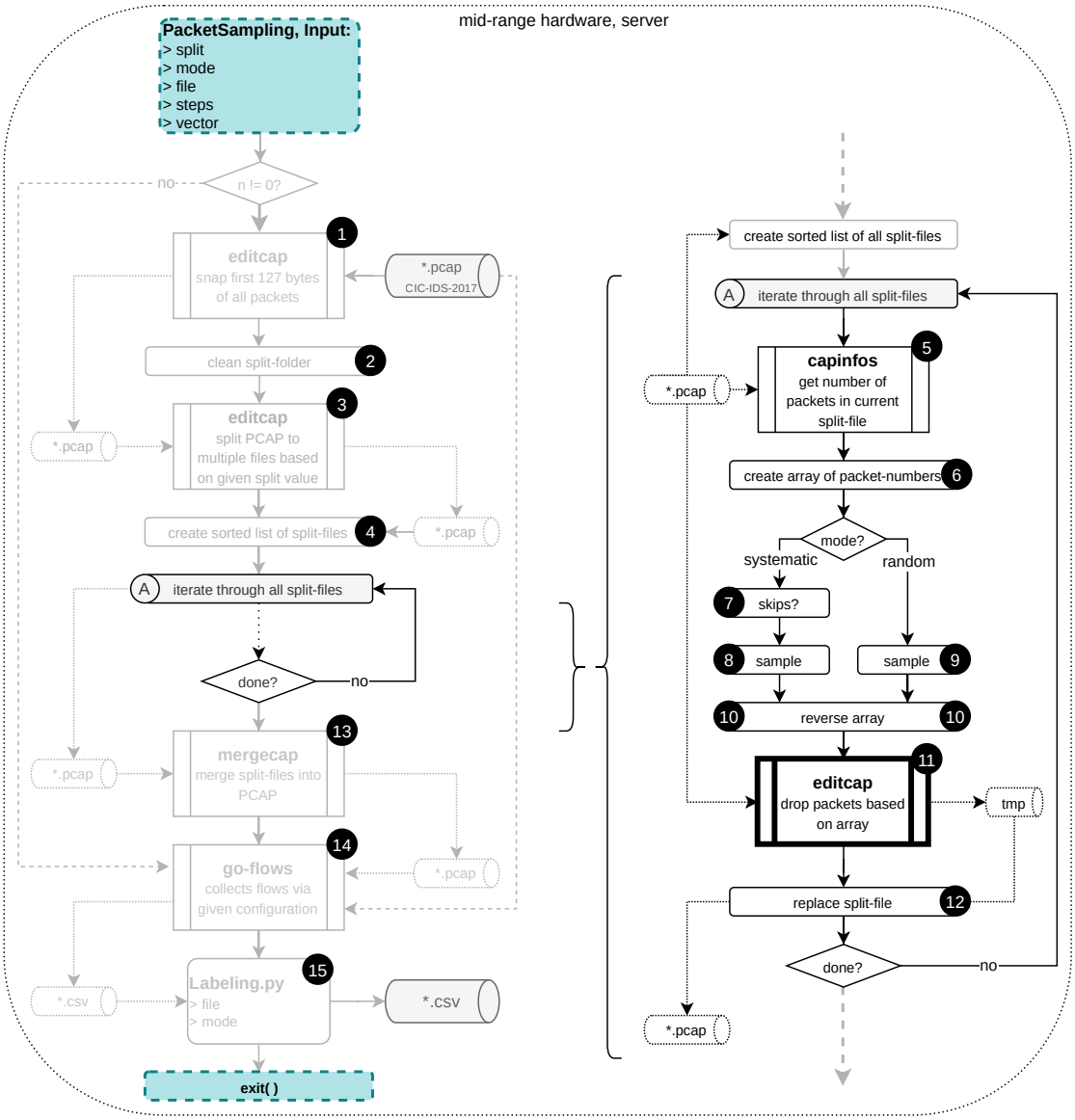

Figure 5.8: Packet-based sampling diagram

To implement packet-sampling methods this module utilizes a selection of tools included with *wireshark* to apply the sampling, followed by *go-flows* collecting the flows based on the remaining sampled packets. Figure 5.8 shows the diagram for this module, the enumeration used in the following describe the corresponding section shown in the diagram.

1. Initially *editcap* is used to remove the payload. This is accomplished by removing all data beyond the 127th byte from each packet in the given capture file. The

purpose of this step is to save disk space and to speed up the entire sampling process by taking save and load times into account, even though these times are not considered crucial for the results of this study. The resulting file is stored in the associated folder.

```
254 editsnapcmd = 'editcap -s 127 {} {}'.format(pcap,pcap_snap)
...
293 print('>>> Dropping payload: {}'.format(editsnapcmd))
294 os.system(editsnapcmd)
```

Listing 2: Remove payload

2 If any files from previous executions remain in the split-folder, they must be removed in preparation for the next stage.

```
251 cleansplitPCAP = 'rm {}'.format(cfg.fpath/'splitPCAP'/'*')
...
297 print('>>> Cleaning folder: {}'.format(cleansplitPCAP))
298 os.system(cleansplitPCAP)
```

Listing 3: Clean split-folder

3 Following the cleanup, *editcap* is used to split the capture file into several smaller ones, each of which contains the number of packets defined in the framework configuration variable *split*[12].

```
252 editsplitcmd = 'editcap -c {} {} {}'.format(split,pcap_snap,pcap_split)
...
301 print('>>> Splitting PCAP: {}'.format(editsplitcmd))
302 os.system(editsplitcmd)
```

Listing 4: Split capture file

The purpose of this step is to minimize reading and writing times when dropping packets with editcap, which has a major limitation of only allowing 512 packets to be dropped in a single execution. When packets are dropped in an *editcap* execution, the input file has to be read before *editcap* drops the packets and writes the output file. Splitting the large capture file into many smaller files significantly reduces the time required for the entire packet-based sampling process. The default size of a split contains 5000 packets per file. This makes it necessary to determine eventual remainder packets for the following split-file in order to apply systematic sampling modes in a correct way as described in item 7 and item 8.

4 Once all split files have been created, the module generates a sorted list based on the content of the split folder. Pyhons *os.listdir()* command is used to obtain the list of split files, manual sorting is applied afterwards. The manual sorting is necessary to maintain the correct packet arrival order because it is not guaranteed that *os.listdir()* creates an ordered list of filenames.

```
306 splitlist = os.listdir(split_folder)
307 splitlist.sort()
```

Listing 5: Create ordered list of split-files

Based on this list, the module iterates through each split file, performing various operations to apply the packet-based sampling.

[12] *split* default value is 5000 packets per split-file

5. *capinfos* is used to calculate the total number of packets in the current iteration split file.

```
capcmd = r'capinfos -M -c {} | grep packets'.format(infosplit)
pcount = subprocess.check_output(capcmd, shell=True, universal_newlines=True)
```

Listing 6: Obtain number of packets in current split-file

6. This number is used to generate a numpy-array containing all packet numbers of the split-file in the current iteration.

```
# array containing packet-numbers of the current split-file
plist = np.arange(1,pcount+1,1) # used to drop packets with editcap
# same array, containing packet-indices
plistindex = np.arange(0,pcount,1)
```

Listing 7: Create array of packet numbers

7. If the systematic sampling mode (every n-th packet) is selected, a modulo division is used to determine the number of eventual packets to skip for the upcoming split-file iteration.

```
if mode == 5:
    modulo = samplepcount % n

    if modulo != 0:
        # number of packets to skip in next iteration
        nextpacketskip = n - modulo
        nextsamplepstart = nextpacketskip
    else:
        nextpacketskip = 0
        nextsamplepstart = 0
```

Listing 8: Determine remainder for next iteration

8. For systematic sampling (every n-th packet), packets to skip are already considered and the sampling is applied to the numpy-array.

```
# index-numbers of packets to sample in current iteration
psample = plistindex[packetskip::n]
# packets to drop in current iteration via editcap
pdrop = np.delete(plist,psample.tolist())
```

Listing 9: Apply systematic sampling on array

The sampled packets are removed from the numpy array that is used to drop packets with *editcap* in the upcoming step.

9. For random sampling a generator object is created with a seed[13] value in order to achieve reproducable draws.

```
s = cfg.seed # use seed number from configuration

# create generator object with seed s
rng = np.random.default_rng(s)
```

Listing 10: Initialize random generator object

This generator object draws samples from the current split-file based on the given value of n, and creates an array to drop packets with *editcap* in the upcoming step.

[13] *seed* can be set in the framework configuration, default is 1000

```
532 if mode == 7:
533     packets = math.ceil(samplepcount/n)
534     # draw n packets out of the whole file
535     sample = rng.choice(plist,size=packets,replace=False)
536     sample = np.sort(sample)
537     pdrop = plist.copy() # list of packets to drop
538     for value in sample:
539         pdrop = np.delete(pdrop,np.where(pdrop==value))
```

Listing 11: Apply random sampling on array

10 The implementation of *editcap* starts dropping the last packets of the split-file and moves in slices of 512 packets towards the beginning of the file. For this reason, the list of packets to drop needs to be reversed before starting this process. If the implementation would start dropping the first packets the number of packets would be invalidated after the first and every following *editcap* execution. After each *editcap* execution, steps would need to be taken to correct the packetnumbers; this is avoided when the list of packets to drop is reversed.

```
pdrop = np.flip(np.sort(pdrop))
iteration = int(len(pdrop)/512)+1
for i in range(0,iteration):
    # create a slice of 512 packets to remove with editcaps
    pslice = pdrop[0:512]
    # remove these 512 packets from droplist for next iteration
    pdrop = pdrop[512:]
```

Listing 12: Prepare array of packets to drop

This already takes into account the *editcap* constraint of a maximum of 512 packets dropped in a single editcap execution.

11 The argument suitable for be used with *editcap* is forged based on the array.

```
# create string containing packet numbers to drop
arg = [str(int) for int in pslice]
# seperated with whitespaces necessary as editcap argument
arg = " ".join(arg)
```

Listing 13: *editcap* argument generation

With this argument, *editcap* is executed to drop the specified packets. These steps are done the same way for systematic and random sampling.

```
tmpsplitfile = split_folder / file # current iteration split-file
tmpfile = split_folder / 'tmp.pcap' # temporary file created with editcap
editcapcmd = 'editcap {} {} {}'.format(tmpsplitfile,tmpfile,arg)
os.system(editcapcmd)
```

Listing 14: *editcap* execution

12 Once the file is sampled and temporaryily stored, the original split file gets replaced at the end of each iteration.

```
movecmd = r'mv {} {} > NUL'.format(tmpfile,tmpsplitfile)
os.system(movecmd)
```

Listing 15: File replacement

13 In preparation for using *go-flows* it is important to utilize *mergecap* to merge all sampled files after they are processed.

```
588 print('>>> Merging split-files: {}'.format(mergecapcmd))
589 os.system(mergecapcmd)
```

Listing 16: Merge split-files

14 *go-flows* is used to collect flows based on the passed configuration file.

```
605 print('>>> Collect flows with go-flows: {}'.format(csv_sampled_export))
606 os.system(goflowscmd)
```

Listing 17: Flow collection

15 As a final step, the labeling script is used to set the ground truth labels, which are required later on to apply the Random Forest classifier.

```
249 labelingcmd = 'python3 {} {} {}'.format(cfg.labelingpath,cfg.packetfolder/cfg.
        filenames[findex],labelmode)
    ...
212 # set mode for later labeling.py
213 if j<cfg.packetlimit or j>5: labelmode = 'AGM'
214 elif j >= cfg.packetlimit: labelmode = '5tuple'
    ...
754 print('>>> Labeling: {}'.format(labelingcmd))
755 os.system(labelingcmd)
```

Listing 18: Initialize random generator object

Optional Arguments

-c, --check
The -c argument can be used to verify the applied packet-based sampling. It is a simple verification that compares the calculated number of packets remaining from the original file to the number of packets currently present in the sampled file. If the verification fails, this optional argument only returns informational output and does not exit the script.

-v, --verbose
The -v argument outputs additional verbose information.

--superverbose
The --superverbose argument outputs the same information as the verbose argument, additionally, loop iteration output for split file sampling is shown.

-t, --time
The -t argument starts a timer on script start, measures runtimes and saves the timestamp into a file when executed automatically via automated experiment execution as described in chapter 5.3.5.

-s, --seed
The -s argument can be set to use a fixed seed number when using numpy's random number routine to produce pseudo random numbers. This can be useful for debugging or simply to create predicatale, comparable results.

5.3.2 Flow-Based Sampling Logic

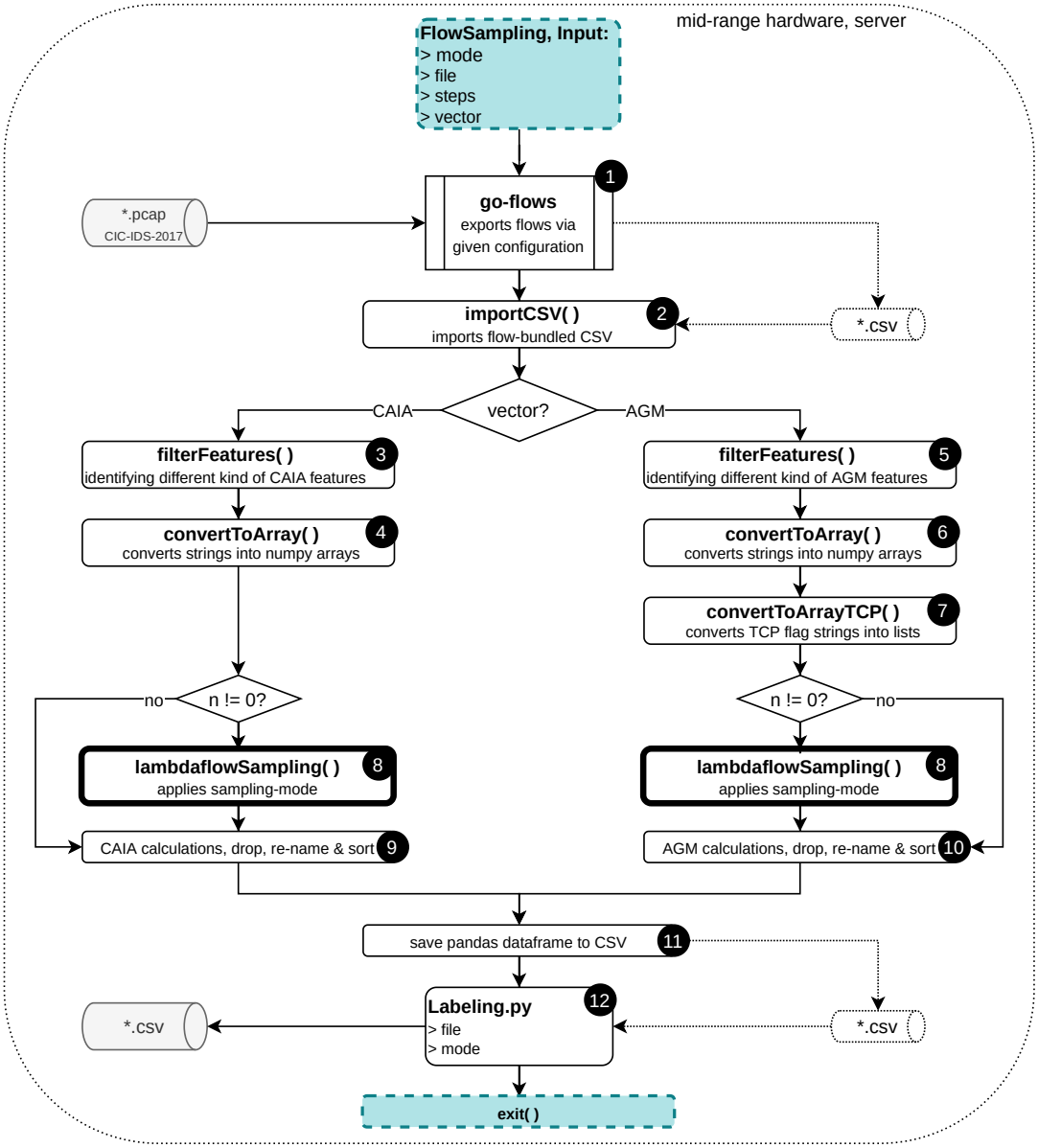

Figure 5.9: Flow-based sampling diagram

This module applies specific sampling patterns on beforehand created flows collected with the flow exporter *go-flows*. In a first step *go-flows* is executed, given a network capture file as input, to generate a specified feature-vector and create an output file that contains the accumulated values from every packet related to a flow. Afterwards the output file created with *go-flows* is imported and sampling is applied on all accumulated values for every feature in every flow. Based on the sampled values all necessary calculations are

applied to finalize the selected feature-vector.

1. The first step in flow-based sampling is to start *go-flows* with the required JSON specification and an infile. The configuration file used allows the accumulation of values for each packet connected to a flow, based on the flow-key defined in the configuration, and saves the collected flows in the form of a CSV file.

```
525 print('\n\n>>> Collect flows with go-flows from {}'.format(pcap))
526 os.system(goflowscmd) # execute go-flows to process passed PCAP file
```

Listing 19: *go-flows* execution

2. Once the flows have been collected and the data has been saved, the module imports the CSV as a pandas dataframe.

```
62 def importCSV(csvpath,csvusecols=None,verbose=False,encoding='utf-8'):
63
64     if verbose: print('\n\n'+40*'~'+' FUNCTION: importCSV '+40*'~')
65     print('\n>>> Importing {}'.format(csvpath))
66     csvdata = read_csv(csvpath,usecols=csvusecols,skipinitialspace=True,encoding=
       encoding)
67     return csvdata
...
527 dataset = importCSV(csv_import,None,verbose)
```

Listing 20: Import collected flows

After this step, depending on the passed kind of feature vector, the module processes the imported flows according to either CAIA or AGM specifications.

3. If the CAIA vector is selected, a function is called to filter CAIA features collected via "accumulate"[14] in the passed *go-flows* in the JSON specification as illustrated in figure 5.3. The *filterFeatures* function checks each feature-label of the imported pandas dataframe. If a label starts with the specified keyword it can be identified as an "accumulate" feature, enabling the sampling of its per-packet values at a later point.

```
90 def filterFeatures(dataset,keyword,verbose=False):
91     features = dataset.columns
92     tmp = []
93
94     for feature in features:
95         if keyword in feature:
96             tmp.append(feature)
97             print('\t+ {}'.format(feature))
98         else: print('\t- {}'.format(feature))
99     return tmp
...
536 keyword = 'apply(accumulate'
537 print('>>> Identifying accumulated features')
538 features = filterFeatures(dataset,keyword,verbose)
```

Listing 21: Identify CAIA specific features

In a similar way other CAIA specific features that need specific processing are identified.

[14] the accumulate function in *go-flows* returns per-packet values as list for each collected flow

```
keyword = 'TotalCount'
print('>>> Identifying features containing: {}'.format(keyword))
totalFeatures = filterFeatures(dataset,keyword,verbose)

keyword = 'ipTotal'
print('>>> Identifying features containing: {}'.format(keyword))
ipTotal = filterFeatures(dataset,keyword,verbose)

keyword = 'interPacket'
print('>>> Identifying features containing: {}'.format(keyword))
interPacket = filterFeatures(dataset,keyword,verbose)
```

Listing 22: Feature identification

4 In order to apply flow-based sampling on all the filtered "accumulate" features, the string that is created with go-flows has to be converted into a useable numpy-array.

```
def convertToArray(dataset,features,mode,verbose=False):
    for feature in features: # iterate over given features
        print('\t> {}'.format(feature))

        if mode == 1: # creates numpy array based on numeric feature
            dataset[feature] = dataset[feature].apply(lambda x:
                np.fromstring(x[1:len(x)-1],dtype=int, sep=" ") if type(x) == str
                else (np.array([float('nan')]) if pd.isna(x)
                else x))

        if mode == 2: # creates list based on textual feature
            dataset[feature] = dataset[feature].apply(lambda x: x[1:len(x)-1].
    split())

    return
...
print('>>> Converting accumulated values')
convertToArray(dataset,features,1,verbose)
```

Listing 23: String conversion

The lambda function converts strings into useable numpy array, replacing the original string with a one-dimensional array that can be used to apply sampling in the following step.

5 If the AGM vector is selected, again a function is called to filter AGM features based on pre-defined keyword strings. First all "accumulate" per-packet features are identified, then the TCP flag feature that needs special treatment before sampling.

```
print('>>> Identifying accumulated features')
keyword = 'apply(accumulate'; features=filterFeatures(dataset,keyword,verbose)

print('>>> Identifying textual feature')
keyword = '_tcp'; textual=filterFeatures(dataset,keyword,verbose)
keyword = 'destinationIP'; textual+=filterFeatures(dataset,keyword,verbose)
```

Listing 24: Identify AGM specific features

6 Similar to the CAIA feature handling, all filtered "accummulate" features are converted are converted into useable numpy array for the upcoming flow-based sampling.

```
553 print('>>> Converting accumulated values')
554 convertToArray(dataset,features,1,verbose)
```

Listing 25: String conversion

7 The TCP flag feature gets special treatment, calling a function that considers possible multi-flag combinations, converting the string into a useable list.

```
746 print('>>> Converting _tcpFlags feature')
747 convertToArrayTCP(dataset,'apply(accumulate(_tcpFlags),forward)',2,verbose,
    superverbose)
```

Listing 26: TCP flags conversion

8 If sampling has to be applied the "lambdaflowSampling()" function is used to sample all "accumulate" features. A sampling mode is chosen based on the passed function argument to process all specified features.

```
if n != 0:
    print('>>> Applying flow-based sampling')
    lambdaflowSampling(dataset,n,features,mode,verbose,time)
```

Listing 27: Apply flow-based sampling

9 For all CAIA features identified in step three, various caluculations are applied to create all features necessary to fullfill the requirements for the chosen vector.

```
568 print('\t>> Calculate packetTotalCount features')
569         key = ['forward','backward']
570         for word in key:
571             newfeature = '{},{}'.format('packetTotalCount',word)
572             print('\t\t> {}'.format(newfeature))
573             dataset.insert(6,newfeature,0)
574             # manually set feature to gather packet counts from
575             if word   == 'forward':    tmp = 'apply(accumulate(octetTotalCount),
    forward)'
576             elif word == 'backward':   tmp = 'apply(accumulate(octetTotalCount),
    backward)'
577             dataset[newfeature] = dataset[tmp].apply(lambda x: 0 if np.isnan(x).
    all() else len(x))
...
580 print('\t>> Calculate min, mean, max and stdev features')
581 key = ['min','mean','max','stdev'] # parameters to calculate
582 for feature in (ipTotal+interPacket):
583     for word in key:
584         newfeature = '{}: {}'.format(feature,word)
585         print('\t\t> {}'.format(newfeature))
586         dataset.insert(len(dataset.columns),newfeature,float(0)) # initialise new
     features after last column
587
588         # apply calculations on new features
589         if   word == 'min':   dataset[newfeature] = dataset[feature].apply(lambda
     x: None if np.isnan(x).any() else int(np.amin(x)))
590         elif word == 'mean':  dataset[newfeature] = dataset[feature].apply(lambda
     x: None if np.isnan(x).any() else sum(x)/len(x))
591         elif word == 'max':   dataset[newfeature] = dataset[feature].apply(lambda
     x: None if np.isnan(x).any() else int(np.amax(x)))
```

Listing 28: CAIA calcualtions example

As already described in chapter 4.3.1, the standard deviation for the CAIA feature vector is calculated according to Welford.

```
592         elif word == 'stdev': # calculating variance according to Welford
593             for i in range (0,len(dataset.index)):
594                 stdev = None
595                 cell = dataset[feature][i] # current cell
596 
597                 if np.isnan(cell).any(): dataset.at[i,newfeature] = stdev
598                 elif len(cell) == 1:     dataset.at[i,newfeature] = 0
599                 else:
600                     # initialize variables
601                     N  = 0
602                     m  = 0
603                     m2 = 0
604 
605                     for val in cell:
606                         N +=1
607                         delta = val - m
608                         m = m + delta/N
609                         delta2 = val - m
610                         m2 = m2 +delta*delta2
611                     stdev = math.sqrt(m2/(N-1))
612 
613                     dataset.at[i,newfeature] = stdev
```

Listing 29: Welford algorithm

Following that, features which are not used in classification are removed from the dataframe. Not required but cosmetic steps are taken, such as renaming the features for improved readability and arranging the remaining features in a pre-ordered manner.

10 AGM specific calculations are applied to all features identified in step five. The TCP flag feature gets special treatment again. A function is called to obtain the number of unique flag combinations (*distinct*), the most often occuring flag or flag combination (*mode*) and the number of packets corresponding to the statistical mode (*modeCount*).

```
208 def tcpflagEncoderTCP(dataset,feature,verbose=False,superverbose=False):
209     if verbose: print(cfg.vcolor+'\n'+40*'~'+' FUNCTION: tcpflagEncoder '+40*'~')
210     print(cfg.vcolor+'\nPre-Encoding: \n{}'.format(dataset[feature]))
211 
212     # features to calculate
213     distinctfeature = '{}: {}'.format(feature,'distinct')
214     modeCountfeature = '{}: {}'.format(feature,'modeCount')
215 
216     # initialize new features with 0
217     dataset.insert(len(dataset.columns),distinctfeature,0)
218     dataset.insert(len(dataset.columns),modeCountfeature,0)
219 
220     # initialise flag features with 0 (as first column)
221     flags = ['A','P','F','R','S','U','E','C','N']
222     for flag in flags: dataset.insert(0,flag,0)
...
763 print('>>> Encoding TCP flags as one feature per flag')
764 tcpflagEncoderTCP(dataset,'apply(accumulate(_tcpFlags),forward)',verbose,
        superverbose)
```

Listing 30: AGM encode TCP flags, initialize flag features, excerpt

It is noteworthy that *go-flows* TCP mode feature is obtained in a slightly different way. Instead of picking multiple flags as mode if they occur the same number of times within a flow, *go-flows* just picks the first encountered flag in such cases as mode.
Following this, features for the total packet count, distinct, mode and modeCount features are calculated for all numerical features. Afterwards, the same is done for the remaining textual feature, *destination network address*. When all calculations are finished, features that are not contained in the AGM feature-vector are removed. Similar to the CAIA vector, not required but cosmetic steps are taken, such as renaming the features for improved readability and arranging the remaining features in a pre-ordered manner.

11 After all necessary operations on the dataframe are done, the script saves the resulting sampled data as a CSV file for later processing.

```
901 print('>>> Saving {}'.format(csv_sampled_export))
902 dataset.to_csv(csv_sampled_export, index=False)
```

Listing 31: Save sampled, pre-processed data

12 In one last step before this module exits, the labeling script is executed to set the ground truth labels which are needed later on to train the Random Forest classifier.

Optional Arguments

-v, --verbose
The -v argument outputs additional verbose information.

--superverbose
The --superverbose argument outputs the same information as the verbose argument, additionally, loop iteration output is shown.

--debug
The --debug argument outputs additional debugging information for NaN handling. It outputs features containing NaN values before and after NaN handling is applied to check the outcome.

-t, --time
The -t argument starts a timer on script start, measures runtimes and saves the timestamp into a file when executed automatically via automated experiment execution as described in chapter 5.3.5.

5.3.3 Sampling Chain

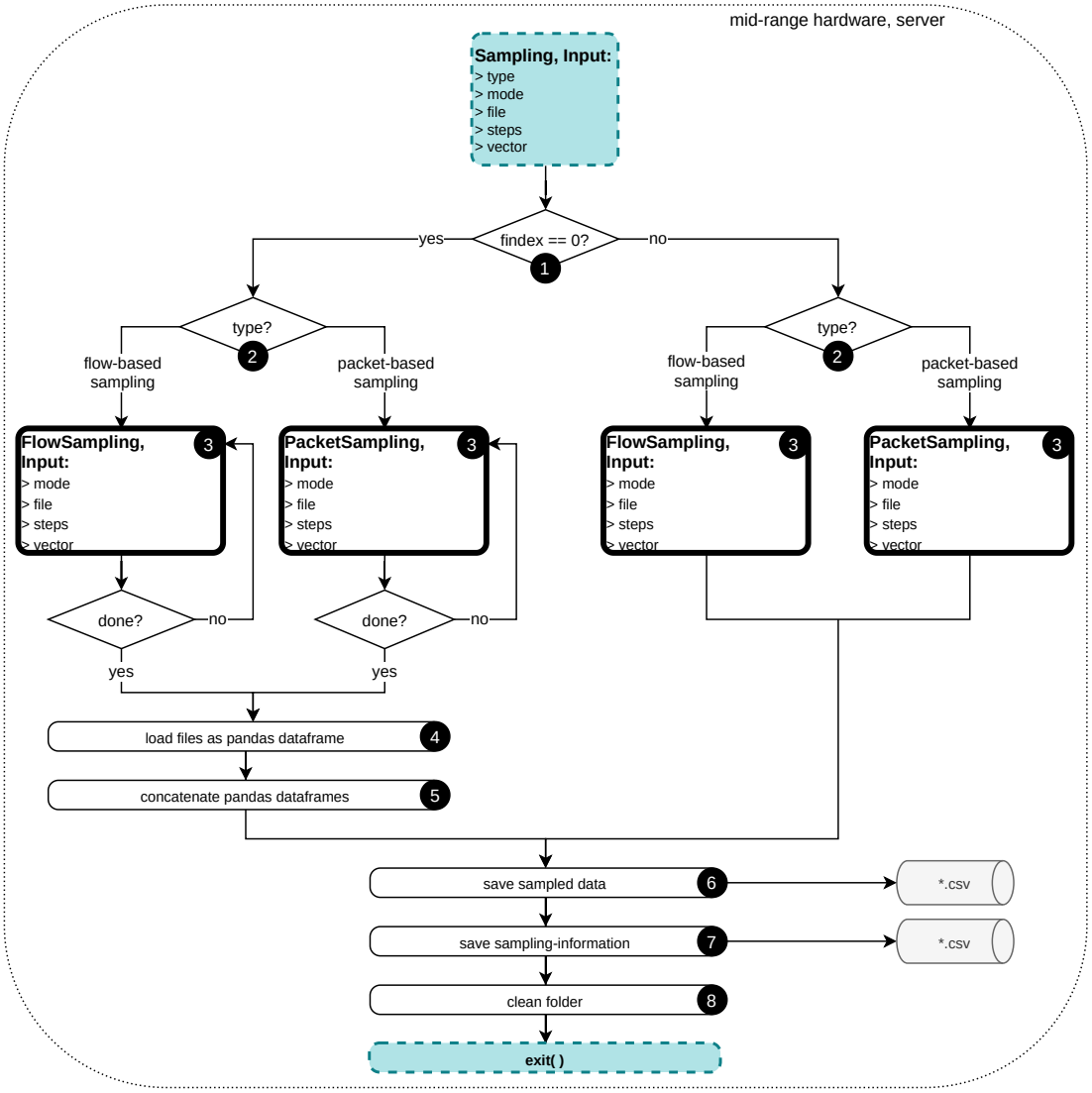

Figure 5.10: Sampling Chain Diagram

This module is wrapped around the modules that apply flow- or packet-based sampling modes. The purpose of this module is to enable automated processing of multiple capture files since the *CIC-IDS-2017* dataset used in this work provides captured network traffic from multiple workdays in the form of one capture file per day.

1. Based on the passed arguments this module either samples one particular workday file from the *CIC-IDS-2017* dataset or it samples every file from the dataset and merges the sampled resulting data into a single file for further processing.
2. The command to execute the correct python module with all necessary arguments for the selected sampling-mode is created, based on all passed arguments.

```
207 if flowsampling:
208     samplearg   = '{} {} {} {}'.format(m,fcount,n,j)
209     samplingcmd = 'python3 rpi-FlowSampling.py {} {} {}'.format(verbosearg,
        timearg,samplearg)
210
211 elif packetsampling:
212     samplearg   = '{} {} {} {} {}'.format(split,m,fcount,n,j)
213     samplingcmd = 'python3 rpi-PacketSampling.py {} {} {}'.format(verbosearg,
        timearg,samplearg)
```

Listing 32: Forge sampling command

3. The appropriate module is executed, applying the selected sampling-mode on the file(s) depending on the passed argument "file"[15].

```
216 print('\n>>> Execute sampling: {}\n\t> {}\n\t> {}\n\t> {}\n\t> n={}'.format(
        samplingcmd,cfg.filenames[fcount],cfg.vectors[j],samplingmode,n))
217 os.system(samplingcmd) # start sampling
```

Listing 33: Execute sampling command

4. When all workday files are processed, the module obtains all files that need to be merged. The variable "pattern"[16] is used to gather all files that match a specific expression, defined in the framework configuration.

```
219 print('\n>>> Merging sampled data')
220 os.chdir(samplingfolder) # change directory for glob usage
221 # save labelled files matching pattern into list
222 matchedfiles = [i for i in glob.glob(cfg.pattern)]
```

Listing 34: Obtain all files to merge

5. This list of files is then used to load each file as pandas dataframe, concatenating the dataframes as they are loaded.

```
223 # concat all labelled files into single CSV
224 singlecsv = pd.concat([pd.read_csv(f) for f in matchedfiles])
```

Listing 35: Conatenate sampled files into one dataframe

6. After the sampling process is completed, the resulting file is stored for later usage. Depending on the passed "file" argument, either a single sampled file is transferred into its appropriate folder:

```
164 movecmd = 'mv {} {}'.format(csv_tmp,csv_save) # moves sampled CSV into folder
...
244 print('>>> Saving file {}'.format(csv_save))
245 os.system(movecmd)
```

Listing 36: Move sampled workday file into folder

Or the merged file is stored by saving the pandas dataframe:

```
226 print('>>> Saving file {}'.format(csv_save))
227 singlecsv.to_csv(csv_save, index = False,encoding='utf-8-sig')
```

Listing 37: Conatenate sampled files into one dataframe

[15] *file* is one positional argument for the executed modules

[16] *pattern* per default is set to '*Hours.csv'

7. Various information about the applied sampling is stored within its appropriate folder in information.csv. This information is later used for the experiment analysis.

```
121 info = {
122     'file':             [cfg.filenames[findex]],
123     'per-flow sampling':[flowsampling],
124     'samplingmode':     [cfg.fsamplingmode[m]],
125     'samplingsteps':    [n],
126     'featurevector':    [cfg.vectors[j]]
    ...
136 info = {
137     'file':             [cfg.filenames[findex]],
138     'packet sampling':  [packetsampling],
139     'samplingmode':     [cfg.psamplingmode[m]],
140     'samplingsteps':    [n],
141     'featurevector':    [cfg.vectors[j]]
142         }
143
144 # df for saving sampling-informations to csv
145 info = pd.DataFrame.from_dict(info,orient='index')
    ...
247 print('>>> Saving information {}'.format(csv_info))
248 info.to_csv(csv_info)
```

Listing 38: Various sampling information

8. The module then cleans the sampling directory as its final stage.

```
252 print('>>> cleanup')
253 for file in Path(samplingfolder).glob('*.csv'): # remove csv-files from sampling
        directory
254     Path.unlink(file)
```

Listing 39: Cleaning sampling directory

Optional Arguments

-v, --verbose
The -v argument outputs additional verbose information.

--superverbose
The --superverbose argument outputs the same information as the verbose argument, additionally also loop iteration output is shown.

-t, --time
The -t argument starts a timer on script start, measuring the script runtime. At the end of the script the runtime is shown.

-e, --export
The -e argument starts a timer on script start just like the -t argument, but additionally this argument starts dstat logging and exports the resulting data when the script is finshed. The arguments -t and -e are mutually exclusive.

5.3.4 Preprocessing Chain

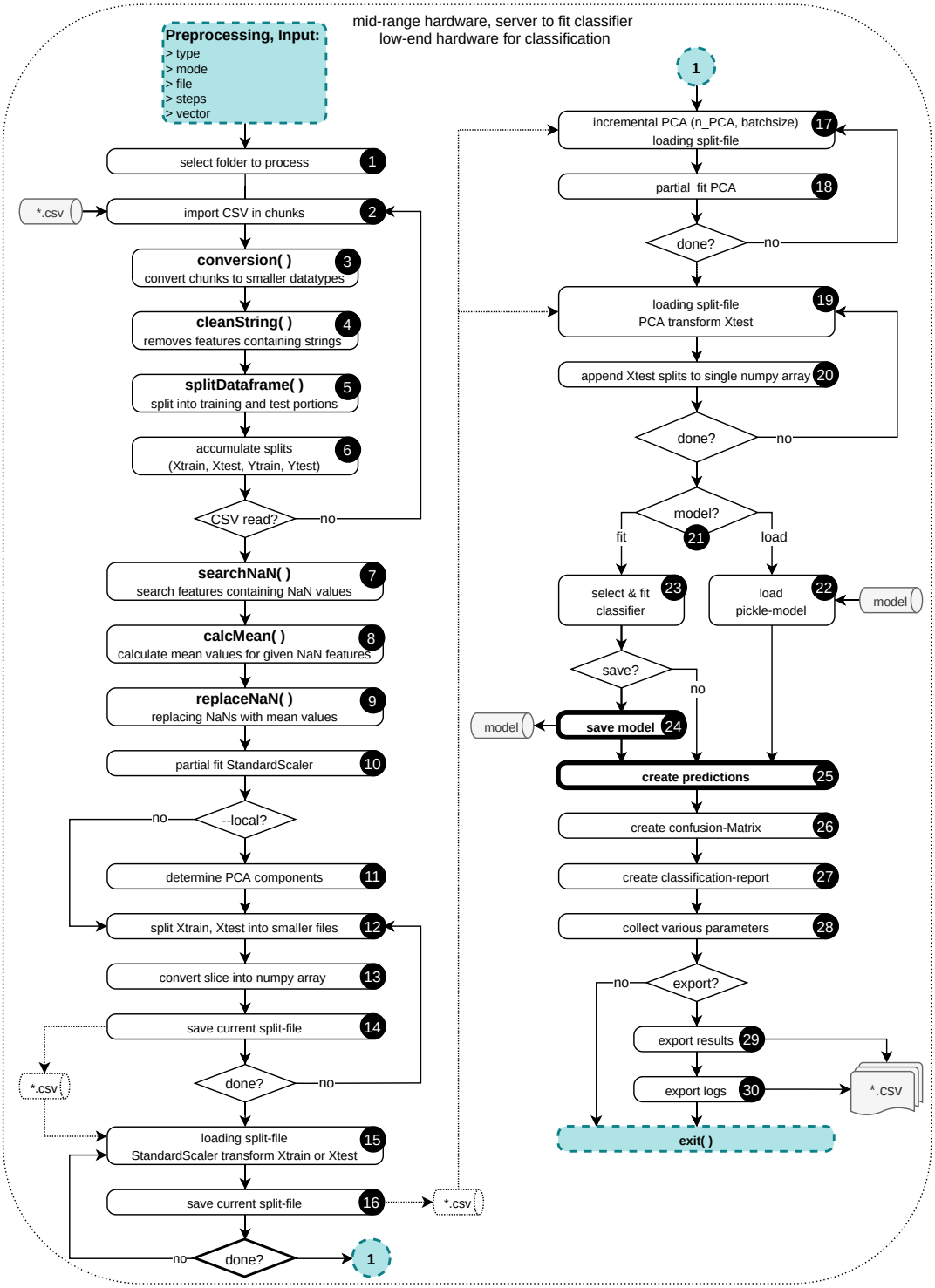

Figure 5.11: Preprocessing chain diagram

This module handles all of the preprocessing necessary for fitting or importing a model for creating predictions with the Random Forest classifier. The data that gets imported with this module already contains all features that represent the selected vector, either CAIA or AGM.

dstat is used to track hardware use when doing this, saving the gathered information, including timestamps, into a file for later evaluation.

0 When the module starts, *dstat* is executed threaded to monitor the systems resources and generate statistics during the whole classification process.

```
 69 # starting dstat logging threaded
 70 def threadFunc(): os.system(dstatcmd.format(cfg.dstat))
 71 th = threading.Thread(target=threadFunc)
    ...
718 dstatcmd = 'dstat --epoch --cpu-adv --disk --mem-adv --swap --output {} > /dev/
        null 2>&1 &'
    ...
778 if export: # write timestamp to csv
779     th.start() # start dstat loggin
```

Listing 40: *dstat* logging

Once the classification is finished, the generated resource statistic table is saved into the relevant experiment folder for later analysis.

1 The module generates the experiment base-foldername from the defintion in the framework configuration and selects the folderpath based on all relevant passed arguments. That way the module can access already generated files and save logs, or other files, into the correct experiment folder.

```
750 foldername = cfg.foldername.format(cfg.filenames[findex],m,j,n,sampling)
751
752 if flowsampling:
753     #foldername = '{}_perflowsampled'.format(foldername) # base folder
754     path    = cfg.flowfolder / foldername / csv_import # sampled CSV directory
755     logs    = cfg.flowfolder / foldername / log # logfolder path
756     modeld  = cfg.flowfolder / foldername / 'model' # pickle model directory
757     infocsv = cfg.flowfolder / foldername / 'information.csv' # PCA information
758 elif packetsampling:
759     #foldername = '{}_packetsampled'.format(foldername)
760     path    = cfg.packetfolder / foldername / csv_import
761     logs    = cfg.packetfolder / foldername / log
762     modeld  = cfg.packetfolder / foldername / 'model'
763     infocsv = cfg.packetfolder / foldername / 'information.csv'
```

Listing 41: Folder name genration, path selection

2 Because this work focuses on the usage of low-end hardware (and used a Raspberry Pi Zero W in its first iteration), the sampled file is imported in chunks based on the value of "chunksize"[17]. The reason to imort the file in chunks is to avoid exceeding the available RAM, which results in data being temporarily stored in the

[17] *chunksize* determines the number of lines read per chunk, can be set in the framework configuration, default is 10^5

slower SWAP memory. This has to be avoided at all costs. The chunks are split into training and test portions on the fly to further reduce the memory footpring, therefore reading a CSV line-by-line is not possible in this implementation.

```
818 for chunk in read_csv(path,chunksize=cfg.chunksize,
819 usecols=None,skipinitialspace=True,encoding='utf-8'):
```

Listing 42: Import file in chunks

3. The first operation performed on every chunk is to convert all imported feature-values into smaller datatypes whenever possible. This is necessary due to pandas importing data as a 64bit datatype by default if it is a numerical data type. For each imported chunk the module obtains the maximum values for all features.

```
155 def conversion(dataset,verbose=False):
...
160     features = list(dataset) # get feature labels
161     types = dataset.dtypes # get datatype per feature
162     maxValues = dataset.max() # get maximum values per feature
```

Listing 43: Obtain maximum values before conversion

If a feature value does not need the 64bit dtype its datatype gets downgraded to 32/16/8bit, depending on the maximum occorring value within this feature and whether it is a float or integer.

```
174 for x in types: # determine int64/float64 features
175     i = i + 1
176     if (x == 'int64'):
177         if (-(2**8)/2 <= maxValues[i] <= (2**8)/2):
178             dicttype[features[i]] = 'int8'
179         elif (-(2**16)/2 <= maxValues[i] <= (2**16)/2):
180             dicttype[features[i]] = 'int16'
181         elif (-(2**32)/2 <= maxValues[i] <= (2**32)/2):
182             dicttype[features[i]] = 'int32'
183     elif (x == 'float64'):
184         if (-(2**16)/2 <= maxValues[i] <= (2**16)/2):
185             dicttype[features[i]] = 'float32' # changed from float16 to float32,
    since pd.mean does not support float16 properly
186         elif (-(2**32)/2 <= maxValues[i] <= (2**32)/2):
187             dicttype[features[i]] = 'float32'
```

Listing 44: Datatype conversion

4. Following datatype conversion, features containing strings are identified and removed entirely.

```
821 cleanString(chunk,False,False) # remove all features with string dtype
```

Listing 45: Clean strings

5. Every chunk of data is divided into 70 percent training and 30 percent test portions using the scikit-learn *train_test_split()* function. The "random_state"[18] parameter is set to 1 to enable the reproducibility. It is noteworthy that, since the sampled data is read and processed in chunks, the specific flows included in the resulting Xtrain and Xtest data portions will always differ from training and test portions if

[18] the *random_state* parameter controls the shuffling applied to the data before applying the split.

a file is read in one go and split afterwards. The percentage obtained for the Xtest portions is not reported in this document, but it can be retraced by calculating the ratio of the number of instances listed in table 7.1 to the total number of instances after sampling saved alongside other experiment related information, as described in chapter 6.4.

```
612 def splitDataframe(dataset,testsize,verbose=False,time=False):
...
628     # all but the very last column put into X
629     X = dataset.iloc[:,:-1]
630     # very last column (label) put into Y as separate column
631     Y = dataset.iloc[:,-1]
632
633     # splitting up the data into training & validation portions
634     # Xtrain & Ytrain for preparing models
635     # Xtest & Ytest to use later for validation
636     Xtrain, Xtest, Ytrain, Ytest = train_test_split(X, Y, test_size=testsize,
        random_state=1)
637
638     data.append(Xtrain)
639     data.append(Xtest)
640     data.append(Ytrain)
641     data.append(Ytest)
...
670     return data
...
822 chunksplit = splitDataframe(chunk,0.30,False,False) # split into training & test
        portion
```

Listing 46: Split chunks into training and test portions

6 As the final step in the import process each chunk is appended to a pandas dataframe.

```
824 # accumulate splits
825 Xtrain = Xtrain.append(chunksplit[0])
826 Xtest  = Xtest.append(chunksplit[1])
827 Ytrain = Ytrain.append(chunksplit[2])
828 Ytest  = Ytest.append(chunksplit[3])
```

Listing 47: Accumulate splits of current chunk

7 Once the whole data is imported, a function is called to search the data for features containing NaNs on the training as well as the test portion of the data.

```
837 print('>>> Search NaN values')
838 NaNtrain = searchNaN(Xtrain,'Xtrain',verbose,False)
839 NaNtest  = searchNaN(Xtest,'Xtest',verbose,False)
```

Listing 48: Search NaN values

8 The mean value for all identified NaN features is calculated.

```
840 print('>>> Calculate NaN mean value replacements')
841 meantrain = calcMean(Xtrain,'Xtrain',NaNtrain,verbose,False)
842 meantest  = calcMean(Xtest,'Xtest',NaNtest,verbose,False)
```

Listing 49: Calculate mean values for NaN features

9 Following this step, the identified NaN containing features are passed combined with their according feature mean values to replace all occuring NaNs with each features' mean value.

```
843 print('>>> Replace NaN values with feature mean values')
844 replaceNaN(Xtrain,'Xtrain',NaNtrain,meantrain,verbose,False)
845 replaceNaN(Xtest,'Xtest',NaNtest,meantest,verbose,False)
```

Listing 50: Replace NaNs with mean values

10 Now the Standard Scaler is partially fitted to the training portion Xtest. The parameter "batch"[19] is used to set the maximum number of lines processed in one iteration. Again, this is done to reduce the overall memory consumption on low-end hardware.

```
857 print('>>> StandardScaling partial fit to Xtrain')
858 scaler = StandardScaler(copy=False)
859 n = Xtrain.shape[0] # total number of rows
860 processed = 0
861 while processed < n: # iterating until done
862     toprocess = min(batch, n-processed) # number of rows to process
863     scaler.partial_fit(Xtrain[processed:processed+toprocess])
864     processed += toprocess # increase number of already processed rows
```

Listing 51: Standard Scaler partial fit

11 When executed on the server or local machine the number of PCA components to reach the desired explained variance[20] needs to be calculated beforehand. This step is necessary because scikit-learns incremental PCA cannot take a value for the explained variance as an argument.

```
868 if local:
869     print('>>> Determine PCA component number for {}% explained variance'.format(
        cfg.PCA_var*100))
870     XtrainPCA = Xtrain.copy()
871     XtrainPCA = scaler.transform(XtrainPCA)
872
873     pca = PCA().fit(XtrainPCA) # fit data to training portion
874     xi = np.arange(1, XtrainPCA.shape[1]+1, step=1)
875     y = np.cumsum(pca.explained_variance_ratio_)
876     nPCAexact = np.interp(cfg.PCA_var,y,xi) # interpolate
877     nPCA      = math.ceil(nPCAexact) # round up to the next integer if float
```

Listing 52: Determine PCA component number

12 Now the entire dataset (Xtrain and Xtest) is split into smaller files to reduce memory usage for low-end hardware devices.

```
711 npy_Xtrain  = 'Xtrain_split_{}v{}.npy'
    ...
895 print('>>> Splitting data to reduce memory consumption')
896 n = Xtrain.shape[0]
897 toprocess = min(splitsize, n)
898 iteration = int(n/splitsize)+1
899 index = 0
900 while toprocess > 0:
901     index += 1
902     print('\t[{}/{}] Xtrain'.format(index,iteration))
903     npsave = cfg.tmp / npy_Xtrain.format(index,iteration)
```

Listing 53: Initialize splitting Xtrain

[19] *batchsize* can be set in the framework configuration, default value is 10^5

[20] *PCA_var* can be set in the framework configuration, default value is 0.90

13 Each slice in the current iteration is converted into a float32 numpy array.

```
905 print('\t\t> Converting dtype') # convert slice into np array
906 npXtrain = Xtrain[:][0:toprocess].to_numpy().astype(np.float32)
907 Xtrain = Xtrain.drop(Xtrain.index[0:toprocess]) # drop processed slice from df
```

Listing 54: Convert current Xtrain slice to np array

14 After datatype conversion is applied the slice is saved as *.npy file into the tmp folder within the working directory for later usage. The number of rows per slice is defined as "splitsize"[21] in the framework configuration.

```
909 print('\t\t> Saving')
910 np.save(npsave,npXtrain)
911 n -= toprocess # number of rows that need to be processed
912 toprocess = min(splitsize, n)# get slice-size for next iteration
```

Listing 55: Save Xtrain transformed slice

These steps are repeated for the test portion Xtest of the data in a simliar way.

15 Following this, the module iterates all previously created split-files of the training portion, loading one split-file at a time. The Standard Scaler transform per split-file is again applied in iterations, similar to the described partial fit. The defined "batch"[22] number of lines is transformed in one iteration.

```
955 for index in iXtrain: # cycle through split-files
956     print('\t[{}/{}] Xtrain'.format(index,len(iXtrain)))
957     Xtrain_scaled = np.empty(shape=[0,len(features)]) # initialise empty np array
958     npload = cfg.tmp / npy_Xtrain.format(index,len(iXtrain))
959     tmp = np.load(npload).astype(np.float32) # load split-file
...
962     print('\t\t> Transforming')
963     n = tmp.shape[0]
964     size = min(batch, n)
965     while size > 0:
966         tmpscaled = scaler.transform(tmp[:][0:size],copy=None) # transform rows
967         tmp = np.delete(tmp,np.s_[0:size:1],axis=0) # delete rows from array
968         Xtrain_scaled = np.append(Xtrain_scaled,tmpscaled,axis=0).astype(np.
    float32)
969         n -= size
970         size = min(batch,n)
```

Listing 56: Loading splits and applying the Standard Scaler transform to Xtrain

16 After the Standard Scaler transform is completed, the processed split gets saved again.

```
975 print('\t\t> Saving')
976 npsave = cfg.tmp / npy_Xtrain.format(index,len(iXtrain))
977 np.save(npsave,Xtrain_scaled)
978 del Xtrain_scaled
```

Listing 57: Save transformed Xtrain split-file

Again these steps are done in a similar way to transform the test portion Xtest of the data.

[21] *splitsize* is set to 250k rows per slice per default

[22] *batchsize* can be set in the framework configuration, default value is 10^5

17 As stated in chapter 4.4.2, the final step before the actual classification is the application of PCA on the training and test portions of the data. More details on the chosen implementation are summarized in chapter 5.3.4.

```
1025 infoPCA = read_csv(infocsv) # read information.csv
1026 n_Xpca = int(infoPCA.loc[6][1]) # load PCA component number
1027 del infoPCA
1028
1029 print('>>> Applying PCA fit with {} components'.format(n_Xpca))
1030 ipca = IncrementalPCA(n_components = n_Xpca, batch_size = cfg.PCA_batch)
1031
1032 for index in iXtrain: # partial fit PCA to Xtrain, iterating over split-files
1033     print('\t[{}/{}] Xtrain'.format(index,len(iXtrain)))
1034     npload = cfg.tmp / npy_Xtrain.format(index,len(iXtrain))
1035     split = np.load(npload).astype(np.float32)
```

Listing 58: Loading Xtrain splits to apply incremental PCA

18 Partial fit is applied to every loaded split-file.

```
1037     print('\t\t> Fitting')
1038     ipca.partial_fit(split)
1039 del split
```

Listing 59: Applying PCA partial fit to split

19 When the PCA fitting is finished, either first the training portion is transformed in order to fit the Random Forest model, or this step is skipped and the test portion is transformed immediately. Either way, the portions split-files are loaded in sequence and transformed.

```
1068 Xtest = np.empty(shape=[0,n_Xpca]) # initialise empty numpy array
1069 if model: print('>>> Applying PCA transform')
1070 for index in iXtest:
1071     print('\t[{}/{}] Xtest'.format(index,len(iXtest)))
1072     npload = cfg.tmp / npy_Xtest.format(index,len(iXtest))
1073     split = np.load(npload).astype(np.float32)
...
1076     print('\t\t> Transforming')
1077     split = ipca.transform(split)
```

Listing 60: Load and PCA transform splits, Xtest

20 Following that, the transformed splits are appended to a numpy-array for the upcoming classification.

```
1081     Xtest = np.append(Xtest,split,axis=0).astype(np.float32)
1082 del split
1083 del ipca;
```

Listing 61: Append transformed splits to numpy array, Xtest

21 Depending on the passed argument, the module now either selects a classifier and fits it to the training portion Xtrain of the data before classifying the test portion Xtest or it loads an already created model and uses that for classification.

22 An already generated model is loaded, this should be the preferred choice on any low-end hardware device.

```
1096 print('>>> Importing model')
1097 model = joblib.load(modelfile)
```

Listing 62: Importing an already fitted model

It is worth noting that a model generated with scikit-learn (whether through joblib or pickle) is not necessarily compatible with different CPU architectures. This problem occurred when attempting to load a model fitted by a server running on 64bit x86 architecture into a Raspberry Pi Zero W, which runs on 32bit ARM6l. Based on the error output, it seemed that there was a 32bit/64bit compatibility issue, but it became apparent that this wasn't the case when the model was still incompatible after it was fitted on a x86 device running a 32bit Debian installation. All systems tested had little-endian "endianness"[23], so it seems like the issue boils down to an architectural incompability. This problem was not investigated further in this study; instead, the experiment data was collected using a virtual machine (LXC container) running on x86 like the server. This obviously is a helpful workaround for this study. More details about the chosen virtualization can be read up in chapter A.1.3.

23 If no model is imported by using the appropriate argument, the module fits the model to the training portion Xtrain before performing the upcoming classification.

```
1106 print('>>> Fitting RandomForestClassifier')
1107 model = RandomForestClassifier(n_estimators=cfg.maxtrees,max_depth=cfg.maxdepth,
         max_leaf_nodes=cfg.maxleaves)
1108 model = model.fit(Xtrain,Ytrain)
1109 del Xtrain
```

Listing 63: Fitting the Random Forest classifier

The parameters "n_estimators"[24], "max_depth"[25] and "max_leaf_nodes"[26] are parameters for the Random Forest classifier set in the framework configuration.

24 The-s/--save option instructs to save the fitted model for later use on other devices.

```
1111 if save:
1112     print('>>> saving model {}'.format(modelfile))
1113     joblib.dump(model,str(modelfile),compress=False)
```

Listing 64: Save Random Forest model

25 When the Random Forest model is ready, the module begins to create predictions for the test portion of the data.

```
1117 print('>>> Creating predictions')
1118 predictions = model.predict(Xtest)
```

Listing 65: Create predictions

[23] *endianess* determines the order or sequence of bytes of a word of digital data in computer memory.
[24] *maxtress* determines the number of trees within the created Random Forest model, per default set to 100.
[25] *maxdepth* sets a limit for tree-depth, per default unlimited (none).
[26] *maxleaves* limits the number of leaves per tree, per default unlimited (none).

26 The confusion-matrix is created based on the classifier's predictions.

```
print('>>> Creating confusion-matrix')
matrix = confusion_matrix(Ytest,predictions)
```

Listing 66: Create confusion-matrix

27 The classification-report is generated.

```
print('>>> Creating classification-report')
report = pd.DataFrame(classification_report(Ytest, predictions, digits=5,
    output_dict=True)).transpose()
```

Listing 67: Create classification-report

28 Various other parameters are collected for later analysis and informational output.

```
print('>>> Obtain various Random Forest estimator values')
depths = [t.get_depth() for t in model.estimators_]
leaves = [t.get_n_leaves() for t in model.estimators_]

print('>>> Saving parameters, accuracy-score and feature-importance')
parameters = model.get_params(deep=True)
accuracyscore = accuracy_score(Ytest,predictions)
featureimportance = model.feature_importances_
```

Listing 68: Collect various parameters

29 When the -e/--export argument is passed various results are exported into a separate folder for later evaluation.

```
if export:
    print('\n>>> Exporting results to folder: {}'.format(cfg.logs))
    evaluation = {'model':[model],'parameters':[parameters],
    'accuracy-score':[accuracyscore],'feature-importance':[featureimportance],
    'confusion-matrix':[matrix],'PCA-components':[n_Xpca],'Trees':[len(depths)],'
    Depths':[depths],'Leaves':[leaves]}
    results = pd.DataFrame.from_dict(evaluation,orient='index',columns=['summary'
    ])
    results.to_csv(cfg.result) # save results
    report.to_csv(cfg.report) # save classification-report
```

Listing 69: Export results

30 All logs in the working directory are exported to the relevant experiment folder.

```
print('>>> Saving logs to folder {}'.format(logs))
if not os.path.exists(logs): os.mkdir(logs) # create logfolder if necessary
for root, dirs, files in os.walk(cfg.logs):
    for filename in files: # iterate over filenames found within the wd logfolder
        log = cfg.logs / filename # full path for current logfile
        print('\t> Saving {}'.format(filename))
        os.system(cplogs.format(log,logs))
```

Listing 70: Export logs

Optional Arguments

-v, --verbose
The -v argument outputs additional verbose information.

--superverbose
The --superverbose argument outputs the same information as the verbose argument, additionally dataset related information are shown when applying the Standard Scaler transforms.

-f, --fit
The -f argument lets the classification model fit on the remote device instead of importing a model file.

-m, --model
The -m argument loads a model file instead of fitting the classifier.

-s, --save
This argument saves a fitted or imported model as file.

-l, --local
Is used to determine the PCA component number required to achieve the explained variance set in the framework configuration; when running an automated experiment on the local machine, this argument is passed. The calculated number of components is saved in information.csv, allowing the script to access the component number on the remote device to perform incremental PCA without using low-end hardware resources.

-r, --remote
Unused with current VM implementation. This argument was utilized to execute the script on the remote device (Raspberry Pi Zero W). Utilizes a different method to kill dstat when the script is finished. Picks the correct foldername to import the model file.

-t, --time
The -t argument starts a timer on script start, measuring the script runtime. At the end of the script the runtime is shown.

-e, --export
The -e argument starts a timer on script start just like the -t argument, but additionally this argument starts dstat logging and exports the resulting data when the script is finshed. The arguments -t and -e are mutually exclusive.

-f m, --flowsampling m
Selects flow-based sampling and the sampling-mode *m*. This is necessary for appropriate folder selection to load all needed data for preprocessing and classification.

-p m, --packetsampling m
Selects packet-based sampling and the sampling-mode *m*. This is necessary for appropriate folder selection to load all needed data for preprocessing and classification. The arguments -f and -p are mutually exclusive.

Classification Results

The results gathered with an experiment include the following information:

Table 5.2: Classification results

Classification Results	
file	Name of the classified capture-file.
sampling technique	Flow-based or packet-based sampling.
sampling mode	Includes the applied mode and the value for n. Information about the portion of actually sampled packets is listed in table 6.1.
feature-vector	Used feature-vector, basically the configuration file used to execute *go-flows*.
classification report	Contains various metrics useful for later evaluation, including precision, recall, f1-scores and support. Obtained results per instance are not recorded.
results	Contains various information and parameters from the chosen classifier: accuracy, PCA feature-importance, confusion-matrix and the PCA component number.
resource log	Log generated with *dstat*, contains various hardware metrics.
time log	Log including timestamps in epochtime for later evaluation.

Random Forest Implementation

The Random Forest classifier is used for classification in this research. The algorithm is implemented in Python's sklearn machine learning library[27] version 0.23.2, the Gini impurity is chosen as the criterion for measuring the quality of a split.

Leaving the parameters controlling the size of the created trees at their default values results in fully grown and unpruned trees. The complexity and size of the trees can be managed to reduce memory consumption by setting certain parameter values. The chosen values for fitting the Random Forest classifiers are described in 23.

By default, the features are randomly permuted at each split; if deterministic behavior for fitting is required, the "random_state" parameter has to be set. The default value for this parameter is *none* and used in this work.

[27] https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

PCA Implementation

It is important to remember that PCA is influenced by scale, so every numerical feature in the dataset has to be scaled prior PCA. The StandardScaler (Z-score normalization) was applied for optimal results, the rescaled features have a mean of zero and a standard deviation of one.

In this implementation, Incremental Principal Component Analysis is utilized to improve memory efficiency. The advantage of this form of PCA is the constant memory complexity, which avoids loading an entire file into memory. The important arguments for the incremental PCA are "n_components" and "batch_size"[28], both of these values are defined in the framework configuration.

As already described in 11, inremental PCA does not support the argument to set a desireable value for the explained variance, therefore the number of components "n_components" has to be calculated beforehand. Instead of simply setting a fixed number of dimensions this method guarantees maintaining certain information with the minimum amount of dimensions necessary.

All experiments in this study reduce the dimensionality of the given data to reach at least 90% explained variance. The reasoning behind choosing 90% explained variance and not a higher value is that the remaining 10% are shared between many features. Not only are those remaining features kind of irrelevant compared to the top features that contain the biggest portions of variance, picking a higher value will also result in less dimensionality reduction.

[28] *PCA_batch* is set to 10^5 rows per increment per default

5.3.5 Automation

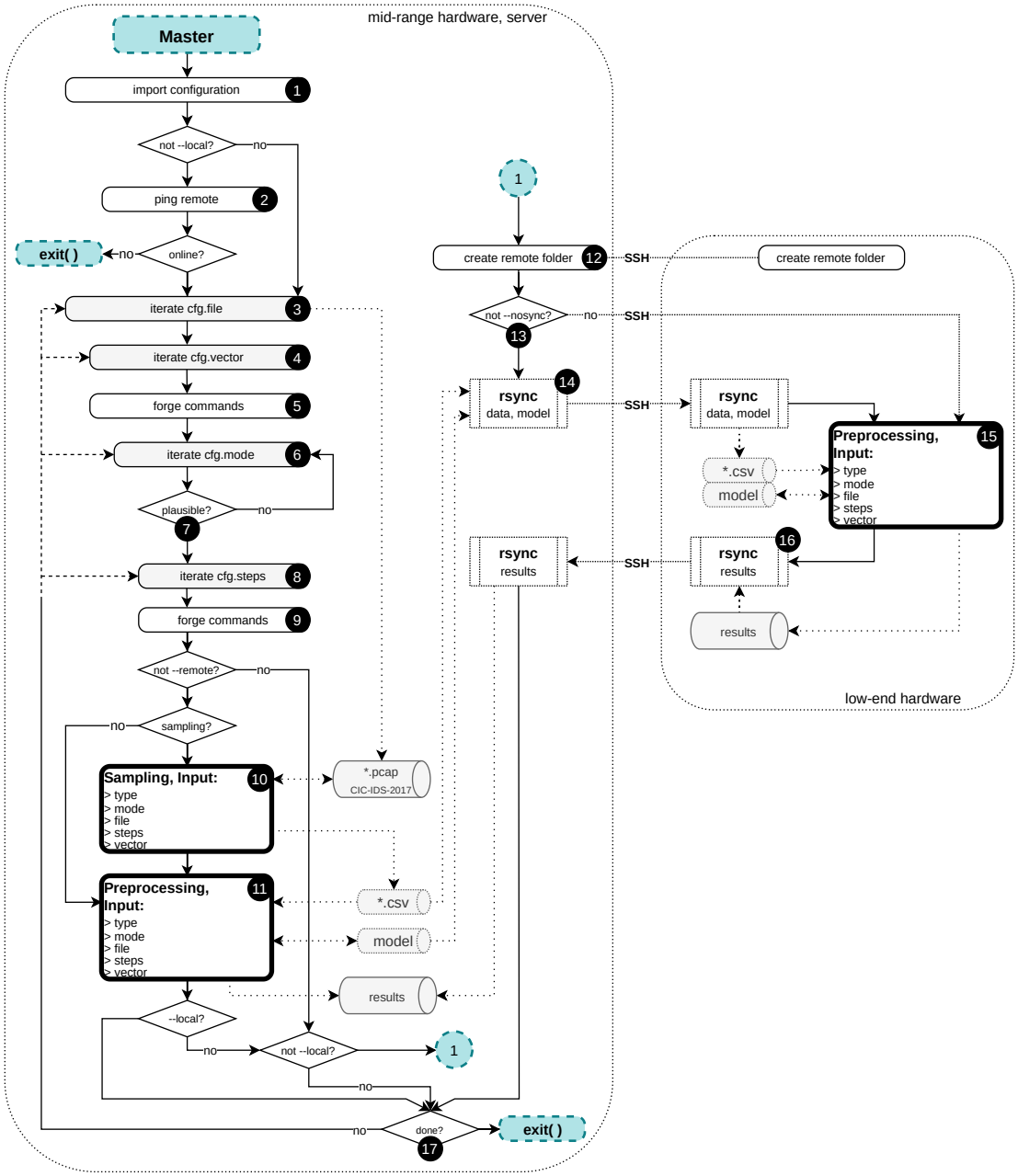

Figure 5.12: Automation diagram

This is the control-module for all previously described modules, executing all required os-commands and modules in the correct sequence, based on the stated framework configuration and all passed arguments.

1 The module imports the framework configuration in order to obtain all informa-

tion needed for automated experiment execution. Chapter A.2 contains a brief description of all configurable parameters.

2. If remote execution is selected by passing the appropriate script argument, or if no arguments concerning remote and local execution are passed, the script attempts to ping the remote device at the IP address specified in its framework configuration. The script only continues if the remote device responds to the ping.

```
64      if not local:
65          ping = os.system('ping -c 1 {} >/dev/null 2>&1'.format(cfg.remoteip))
```

Listing 71: Check remote device online status

3. The script loops through the specified capture files.

4. As well as over all specified feature-vectors defined in the framework configuration in order to execute the experiments in sequence.

5. Based on the current feature-vector, the basefolder path and foldername to store the generated data and results are created. Additionally the necessary argument to execute the sampling is forged for later usage.

6. The script iterates through all specified sampling modes.

7. Before proceeding, a plausibility check is performed to decide if the current feature-vector and sampling mode combination is a legitimate experiment.

```
87 # plausibility check of given configuration, continue on pointless combinations
88 # flow-based feature-vectors and packet-based sampling-modes
89 if vector < cfg.vectorlimit and mode >= cfg.samplinglimit: continue
90 # packet-based feature-vectors and flow-based sampling-modes
91 elif vector >= cfg.vectorlimit and mode < cfg.samplinglimit: continue
```

Listing 72: Current configuration plausability check

8. Iterate through the specified sampling "steps"[29].

9. Forge commands that are used to execute sampling and preprocessing scripts in the current iteration based on all passed arguments and the experiment configuration.

10. If the -r/--remote and the --nosampling argument are not passed, sampling is performed on the local machine to prepare the data for later classification.

```
110 if (not remote): # LOCAL
111     if nosamp:
...
121     if samp:
122         print(Fore.YELLOW+'\n>>> Sample PCAP on local machine: {}'.format(
        sampling)+Style.RESET_ALL)
123         callCommand(sampling)
```

Listing 73: Execute sampling on local machine

Calling the callCommand function, os.system() is invoked to execute all operating system-related commands. If os.system() does not return 0 (success) as process exit value, the script exits to prevent corrupted data from being generated.

[29] n in combination with the selected mode determine the applied sampling pattern

```
32 def callCommand(function):
33     if (os.system(function)) != 0:
34         print(Fore.RED+'\n<<< ERROR for os.system({})\n'.format(function)+Style.
       RESET_ALL)
35         exit()
36     return
```

Listing 74: callCommand function definition

11 Following that, preprocessing and classification are performed on the local machine in order to fit and export the resulting model file for future use. Post completion of both scripts for all iterations for the specified steps, either the next configuration is processed on the local machine or the generated data (sampled capture and model) is synced to the remote device and classification is performed via SSH execution.

12 To begin classification on the remote device, the first step is to use SSH to create the appropriate folder on the remote device.

```
83 mkdir = "ssh {} 'mkdir -p {}'".format(cfg.remote,basefolder)
   ...
133 if (not local): # REMOTE
134     print(Fore.YELLOW+'\n>>> Create base-folder on remote machine: {}'.format(
       mkdir)+Style.RESET_ALL)
135     callCommand(mkdir)
```

Listing 75: Create remote folder

13 Depending on the usage of the --nosync argument, the script either continues syncing the prepared sampled data and model to the remote device before starting the preprocessing and classification script remotely, or it executes the scripts directly on the remote device, assuming the necessary data is already located on the remote device.

14 *rsync* is used to transfer the sampled data, fitted model and currently used framework configuration from the local computer to the remote device. When the results are obtained, *rsync* is again utilized to sync all results back to the local machine.

```
106 sync = r'rsync -avz --progress {}/{}/ {}:{}/{}/'.format(basefolder, folder, cfg.
       remote, basefolder, folder) # sync to remote
107 csync = r'rsync -avz --progress {}/{} {}:{}'.format(cfg.wd, cfg.configuration,
       cfg.remote, cfg.remoteconf)  # sync configuration
108 resync = r'rsync -avz --progress {}:{}/{}/ {}/{}/'.format(cfg.remote, basefolder,
        folder, basefolder, folder) # sync to local
```

Listing 76: *rsync* commands

15 The remote device performs preprocessing and classification. The model is either imported or fitted (via the -f/--fit argument) on the remote computer, depending on the passed argument.

16 All results are gathered, saved into the according experiment folder, and synced back to the local machine for further analysis.

17 Once all experiment configurations are processed the script exits.

Optional Arguments

-v, --verbose
The -v argument outputs additional verbose information.

-f, --fit
The -f argument lets the classification model fit on the remote device instead of importing a model file.

-m, --model
The -m argument loads a model file on the local machine instead of fitting the classifier and exporting the model file.

--nosync
This argument surpresses the sync from the local machine to the remote device. Useful when all necessary files have been processed and are already synced to the remote device.

--nosampling
This argument checks if a folder matching the current experiment configuration already exists. If such foler is found, the script skips the sampling process on the local machine.

-l, --local
This argument executes all scripts on the local machine. Nothing is done on the remote device when this argument is passed.

-r, --remote
Skips all script executions on the local machine and runs preprocessing and classificaton only on the remote device. The arguments -l and -r are mutually exclusive.

CHAPTER 6

Experiments

The examined and compared experiment configurations are listed in this chapter, and various configuration details are thoroughly described.

Every experiment in this research uses *go-flows* to collect flows, as described in chapter 5.1.1, and all experiments are based on the raw capture files from the *CIC-IDS-2017* dataset, as described in chapter 6.3. Different sampling techniques and modes, specified in chapter 3.2, are applied on the raw dataset before being passed to the selected classifier. In this study the Random Forest classifier, a decision tree-based, supervised learning algorithm is used to create the machine learning model on a server for different sampled data. This generated model is then passed to a low-end hardware device to perform the classification. Information about the Random Forest classifier is located in chapter 4.2.

The experiment configurations listed in the upcoming section contain all necessary information about the applied sampling and parameters used for the classification algorithm. All metrics used for evaluation are explained in detail in section 6.2. All scripts from the utilized *Python* framework explained in chapter 5.3 are publicly available for experiment repeatability from the CN-TU repository[1].

[1] https://github.com/CN-TU/packet-sampling-for-lightweight-ml-based-anomaly-detection

6.1 Details

Table 6.1 lists all of the experiment configurations investigated in this study. In conjunction with the framework's default settings and the publicly accessible repository for this project, the information provided is sufficient to ensure the repeatability of all experiments.

Table 6.1: Experiment configurations

JSON	**Sampling**				**PCA**	**ID**
feature-vector	technique	mode	steps	packets[1]	components	experiment
AGM10s	flow-based	Unsampled		100%	14	**I**
		every n-th	n = 3	34.18%	14	**II**
			n = 5	21.20%	15	**III**
			n = 7	15.63%	14	**IV**
		first n packets	n = 3	4.62%	14	**V**
			n = 5	6.28%	14	**VI**
			n = 7	7.61%	14	**VII**
	packet-based	Unsampled		100%	14	**VIII**
		every n-th	n = 3	33.33%	15	**IX**
			n = 5	20%	14	**X**
			n = 7	14.28%	14	**XI**
		random	p = 20%	20%	14	**XII**
			p = 10%	10%	15	**XIII**
CAIA	flow-based	Unsampled		100%	10	**XIV**
		every n-th	n = 3	36.44%	11	**XV**
			n = 5	24.41%	11	**XVI**
			n = 7	19.06%	11	**XVII**
		first n packets	n = 3	16.31%	9	**XVIII**
			n = 5	21.86%	9	**XIX**
			n = 7	26.15%	9	**XX**
	packet-based	Unsampled		100%	10	**XXI**
		every n-th	n = 3	33.33%	14	**XXII**
			n = 5	20%	15	**XXIII**
			n = 7	14.28%	15	**XXIV**
		random	p = 20%	20%	15	**XXV**
			p = 10%	10%	15	**XXVI**

[1] ***packets:*** Percentage of sampled packets for each experiment configuration based on the total number of 56,370,702 packets contained in the unsampled dataset (see table 6.3).

Every experiment was carried out with the default values set in the framework configuration. Table 6.2 contains an overview of these default values for all experiment-related parameters.

A detailed explanation about the framework configuration, including its structure, is described in chapter A.2. The training and test portions used to fit the classifier and create the predictions evaluated in the results are always split 70/30. All of the results and findings described in chapter 7 are linked to the experiment ID listed in the last column of table 6.1.

Table 6.2: Experiment-related framework default values

PCA		Scaler	CSV	Sampling	Classification	Estimator		
PCA_var[1]	PCA_batch[2]	batchsize[3]	chunksize[4]	split[5]	splitsize[6]	maxtrees	maxdepth	maxleaves
0.90	10^5	10^5	10^5	5000	$25 \cdot 10^4$	100	None	None

[1] ***PCA_var:*** The explained variance target value to achieve with the PCA components used in the experiment execution.
[2] ***PCA_batch:*** Number of rows per increment when applying incremental PCA on the remote device.
[3] ***batchsize:*** Number of rows per call for fitting the Standard Scaler in *partial_fit* mode.
[4] ***chunksize:*** Number of rows per chunk when importing CSV files.
[5] ***split:*** Number of packets per split-file generated during packet-based sampling with *editcap*.
[6] ***splitsize:*** Number of rows per slice when iterating through data during classification.

6.2 Metrics

The terminology and metrics required to understand the results of the experiments are explained in this section. The classification problem investigated in this study is, in general, a binary one. An instance is classified as either negative as benign traffic or positive as attack traffic.

There are four possible outcomes for predictions:

- **TP (True Positive):**
  When an instance is positive and predicted positive.
- **FP (False Positive):**
  When an instance is negative but predicted positive.
- **TN (True Negative):**
  When an instance is negative and predicted negative.
- **FN (False Negative):**
  When an instance is positive but predicted negative.

6.2.1 Confusion Matrix

The confusion matrix is a type of matrix or table that visualizes predictions for a classification problem. The purpose is already implied by the name, and the layout of the matrix makes it quite simple to determine the number of confused classes returned in the classifiers' predictions.

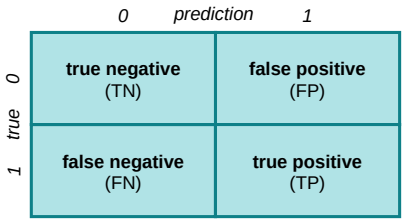

Figure 6.1: Sklearn confusion matrix

Figure 6.1 illustrates the layout of the confusion matrix made with the *sklearn* module in *Python*. The columns represent the classifier's predictions, while the rows represent the true classes. True negatives, false negatives, false positives, and true positives are

all reported by the cells. The true positives and true negatives are represented on one diagonal of the matrix by the number of correctly predicted instances while the false positives and false negatives are shown on the other diagonal. These values allow for far more detailed analysis than the proportion of correct classifications alone.

6.2.2 Performance Metrics

The classification report[2] is also created in *Python* using the *sklearn* module. In general, this report assesses the quality of predictions for a given classifier by utilizing the previously mentioned values contained in the confusion matrix to calculate new metrics as described in the subsections that follow. It should be noted that some metrics on their own easily provide wrong impressions about a classifiers performance; therefore, it is always recommended to examine all relevant metrics rather than just one.

Predictive Values

PPV (positive predictive value) or *Precision* is a measure that indicates the accuracy of positive predictions made, or, in other words, the classifier's ability to not label an instance as an attack when it is actually harmless traffic. The more false positives the classifier detects, the lower this metric becomes.

$$PPV = Precision := \frac{TP}{TP + FP} \tag{6.2.1}$$

NPV (negative predictive value) is a measure that indicates the accuracy of negative predictions made, or, in other words, the classifier's ability to not label an instance as benign when it is actually attack traffic. The more false negatives the classifier detects, the lower this metric becomes.

$$NPV := \frac{TN}{TN + FN} \tag{6.2.2}$$

Sensitivity

The recall for the positive class in binary classification problems is also known as *Sensitivity* or *TPR (true positive rate)*. This measure shows how well a classifier predicts the positive class. It should be noted, however, that simply classifying every single instance as positive allows this measure to be easily manipulated.

$$Sensitivity = Recall_1 := \frac{TP}{Positives} = \frac{TP}{TP + FN} \tag{6.2.3}$$

[2] https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

Selectivity

Selectivity, as opposed to *Sensitivity*, is the recall for the negative class and can be viewed as a measure of a classifier's ability to predict negative classes, or in other words, the ability to avoid returning false positives. *TNR (true negative rate)* is an abbreviation for this metric.

$$Selectivity = Recall_0 := \frac{TN}{Negatives} = \frac{TN}{TN + FP} \tag{6.2.4}$$

F_β-score

F_β provides the generalized formula to adjust the weighting of *PV (predictive value)* and *Recall*, depending on which metric is more important for a specific use case. If β has a value of 1 the calculated metric provides the harmonic mean of PV and *Recall*.

$$F_\beta := (1 + \beta^2) \cdot \frac{PV \cdot Recall}{(\beta^2 \cdot PV) + Recall} \tag{6.2.5}$$

$F_{1,1}$-score

The $F_{1,1}$-score is the harmonic mean of *Precision* and *Sensitivity*, giving equal weight to both measures. Because it combines two important measures, the ability to predict attacks while not returning false positives, this score is well suited as a metric for this classification problem.

$$F_{1,1} := \frac{2}{Precision^{-1} + Sensitivity^{-1}} \tag{6.2.6}$$

$F_{1,0}$-score

The $F_{1,0}$-score is the harmonic mean of *NPV (negative predictive value)* and *Selectivity*, giving equal weight to both. This score combines two measures, the ability to predict benign traffic while not returning false negatives.

$$F_{1,0} := \frac{2}{NPV^{-1} + Selectivity^{-1}} \tag{6.2.7}$$

6.2.3 Own Metrics

Two parameters were developed to provide a meaningful metric for comparing all of the different types of sampling technique, mode, and feature-vector combinations investigated in this study in the form of all of the experiments listed in table 6.1.

The idea is to combine various metrics, some of which are related to classification performance in terms of achieved classification results, and others which show the impact

of different experiment configurations on hardware usage. As described in the following paragraphs, all metrics are min-max scaled and use different notations depending on where the minimum and maximum values derive from.

The following metrics are used to assess the classification algorithms computational performance, where the number of instances represents the number of flows that were passed to the Random Forest Classifier in order to generate preditions:

- **$\mathbf{Recall_1}$:** Demonstrates the classifier's ability to recognize attack instances for a specific experiment configuration.
- **$\mathbf{F_{1,1}}$-score:** This score, like $Recall_1$, acknowledges the ability to recognize attack instances, but it also takes into account the generation of false positive predictions.
- **Instances:** The number of instances is used to assess a sampling mode's ability to pass as many instances to classification as possible. This trait not directly related to classification performance, but is still an important attribute for reliable anomaly detection.

Metrics used to assess the hardware resource usage:

- **$\mathbf{t_{Runtime}}$:** Total runtime required to complete the entire preprocessing chain, including classification, as described in chapter 5.3.4. Importing (sampled) data, datatype conversions, cleaning, scaling, PCA, generating predictions, and exporting all data for analysis are all part of this process.
- **$\mathbf{t_{Classification}}$:** The amount of time required for the Random Forest classifier to make predictions from the given test portion of the data. This time period begins after PCA is applied to the test portion of the data and ends when the predictions are made, and it does not include any subsequent report generation or file exports.
- **mem:** System memory usage observed and logged by *dstat* while executing the preprocessing chain on the low-end hardware device. Information about *dstat* usage in this study can be found in chapter 5.1.3, implementation details in chapter 5.3.4, specifically item 0.

Metric Scaling

The scaling used for the previously mentioned metrics is known as min-max normalization or min-max scaling. This type of scaling is used to normalize the range of independent variables or features in order to produce a meaningful parameter that is unaffected by the numerical ranges of the variables involved. The general formula is:

$$x' := \frac{x - min(x)}{max(x) - min(x)} \tag{6.2.8}$$

The outcome for the scaled variable will be a numerical value in the range [0,1].

Notation

The main distinction between the two parameters P_{total} and P_{vector} developed for this study is how metric scaling is applied, specifically where the minimum and maximum values for each metric are derived. The first parameter P_{total} takes all different experiment configurations into account and derives minimum and maximum values based on all results, whereas the second parameter P_{vector} only considers results within its feature-vector and regardless of the sampling mode used. This distinction is represented by different notations in the formula for both parameters. Scaled metrics derived from all experiment results are denoted by a prime symbol, whereas metrics derived from results within its feature-vector are represented by a double prime symbol.

Description

In general both metrics contributed in this study share a similar behaviour. Better results in classification performance increase each metric's score while higher ressource usage in terms of overall script runtime $t_{Runtime}$, classification time $t_{Classification}$ or memory usage mem decreases its score, regardless of the feature-vector.

Because different flow-keys will naturally deliver different numbers of instances for classification, the number of instances is always scaled within each feature-vector category. It would be illogical to compare this number across different types of feature-vectors. The number of instances is interesting because it represents one distinction between the sampling implementations for packet-based and flow-based techniques. While flow-based sampling always collects every observed flow for classification, packet-based sampling may skip entire flows depending on the packets that are actually sampled. Lower number of instances lead to lower scores for both metrics.

The first metric P_{total} defined in equation 6.2.9 incorporates achieved scores for $F_{1,1}$ and Sensitivity based on all results across all experiment configurations investigated in this study. The goal of this metric is to compare both feature-vectors and all applied sampling configurations; the higher its value, the better a particular experiment configuration performs.

$$P_{\text{total}} := \frac{F_{1,1}{}' + Sensitivity'}{t_{\text{Runtime}}{}' + t_{\text{Classification}}{}' + mem'} \cdot Instances'' \tag{6.2.9}$$

The second metric P_{vector} defined in equation 6.2.10 is built similar to P_{total}, but scales the achieved scores for $F_{1,1}$ and Sensitivity within each feature-vector category. Based on the sampling configuration used, this metric attempts to assess the degree of deterioration within each feature-vector category. Its goal is to determine which sampling configurations perform best for a given feature-vector while disregarding classification performance achieved with other feature-vectors. A higher value indicates better overall performance.

$$P_{\text{vector}} := \frac{F_{1,1}{}'' + Sensitivity''}{t_{\text{Runtime}}{}' + t_{\text{Classification}}{}' + mem'} \cdot Instances'' \tag{6.2.10}$$

6.3 CIC-IDS-2017

The Intrusion Detection Evaluation Dataset *CIC-IDS-2017*[3] was used for all experiments in this study and is available from the Canadian Institute for Cybersecurity. Sharafaldin et al. [36] generated this publicly available dataset that includes seven common updated types of attacks such as:

- Web (Server) Attacks
- Brute Force
- DoS
- DDoS
- Infiltration
- Heartbleed
- Bot and Scan

This provides a comprehensive and valid dataset for research in this domain.

The dataset description linked in the footnote lists all of the relevant information for all types of attacks, such as victim and attacker network information and timestamps. As already mentioned in the introduction of this document, this information is used to label the traffic in order to train the Random Forest decision tree algorithm for classification and to generate the classification report.

Table 6.3 summarizes the basic characteristics of the unsampled *CIC-IDS-2017* dataset. This includes the total number of packets contained in each dataset file, as well as various statistics when using the CAIA and AGM feature-vectors.

Table 6.3: CIC-IDS-2017, flow distributions

File	Packets	Flows					
		CAIA			AGM		
		total	benign	attacks	total	benign	attacks
Monday	11,709,971	395,857	395,857	0	278,191	278,191	0
Tuesday	11,551,954	333,687	330,867	2,820	247,487	246,758	729
Wednesday	13,788,878	636,386	386,657	249,729	258,985	257,721	1,264
Thursday	9,322,025	393,366	314,818	78,548	242,575	241,226	1,349
Friday	9,997,874	558,626	302,967	255,659	228,485	227,383	1,102
	56,370,702	**2,317,922**	**1,731,166**	**586,756**	**1,255,723**	**1,251,279**	**4,444**
			(74.69%)	(25.31%)		(99.65%)	(0.35%)

The number of collected flows is highy dependent on the used flow-key, as described in chapter 3.2.1. This also includes durations for active and idle timeouts when using the CAIA feature-vector, as described in chapter 5.1.1. When using the AGM feature-vector the set value for the observation window, as described in chapter 4.3.2, is as important as the durations for active and idle timeouts with CAIA. All utilized values are set within the JSON configuration files and depicted in figures 5.2 to 5.5.

[3] https://www.unb.ca/cic/datasets/ids-2017.html

In addition to the basic characteristics of the unsampled dataset, table 6.4 shows the distribution of all occuring attack types for both feature-vectors, listing the number of collected flows tied to specific attack types.

Table 6.4: CIC-IDS-2017, attack distributions

Attack	**CAIA**	**AGM**
Web Attack:Brute Force	1362	498
Web Attack:SQL Injection	13	38
Web Attack:XSS	679	257
Brute Force:FTP-Patator	255	40
Brute Force:SSH-Patator	2,565	689
DoS/DDoS:DoS GoldenEye	7,472	172
DoS/DDoS:DoS Hulk	234,143	242
DoS/DDoS:DoS Slowhttptest	4,217	291
DoS/DDoS:DoS Slowloris	3,894	294
DDoS:LOIT	94,535	261
Infiltration:Cool disk	26	125
Infiltration:Dropbox download	19	172
Infiltration:Dropbox download (Portscan & Nmap) from victim	76,449	259
DoS/DDoS:Heartbleed	3	265
Botnet:ARES	757	257
PortScan:PortScan - Firewall off	159,984	304
PortScan:PortScan - Firewall on	383	280
Total	**586,756**	**4,444**

6.4 Files

While executing all steps necessary to gather the results for every experiment configuration, various informations are saved in form of files for later evaluation. Every experiment has its own labeled basefolder containing files as well as various subfolders. Table 6.5 lists and describes the structure of the basefolder, its subfolders and all saved files obtained for every experiment throughout this study.

Table 6.5: Folders & files

Folder	File	Description
/logs_model-*[1]	dstat.csv	*dstat* logs when executing classficiation, includes memory, swap and CPU usage, includes epoch timestamps.
	report.csv	Classification report generated with scikit-learn.
	result.csv	Information about the chosen model, including the used classifier and its related parameters, PCA feature importance, PCA component number, accuracy-score, confusion-matrix.
	time.csv	Epoch timestamps when executing the preprocessing chain.
/logs_Sampling	dstat.csv	*dstat* logs when executing sampling.
	time.csv	Epoch timestamps when executing sampling.
/model	*.pkl	Saved pickle model used for classification on the remote device.
.	information.csv	Sampling information (technique, mode, steps), JSON configuration used to collect flows with *go-flows*, PCA variance and its necessary component number.
	Merged.csv	Sampled network traffic data used for classification.

[1] ***logs_model-import_remote***:
Subfolder containing gathered data during classification on the remote device.
logs_model-fit_local:
Subfulder containing gathered data while fitting the model on the local machine.

CHAPTER 7

Results and Discussion

The purpose of this research was to investigate the impact of packet sampling techniques on the performance of a cutting-edge Machine Learning algorithm, such as the chosen Random Forest classifier, for network traffic classification and anomaly detection. Simple classification models can be demonstrated to run on low-end hardware devices. The results should help to determine the trade-off between classification accuracy and system resource requirements.

This chapter presents the study's findings, comparing the outcomes of each experiment configuration listed in table 6.1. Table 7.1 in the following section summarizes the main findings. This includes the classification report results described in chapter 6.2.2 as well as the values of the combined performance metrics introduced in chapter 6.2.3. The table also displays various metrics that were evaluated while running the preprocessing chain on the low-end hardware device and can be considered important for the resources spent on the classification itself. This includes obvious script execution parameters as well as parameters bound to the Random Forest estimator model.

7.1 Results

Table 7.1 lists the metrics that were determined during the course of this study. Bold numbers in the tables column showing the results for P_{total} and P_{vector} represent the best ranked experiment configurations for each metric and are further presented in chapter 7.2. In the context of this work, one important factor is an algorithm's ability to reliably detect network anomalies; thus, maintaining high $F_{1,1}$ and Sensitivity scores is critical for successful anomaly detection. The other important factor is resource usage, particularly when the classification is performed on a hardware device that provides so-called low-end performance.

The table is visually divided into two sections: the first section (**I** - **XIII**) lists all experiments carried out in accordance with the AGM specification, while the second section (**XIV** - **XXVI**) is utilizing the CAIA feature representation to obtain the desired feature-vector. Table 6.1 describes how the upper and lower parts of each previously mentioned sections are divided and dedicated to flow-based and packet-based sampling techniques.

7.1.1 Classification

Looking at the achieved scores for Accuracy in table 7.1 across all experiment configurations, it appears to be a impressive result for this specific metric. However, as already briefly mentioned in section 6.2.2, this gives a wrong impression and certainly does not tell the whole story. Due to the obvious significant disparity between benign and attack-related instances, the heavily class-imbalanced dataset (table 6.3) used in this study explains why the achieved Accuracy never falls below 95%. This conclusion also applies to achieved $Recall_0$, $Precision_0$, and $F_{1,0}$ scores.

As the results show, for both the AGM and the CAIA feature vectors, packet-based sampling techniques lower $F_{1,1}$ and $Recall_1$ scores more severely than flow-based techniques. These deteriorating scores can be a result of missing the important first few packets of a flow, which was already one conclusion in the work of Lim et al.[17].

Another explanation for this observation is that, due to the nature of the flow-based sampling technique implementation, flow-based related experiment configurations tend to sample more packets overall compared to their packet-based counterparts. The percentage of actually sampled packets can be looked up in table 6.1 for every experiment configuration.

The flow-based systematic mode *every n-th* packet (**II** - **IV**, **XV** - **XVII**) always collects a larger number of packets compared to similar packet-based configurations (**IX** - **XI**, **XXII** - **XXIV**). Nevertheless, there is a distinction between the AGM and the CAIA feature representation. Based on the total number of packets in the dataset, flow-based AGM samples +1.24% of the packets, whereas CAIA feature representation appears to have a greater impact on the number of sampled packets, with increases ranging from +3.11% to +4,78%.

ID	P_{total}[1]	P_{vector}[2]	Classification[0]							Resources							
			Acc.	Rec_0	Rec_1	$Prec_0$	$Prec_1$	$F_{1,0}$	$F_{1,1}$	t_R	t_C	Inst.	v_C[3]	mem[4]	mdp[5]	mlf[6]	model[7]
I	0.410	**1.299**	0.9987	0.9999	0.6733	0.9989	0.9470	0.9994	0.7870	80.92	4.51	376,717	83,476	483.52	47	1,427	19.41
II	0.269	0.853	0.9986	0.9998	0.6330	0.9990	0.9250	0.9993	0.7511	79.95	4.50	376,717	83,652	490.24	43	1,576	21.51
III	0.251	0.796	0.9986	0.9998	0.6317	0.9987	0.9234	0.9993	0.7502	81.04	4.48	376,717	84,026	512.29	43	1,619	21.95
IV	0.264	0.842	0.9985	0.9998	0.6256	0.9987	0.9093	0.9992	0.7412	81.95	4.60	376,717	81,933	513.65	46	1,571	21.58
V	0.340	**1.095**	0.9985	0.9996	0.6726	0.9989	0.8618	0.9992	0.7555	84.00	4.54	376,717	82,985	509.49	46	1,497	20.50
VI	0.245	0.788	0.9984	0.9997	0.6233	0.9987	0.8746	0.9992	0.7278	83.73	4.61	376,717	81,740	473.30	51	1,762	24.09
VII	0.223	0.713	0.9984	0.9997	0.6233	0.9987	0.8871	0.9992	0.7321	83.80	4.52	376,717	83,333	473.43	57	1,844	24.91
VIII	0.407	**1.290**	0.9987	0.9999	0.6720	0.9989	0.9458	0.9994	0.7856	82.24	4.54	376,717	83,041	503.23	49	1,454	19.81
IX	0.093	0.295	0.9977	0.9998	0.5962	0.9979	0.9314	0.9989	0.7270	52.73	2.98	245,310	82,396	495.34	46	1,642	22.15
X	0.070	0.220	0.9971	0.9997	0.5898	0.9974	0.9373	0.9986	0.7240	41.65	2.42	195,702	80,916	397.55	44	1,671	22.82
XI	0.021	0.069	0.9964	0.9995	0.5638	0.9969	0.8924	0.9982	0.6910	35.00	2.13	167,167	78,423	358.67	48	1,637	22.26
XII	0.033	0.105	0.9969	0.9996	0.5692	0.9973	0.9058	0.9984	0.6991	40.12	2.36	190,230	80,541	375.69	45	1,678	23.03
XIII	0	0	0.9955	0.9995	0.5453	0.9960	0.9116	0.9978	0.6824	28.65	1.71	135,892	79,658	317.18	42	1,629	22.35
XIV	**0.816**	0.791	0.9776	0.9974	0.9187	0.9732	0.9918	0.9852	0.9540	192.28	8.75	695,377	79,473	718.71	85	6,776	94.50
XV	0.630	0.581	0.9592	0.9965	0.8484	0.9512	0.9880	0.9733	0.9129	196.30	8.76	695,377	79,414	796.67	83	9,766	135.96
XVI	0.728	0.670	0.9581	0.9950	0.8489	0.9513	0.9828	0.9726	0.9110	197.91	8.83	695,377	78,792	666.34	84	12,350	174.86
XVII	**0.856**	0.827	0.9656	0.9724	0.9453	0.9814	0.9203	0.9769	0.9326	205.70	9.15	695,377	78,977	848.64	118	21,649	308.07
XVIII	**0.973**	**0.961**	0.9761	0.9716	0.9895	0.9964	0.9215	0.9838	0.9543	188.56	8.76	695,377	79,336	645.23	82	8,142	114.41
XIX	**0.924**	0.919	0.9910	0.9968	0.9740	0.9913	0.9903	0.9940	0.9821	191.92	8.65	695,377	80,355	656.12	84	6,349	88.80
XX	**0.931**	**0.930**	0.9938	0.9974	0.9829	0.9943	0.9923	0.9958	0.9876	191.48	8.71	695,377	79,793	707.07	84	6,196	87.05
XXI	0.753	0.731	0.9717	0.9817	0.9421	0.9805	0.9455	0.9811	0.9438	195.20	8.78	695,377	79,220	835.22	89	6,919	96.55
XXII	0.364	0.311	0.9673	0.9937	0.7843	0.9694	0.9502	0.9814	0.8653	143.40	6.41	506,928	79,075	690.87	82	16,801	236.28
XXIII	0.283	0.247	0.9725	0.9912	0.8197	0.9782	0.9194	0.9847	0.8667	113.72	5.32	410,149	77,075	706.13	76	17,656	249.47
XXIV	0.054	0	0.9550	0.9933	0.6166	0.9582	0.9123	0.9754	0.7358	94.22	4.63	351,056	78,893	690.20	78	17,489	247.07
XXV	0.205	0.159	0.9632	0.9899	0.7586	0.9692	0.9070	0.9794	0.8262	104.70	5.09	387,647	76,112	719.14	71	18,240	258.20
XXVI	0	0	0.9538	0.9897	0.6357	0.9602	0.8738	0.9747	0.7359	73.05	3.65	279,048	79,520	660.47	85	16,801	237.61

Table 7.1: Results

[0] *Classification*: all metrics described in chapter 6.2.2, [1] P_{total}: performance related metric, see equation 6.2.9, **bold** numbers are ranked in table 7.2,
[2] P_{vector}: performance related metric, see equation 6.2.10, **bold** numbers are ranked in table 7.3 [3] v_C: classification speed in predictions/second,
[4] ***mem***: systems max. memory usage in MB, [5] ***mdp***: max. tree depth of the Random Forest model, [6] ***mlf***: max. number of leafes of the Random Forest model,
[7] ***model***: Random Forest model filesize in MB

When compared to sampling *every n-th* packet, the second systematic flow-based sampling mode *first n* packets (**V** - **VII**, **XVIII** - **XX**) collects fewer packets on average. Across all investigated experiment configurations, the AGM feature representation sampling the *first n* packets collects the fewest amount of packets, ranging from 4.62% to 7.61% of the total number of packets in the dataset. The CAIA feature representation for the same mode collects significantly more packets, ranging from 16.31% to 26.15%.

The granularity resulting from the different flow-keys used to collect the flows for the AGM and CAIA feature representation causes this difference. As a result, the CAIA feature representation in general generates a significantly higher number of flows, which affects the number of collected packets for the flow-based mode sampling the *first n* packets of a flow.

Because all packet-based sampling modes (**IX** - **XIII**, **XXII** - **XXVI**) are applied directly to raw network captures, the number of packets collected is consistent across all related experiment configurations. Nonetheless, the number of flows exported decreases significantly after using packet-based sampling. This has a negative impact on classification because instances are not being passed for classification, but it can also reduce the demand for resources.

7.1.2 Resources

This work demonstrates that memory usage, in particular, can be one of the most limiting factors in such a low-end hardware environment. It is critical to avoid using SWAP memory at all costs, as this will significantly slow down the entire classification process. The script operates in this manner to avoid using the system's SWAP space by applying datatype conversions when possible, splitting files, re-loading splits, reading data in chunks, or applying incremental PCA.

The experiments started with a Raspberry Pi Zero W with only 500MB of RAM for classification. From the beginning, it was clear that such a low-end device would never be able to fit a model to a larger set of instances in a reasonable amount of time, but classification would be possible. When the number of instances increased by merging the workday files from the *CIC-IDS-2017* dataset, memory usage immediately became an issue when executing the classification script.

As previously stated, one obvious finding is that when packet-based sampling modes are used, the overall number of instances to classify decreases significantly. Depending on the feature-vector used, the remaining instances to classify can be as low as 36% of the total number of flows in the unsampled traffic for the CAIA feature representation. This number can drop to 40% with the AGM feature vector. Even though both specifications collect flows using different flow-keys, it appears that the effect of sampling affects the number of instances for both feature vectors in a similar way. In order to respect the different flow-keys and, as a result, the different number of instances to classify, feature scaling for the number of instances is based on the respective feature-vector when calculating both performance parameters, as described in chapter 6.2.3.

While packet-based sampling appears to have a negative effect on the achieved classification results, its positive effect is that the expenditure on resources to first collect flows and sample packets within each flow afterwards is not necessary when compared to flow-based sampling. This results in less necessary resources which will not only reduce the hardware usage but also the time required to generate processable network traffic data for classification. The number of instances to classify has a direct impact on the overall runtime of the classification because it requires more processing within the preprocessing chain, as described in chapter 5.3.4. This aspect was not addressed in this study, but has to be considered at some point.

The classification speed, defined as the number of instances classified per second, is consistent across all experiment configurations, with minor variations that appear to be related to the complexity of the estimator model. The CPU single-core processing power is the limiting factor for this metric, regardless of the sampling technique, mode, or pattern used. Beside the CPU being the main contributor to improve classification speed, SWAP usage significantly reduces classification speed.

Preprocessing

The overall script runtimes, as exemplary illustrated for experiment **XVII** in figure 7.1, heavily rely on four steps in the preprocessing chain:

- **Import:** Importing the data in chunks.
- **Split:** The necessity to split the files to avoid SWAP usage.
- **PCA:** Fitting PCA to the training portion of the data.
- **Classification:** Generating predictions for the test portion of the data.

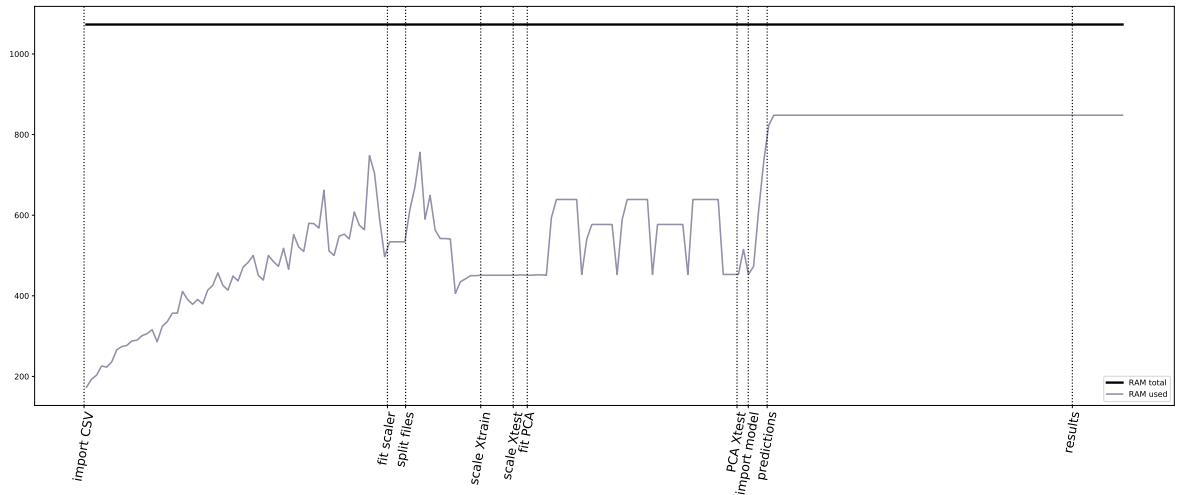

Figure 7.1: Memory usage over time, experiment **XVII**

To improve the overall runtime in the preprocessing chain, it is possible to create the models for the scaler and PCA on more powerful devices, similar to the classifier, and

then import the generated model on low-end hardware. The PCA, in particular, requires a significant portion of the overall runtime and can be regarded as a major bottleneck in this work concerning runtimes.

Model

In a real-world environment, importing data can be manageable; the number of packets to process in a given time period is determined by a network's bandwidth and actual throughput. What can be a problem is fitting a model because this task requires a lot of time and resources, making it seem unreasonable to ever do this on any low-end hardware device. Fitting models elsewhere and importing updated models on low-end hardware should be the preferred method. Asadi et al. [22] show in their work about runtime optimization that it is possible to reduce the per-instance prediction time for a decision tree-based algorithm when limiting the model complexity.

Different feature representations appear to result in different model sizes. Looking at model-related statistics such as model size, maximum number of leaves, and maximum tree depth, it is clear that all experiments using the AGM feature representation (**I** - **XIII**) produce less complex models. As described in the following section, this can have an impact on resource usage.

7.2 Performance

This section goes into detail, describing the most promising experiment configurations to achieve less resource usage while maintaining good classification results, based on the parameters P_{vector} and P_{total} introduced in chapter 6.2.3.

Table 7.2 ranks experiment configurations according to P_{total}.

Table 7.2: Ranking based on P_{total}

ID	P_{total}	Sampling			
		JSON	**technique**	**mode**	**steps**
XVIII	0.973	CAIA	flow-based	first n	n=3
XX	0.931	CAIA	flow-based	first n	n=7
XIX	0.924	CAIA	flow-based	first n	n=5
XVII	0.856	CAIA	flow-based	every n-th	n=7
XIV	0.816	CAIA	flow-based	unsampled	

Another advantage of the chosen flow-based sampling implementation is the consistent number of instances to classify when compared to flow-based sampling. This behaviour is taken into account for both metrics, P_{vector} and P_{total}. More instances to classify leads to a higher score for both metrics, as already described in chapter 6.2.3. This can be considered as a negative effect when considering the resources necessary to collect the

network traffic before classification happens, however this aspect is not investigated in this study.

When the absolute classification results are considered throughout all experiment configurations, like it is done with P_{total}, the gathered results show that the CAIA feature-vector performs best. As shown in table 7.1, the CAIA feature-vector clearly outperforms AGM in all important classification-related metrics.

It is noticeable that the *unsampled* CAIA feature representation is not ranked as the best P_{total} configuration. Looking at all results reveals that the main reason for this ranking can be found in slightly lower $F_{1,1}$ and $Recall_1$ scores, while memory usage during classification is above average.

Figure 7.2 compares the best ranked experiment configurations for P_{total} in form of a spider chart. All shown configurations maintain high scores for $Recall_1$, $F_{1,1}$ and $Precision_1$.

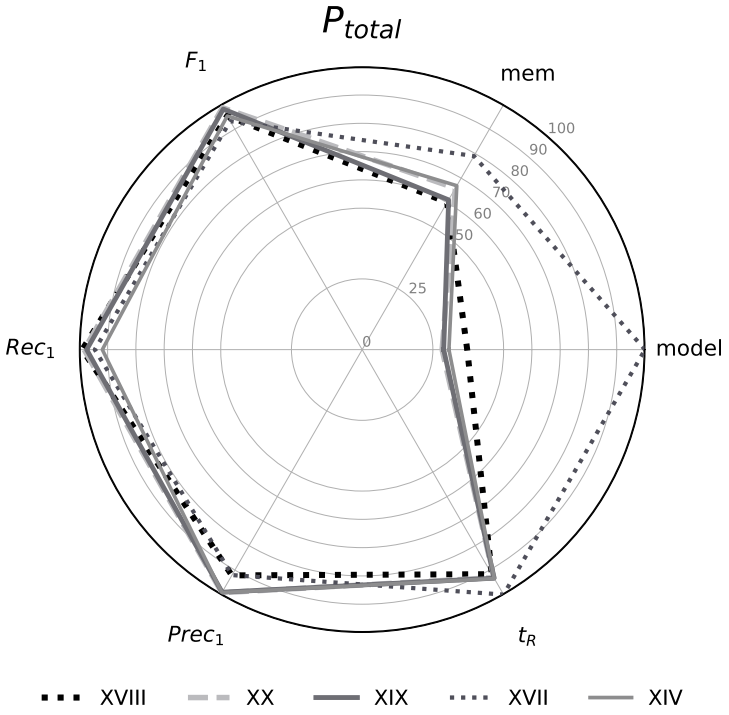

Figure 7.2: P_{total} comparison chart

It is also worth mentioning that the best results with P_{total} are obtained using the same mode that samples the *first n* packets of a flow. This finding is consistent with Lim et al. [17], concluding that the first few packets of a flow are critical for good classification results. When looking at performance related metrics in table 7.1 it seems like CAIA models for sampling the *first n* packets of a flow tend to be a bit less complex when it

comes to their maximum numbers of leaves or the model sizes compared to all other sampling modes in this study.

In this study, experiment configuration **XVII** (flow-based, sample *every n-th* packet of a flow, n=7) produced the most complex model in terms of model size, maximum tree depth, and maximum number of leaves. This specific configuration put the most strain on system memory and total runtime, which appears to be consistent with the findings of Asadi et al.[22], as previously mentioned. Similar configurations with the same sampling mode but lower values for n produce significantly less complex Random Forest classifier models. This behavior could be caused by missing packets in a flow, resulting in feature values that require the classifier to generate more complex trees. It should be noted that lower sampling rates do not always imply less complex models and less hardware resource usage.

Table 7.3 ranks experiment configurations according to P_{vector}.

Table 7.3: Ranking based on P_{vector}

ID	P_{vector}	Sampling			
		JSON	**technique**	**mode**	**steps**
I	1.299	AGM	flow-based	unsampled	
VIII	1.290	AGM	packet-based	unsampled	
V	1.095	AGM	flow-based	first n	n=3
XVIII	0.961	CAIA	flow-based	first n	n=3
XX	0.930	CAIA	flow-based	first n	n=7

When classification results are weighted towards achievable results within each experiment's feature-vector category, as with P_{vector}, the AGM feature-vector performs best when used on *unsampled* traffic, followed by AGM sampling the *first n* packets of a flow. The CAIA feature-vector is also present in experiment configurations **XVIII** and **XX**.

Figure 7.3 compares the best ranked experiment configurations for P_{vector}.

This comparison chart explains why the AGM feature-vector is ranked higher than the CAIA feature-vector when the parameter P_{vector} is considered. AGM appears to produce smaller, less complex models for the Random Forest classifier in general, resulting in less memory usage and significantly shorter runtimes. The reason that the *unsampled* AGM experiment configurations **I** and **VIII** are the best ranked is because they use less memory and achieve the best classification results within their feature-vector category. When compared to all CAIA-related experiments (**XIV** - **XXVI**), the AGM classification module runtimes are significantly shorter, increasing the performance metric. It should be noted that classification runtimes are highly dependent on the number of instances to classify; while AGM in this study has slightly faster classification speed, the main cause of the shorter runtimes is the difference in the number of instances resulting from different flow-keys.

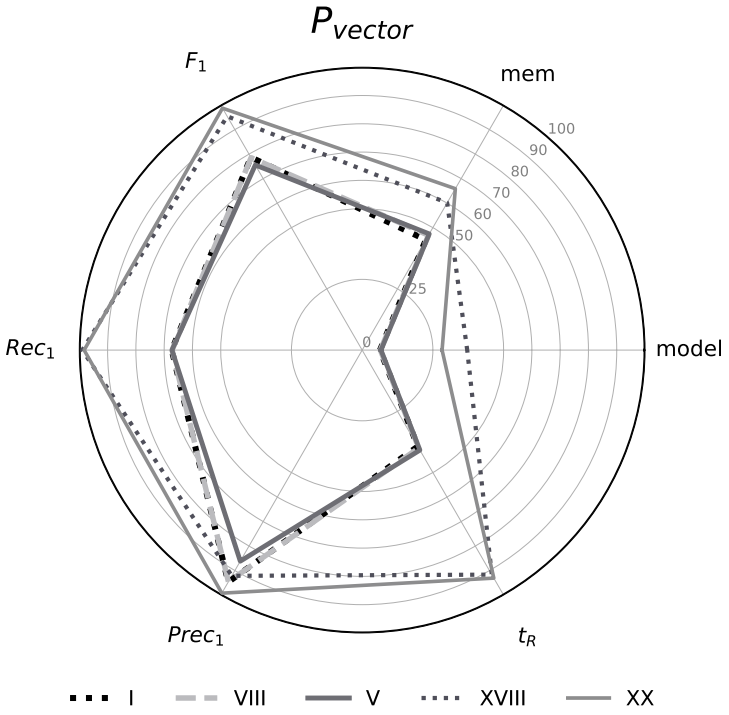

Figure 7.3: P_{vector} comparison chart

The differences in results for packet-based and flow-based AGM on *unsampled* traffic are most likely due to different methods used to create the mode feature for the TCP flags, as mentioned briefly in chapter 5.3.2, item 10.

7.3 Summary

Overall, having a closer look at both parameters P_{vector} and P_{total} and comparing applied sampling techniques with each other for both feature-vectors, AGM and CAIA, it seems like CAIA outperforms AGM when sampling is applied. Beside CAIA classification scores being baseline better than AGM scores in this study, the classification scores with CAIA tend to deteriorate less compared to AGM for all forms of applied sampling in this study.

This behavior could be based on the argument that CAIA features focus on minimum and maximum values, which are less affected by missing packets due to sampling than aggregated feature information used with the AGM feature-vector. As a result, missing packets may have no effect on the feature values obtained for a flow collected using the CAIA feature-vector. Multiple packets within the same flow may have similar, if not identical, minimum and maximum feature-values for the CAIA feature-vector. As a result, sampling may have only a minor impact on the collected flow. The aggregated feature values of the AGM vector, on the other hand, may affect more flows compared to CAIA.

Another finding of this study is that when compared to all flow-based configurations, packet-based sampling configurations achieve quite poor results for classifying sampled traffic. When sampling is applied across all configurations for packet-based sampling, the classification scores appear to degrade more, while having no positive effect on the complexity of the generated classifier model. The models for packet-based sampling are at least as large and complex as the flow-based models, if not more so, especially for the CAIA feature-vector. Nonetheless, it is important to note that the flow-based sampling modes investigated in this study collect more packets than packet-based sampling because the flow-based implementation exports every flow before sampling. When a similar number of packets is collected for classification, the difference in classification performance may be reduced.

The results also show that the AGM feature representation creates smaller, less complex models in combination with the Random Forest classifier compared to the CAIA feature representation for all related experiment configurations. The model size complexity seems to correlate with the memory usage. Comparing AGM related configurations (**I** - **XIII**) with specific CAIA experiment configurations (**XXIII** - **XXVI**) classifying a similar number of instances, the AGM related experiments system memory usage seems to be significantly lower, which could be a result of the lower model complexity. Based on the work of Van Essen et al.[20], one way to potentially improve performance is to limit the complexity of the generated Random Forest model by adjusting the classifier parameters for maximum tree depth and maximum leave numbers while increasing the maximum number of trees generated for the random forest. However, such an improvement is left for future work.

CHAPTER 8

Conclusion

This study used a variety of experiment configurations to investigate the effect of sampling, not only on classification results but also on resource usage for the device on which the classification is performed.

To obtain results for the classification, the work began with a Raspberry Pi Zero W as low-end hardware device. This approach demonstrated that available memory is an important factor in classification. While the CPU is the most important factor in classification speed, if the classification device lacks memory, the classification speed will suffer greatly due to SWAP usage. This necessitated some steps throughout the classification process, such as splitting files or converting datatypes, in order to reduce the memory footprint.

Another looming issue was the incompatibility of the Random Forest model created on the x86_64 machine and the armv6l Rasbperry Pi Zero W for classification. The exported model could not be used for the Raspberry Pi Zero W. This problem could not be easily resolved, so the classification results were gathered on a virtual machine.

The findings revealed that certain sampling modes have a net positive effect on resource usage while maintaining good classification scores with the Random Forest classifier. Flow-based sampling techniques appear to be more suitable than packet-based sampling for reliable classification of every instance, including the ability to obtain the first few packets of every flow. For sampled traffic, the CAIA feature-vector outperforms AGM in this study. While the AGM feature representation generates smaller and less complex models in general, the classification results appear to degrade more severely when network traffic is sampled. Aside from that finding, the absolute classification results with AGM in this study are significantly worse than CAIA.

For deployment, the applicability of the investigated sampling schemes in a real environment must be considered. This includes several network characteristics, the most

important of which is the sheer volume of traffic on large networks. Packet-based implementations put less strain on hardware because sampling can be performed directly on appropriate hardware devices without the need for processing resources to apply the sampling itself. Flow-based techniques as described in this work, on the other hand, are significantly more demanding to implement. First, all packets passing through an observation point must be considered for flow collection; then, at some point, a flow must be terminated and exported. At this point additional resources are necessary to apply the sampling before further processing and classification happens. These requirements, when compared to packet-based sampling, make predicting the resources required to properly dimension the hardware more difficult.

Future research on this topic could revolve around the question of when sampling becomes necessary to deal with high traffic volumes. Another intriguing question is what minimal hardware requirements are required to perform classification on low-end hardware devices while also including flow-collection and feature calculations to obtain a certain feature-vector for classification. This is a critical question when it comes to online processing of network traffic for classification. The findings could also enable machine learning-based anomaly detection for network security solutions in home networks such as *pfSense*[1] or its fork *OPNsense*[2], both of which can already run reliably on low-end hardware.

[1] https://www.pfsense.org/
[2] https://opnsense.org/

APPENDIX A

Appendix

A.1 Environment

A detailed description of the environmental setup process for all machines can be found in the Git repository within its *docs/setup* folder.

A.1.1 Server

For this research the server machine applies various sampling techniques on the network traffic captures provided by the *CIC-IDS-2017* dataset. The server also fits and exports models of the Random Forest classifier for every experiment configuration investigated in this study. Those exported models are used for classification on the low-end hardware device.

Specifications

The hardware specifications used for all server-side script executions are shown in table A.1:

Table A.1: Server specifications

CPU	Byte Order	RAM	disk	OS
AMD Ryzen 5 3600[1]	Little Endian	16GB	500GB	Ubuntu
6-Core Processor 3.6GHz		DDR4	Samsung 860 EVO	20.10
x86_64		2667 MT/s		groovy

[1] https://www.amd.com/en/products/cpu/amd-ryzen-5-3600

A.1.2 Raspberry Pi Zero W

This research started utilizing a Raspberry Pi Zero W as low-end device. Due to upcoming issues mentioned in chapter 8 the study gathered its results utilizing a virtual machine. Details about the error output on the Raspberry Pi Zero W are documented in the Git repository.

Specifications

Table A.2: Raspberry Pi Zero W specifications

CPU	Byte Order	RAM	Disk	OS
Broadcom BCM2835	Little Endian	512MB	8GB	DietPi
ARM11 Single Core 1GHz armv6l		LP-DDR2	microSD	7.0.2

A.1.3 Virtual Machine

After encountering the issue with the Raspberry Pi Zero W a virtual machine has been configured to gather results. Its hardware specifications are similar to the Raspberry Pi Zero W. However, the machine has more memory available. This was necessary since *DietPi* as installed OS on the Raspberry Pi Zero W showed superiour memory efficiency compared to the headless *Debian* installation on the virtual machine. *Proxmox Virtual Environment*[2] was used to virtualize the low-end hardware device as LXC container.

Specifications

Table A.3: LXC container specifications

CPU	Byte Order	RAM	Disk	OS
Intel Atom D525[3]	Little Endian	1024MB	8GB	Debian
1-Core 1.80GHz x86_64		DDR3 800 MT/s	SSD	10.9 buster

[2] https://www.proxmox.com/en/

[3] https://ark.intel.com/ ... /products/49490/intel-atom-processor-d525-1m-cache-1-80-ghz.html

A.2 Framework Configuration

All configurations and parameters necessary can be adjusted within this configuration file. Throughout all scripts Pathlib is used to generate filepaths. Detailed explanations on all parameters are commented within the configuration. The structure is organized into the following groups:

A.2.1 Directories

This section contains all necessary information for proper path generation, it provides the full path to directory containing all original PCAP files, filenames as dictionary and the pattern used to merge multiple files.

A.2.2 Logs

Contains filenames of all relevant logs, the corresponding folders within the working directory and full paths.

A.2.3 Tools

Contains paths to executable files and scripts.

go-flows[4] is a highy customizable general-purpose flow exporter written in go, available at the CN-TU github repository. It's highly recommended to have a look at the *go-flows* documentation page[5] to get an idea about the possibilities that can be provided within the passed configuration file, which acts as feature-vector.

In addition to *go-flows* another script available at the CN-TU repository is used to label all sampled and flow-collected data from the Intrusion Detection Evaluation Dataset *CIC-IDS-2017*. The labeling script[6] itself takes a CSV containing packets or flows ending with '_unlabeled.csv' and saves the resulting labelled data as '.csv' for further processing.

A.2.4 Sampling

This section contains all available feature-vectors, the folder where these vectors need to be stored as well as all available sampling-modes. There is a possibility to set certain treshold values so it is easy to distinguish flow-based and packet-based sampling modes.

A.2.5 Experiments

Set files, feature-vectors, steps and modes for automated experiment execution. There is also the possibility to tweak other parameters like batchsize, split, splitsize, chunksize and the PCA component number.

[4] https://github.com/CN-TU/go-flows
[5] https://pkg.go.dev/github.com/CN-TU/go-flows
[6] https://github.com/CN-TU/Datasets-preprocessing/tree/master/CIC-IDS-2017/labeling

A.2.6 Remote Machine

Set necessary information to transfer pickle modelfiles and sampled CSVs via rsync and execute scripts via SSH on the remote machine. It's recommended to setup key-based SSH authentication.

```
5  ##############################################################################################
6  # BASIC CONFIGURATION
7  ##############################################################################################
8  # BASE directories
9  # used for proper path generation
10 mntd    = PurePosixPath('/mnt')
11 wd      = Path.cwd()
12 hd      = Path.home()
13 ##############################################################################################
14 # PCAP filepath & filenames (without extension)
15 fpath = mntd / 'data' / 'CIC-IDS2017' / 'PCAP'
16 filenames = {
17 0:'Merged',
18 1:'Monday-WorkingHours',
19 2:'Tuesday-WorkingHours',
20 3:'Wednesday-WorkingHours',
21 4:'Thursday-WorkingHours',
22 5:'Friday-WorkingHours'
23 }
24 pattern = '*Hours.csv' # pattern used to merge files in Sampling.py
25 ##############################################################################################
26 # FOLDERS, FILES & LOGS
27 # sampled CSVs & result logs, temporary logs in working directory
28 flowfolder    = mntd / 'data' / 'CIC-IDS2017' / 'PCAP' / 'flow-sampledCSV'
29 packetfolder  = mntd / 'data' / 'CIC-IDS2017' / 'PCAP' / 'packet-sampledCSV'
30 # folder containing results, logs, temporary logs and sampled CSV for later evaluation
31 eflowfolder   = mntd / 'data' / 'CIC-IDS2017' / 'Experiments' / 'flow-sampledCSV'
32 epacketfolder = mntd / 'data' / 'CIC-IDS2017' / 'Experiments' / 'packet-sampledCSV'
33 # experiment configuration foldername
34 foldername    = '{}_mode{}_vector{}_steps{}_{}'
35 # logfiles
36 csv_dstat   = 'dstat.csv'
37 csv_time    = 'time.csv'
38 csv_result  = 'result.csv'
39 csv_report  = 'report.csv'
40 csv_info    = 'information.csv'
41 # working directory folders
42 tmp         = wd / 'tmp'
43 logs        = wd / 'logs'
44 figures     = wd / 'figures'
45 # full path to wd logfiles
46 time        = logs / csv_time
47 dstat       = logs / csv_dstat
48 result      = logs / csv_result
49 report      = logs / csv_report
50 # pickle-model
51 model_remote    = '{}_model_remote.pkl'
52 model_local     = '{}_model_local.pkl'
53 # framework configuration
54 configuration   = 'config.py'
55 ##############################################################################################
56 # TOOLS
57 # path to executables
58 goflowspath     = hd / 'Git' / 'go-flows' / 'go-flows'
59 labelingpath    = mntd / 'data' / 'BSc-e1027075' / 'Labeling.py'
60 ##############################################################################################
```

Listing 77: Framework configuration: basic options

```
62 ##############################################################################################
63 # SAMPLING CONFIGURATION
64 ##############################################################################################
65 # FEATURE-VECTORS
66 # directory within working directory and filenames of available feature-vectors
67 vectorfolder = 'go-flows-configurations'
68 vectors = {
69 1:'AGM_10s_flowbased.json',
70 2:'AGM_60s.json',                       # unused
71 3:'AGM_3600s.json',                     # unused
72 4:'CAIA_flowSampling.json',
73 5:'CAIA_packetSampling.json',
74 6:'AGM_10s.json',
75 7:'AGM_60s.json',                       # unused
76 8:'AGM_3600s.json'                     # unused
77 }
78 ##############################################################################################
79 # SAMPLING-MODES
80 # modes available for perflow- and packetsampling
81 fsamplingmode = {
82 1:'every n-th packet',
83 2:'sample & skip n packets',            # unused
84 3:'first n packets',
85 4:'sample n, skip n-1, sample n-2 ...'  # unused
86 }
87 psamplingmode = {
88 5:'every n-th packet',
89 6:'n out of N',                         # unused
90 7:'probability',
91 8: 'timebased'                         # unused
92 }
93 # dictionary used for automated execution
94 samplingmode = fsamplingmode.copy()
95 samplingmode.update(psamplingmode)
96 ##############################################################################################
97 # TRESHOLD, change details here
98 # numbers marks specific limits used for plausibility on specific
99 # argument combinations to guarantee correct automated execution
100 vectorlimit     = 5 # used to make experiment plausibility check in Master.py
101 samplinglimit   = 5 # used to make experiment plausibility check in Master.py
102 modelimit       = 4 # unused
103 flowlimit       = 4 # used to set labeling mode in FlowSampling.py
104 packetlimit     = 4 # used to set labeling mode in PacketSampling.py
105 ##############################################################################################
```

Listing 78: Framework configuration: sampling options

```
107 ################################################################################
108 # EXPERIMENT CONFIGURATION
109 ################################################################################
110 # REMOTE MACHINE
111 # username, IP and working directory for the remote machine
112 # LXC container
113 remotewd    = 'BSc-e1027075'
114 remoteuser  = 'thesis'
115 remoteip    = '10.10.40.209'
116 remote      = '{}@{}'.format(remoteuser,remoteip)
117 remoteconf  = '/home/{}/{}/{}'.format(remoteuser,remotewd,configuration)
118 ################################################################################
119 # EXPERIMENTS
120 # files, feature-vectors, sampling-modes & sampling-steps to process
121 file        = [5]  # 0 to process all all workday files, 1 to 5 for workdays
122 vector      = [4,6]  # determines used go-flows specification file specified above
123 mode        = [3]  # determines applied sampling mode specified above
124 steps       = [9]  # value for n, 0 to process unsampled PCAP
125 seed        = 1000 # seed number used for random sampling
126 ################################################################################
127 # SCRIPTS
128 # Principal Component Analysis
129 n_PCA       = 12       # unused
130 PCA_var     = 0.90     # explained variance to achieve with PCA components
131 PCA_batch   = 10**5    # batchsize used for incremental PCA
132 # StandardScaler
133 batchsize   = 10**5    # batchsize used for partial fit (default 10**5)
134 # CSV import
135 chunksize   = 10**5    # lines per chunks (default 10**5)
136 # Sampling
137 split       = 5000     # number of packets per splitfile created with editcap
138 # Classification
139 splitsize   = 25*10**4 # number of lines (flows) per file (default 25*10**4)
140 ################################################################################
141 # RANDOM FORST CLASSIFIER
142 # estimators
143 maxtrees    = 100      # maximum number of trees
144 maxdepth    = None     # maximum tree-depth
145 maxleaves   = None     # maximum number of leafes per tree
146 ################################################################################
147
148 ################################################################################
149 # OUTPUTS
150 ################################################################################
151 # EVALUATION
152 types = ['*.png','*.csv','*.pdf'] # used to clean figures folder when before processing
        data
153 ################################################################################
154 # COLORS
155 # uses terminal color palette, options are:
156 # BLACK, RED, GREEN, YELLOW, BLUE, MAGENTA, CYAN, WHITE
157 vcolor = Fore.GREEN
158 ################################################################################
```

Listing 79: Framework configuration: experiment & output options

List of Figures

List of Tables

Bibliography

[1] Vivens Ndatinya, Zhifeng Xiao, Vasudeva Rao Manepalli, Ke Meng, and Yang Xiao. Network forensics analysis using wireshark. *International Journal of Security and Networks*, 10(2):91–106, 2015.

[2] Basil Alothman. Raw network traffic data preprocessing and preparation for automatic analysis. In *2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, pages 1–5. IEEE, 2019.

[3] Leslie F Sikos. Packet analysis for network forensics: A comprehensive survey. *Forensic Science International: Digital Investigation*, 32:200892, 2020.

[4] Rick Hofstede, Pavel Čeleda, Brian Trammell, Idilio Drago, Ramin Sadre, Anna Sperotto, and Aiko Pras. Flow monitoring explained: From packet capture to data analysis with netflow and ipfix. *IEEE Communications Surveys & Tutorials*, 16(4):2037–2064, 2014.

[5] Gernot Vormayr, Joachim Fabini, and Tanja Zseby. Why are my flows different? a tutorial on flow exporters. *IEEE Communications Surveys & Tutorials*, 22(3):2064–2103, 2020.

[6] Jürgen Quittek, Tanja Zseby, Benoit Claise, and Sebastian Zander. Requirements for ip flow information export (ipfix). Technical report, RFC 3917 (informational), 2004.

[7] Salvatore D'Antonio, Tanja Zseby, Christian Henke, and Lorenzo Peluso. Flow selection techniques. *RFC 7014 (Standards Track), Internet Engineering Task Force*, 2013.

[8] Tanja Zseby, Maurizio Molina, Nick Duffield, Saverio Niccolini, and Fredric Raspall. Sampling and filtering techniques for ip packet selection. *draft-ietf-psamp-sample-tech-07 (work in progress)*, 2005.

[9] Félix Iglesias and Tanja Zseby. Pattern discovery in internet background radiation. *IEEE Transactions on Big Data*, 5(4):467–480, 2017.

[10] Jonathan Jedwab, Peter Phaal, and Bob Pinna. *Traffic estimation for the largest sources on a network, using packet sampling with limited storage.* Hewlett-Packard Laboratories, Technical Publications Department, 1992.

[11] Nick Duffield. Fair sampling across network flow measurements. *ACM SIGMETRICS Performance Evaluation Review*, 40(1):367–378, 2012.

[12] Muhammad Shafiq, Xiangzhan Yu, Asif Ali Laghari, Lu Yao, Nabin Kumar Karn, and Foudil Abdessamia. Network traffic classification techniques and comparative analysis using machine learning algorithms. In *2016 2nd IEEE International Conference on Computer and Communications (ICCC)*, pages 2451–2455. IEEE, 2016.

[13] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM computing surveys (CSUR)*, 41(3):1–58, 2009.

[14] Shikha Agrawal and Jitendra Agrawal. Survey on anomaly detection using data mining techniques. *Procedia Computer Science*, 60:708–713, 2015.

[15] Monowar H Bhuyan, Dhruba Kumar Bhattacharyya, and Jugal K Kalita. Network anomaly detection: methods, systems and tools. *Ieee communications surveys & tutorials*, 16(1):303–336, 2013.

[16] Dhruba Kumar Bhattacharyya and Jugal Kumar Kalita. *Network anomaly detection: A machine learning perspective.* Chapman and Hall/CRC, 2019.

[17] Yeon-sup Lim, Hyun-chul Kim, Jiwoong Jeong, Chong-kwon Kim, Ted" Taekyoung" Kwon, and Yanghee Choi. Internet traffic classification demystified: on the sources of the discriminative power. In *Proceedings of the 6th International COnference*, pages 1–12, 2010.

[18] Petr Velan, Milan Čermák, Pavel Čeleda, and Martin Drašar. A survey of methods for encrypted traffic classification and analysis. *International Journal of Network Management*, 25(5):355–374, 2015.

[19] Fares Meghdouri, Tanja Zseby, and Félix Iglesias. Analysis of lightweight feature vectors for attack detection in network traffic. *Applied Sciences*, 8(11):2196, 2018.

[20] Brian Van Essen, Chris Macaraeg, Maya Gokhale, and Ryan Prenger. Accelerating a random forest classifier: Multi-core, gp-gpu, or fpga? In *2012 IEEE 20th International Symposium on Field-Programmable Custom Computing Machines*, pages 232–239. IEEE, 2012.

[21] Shay Vargaftik, Isaac Keslassy, Ariel Orda, and Yaniv Ben-Itzhak. Rade: Resource-efficient supervised anomaly detection using decision tree-based ensemble methods. *arXiv preprint arXiv:1909.11877*, 2019.

[22] Nima Asadi, Jimmy Lin, and Arjen P de Vries. Runtime optimizations for prediction with tree-based models. *arXiv preprint arXiv:1212.2287*, 2012.

[23] Nigel Williams and Sebastian Zander. Evaluating machine learning algorithms for automated network application identification. 2006.

[24] Daniel J Arndt and A Nur Zincir-Heywood. A comparison of three machine learning techniques for encrypted network traffic analysis. In *2011 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 107–114. IEEE, 2011.

[25] Jeffrey Erman, Anirban Mahanti, and Martin Arlitt. Qrp05-4: Internet traffic identification using machine learning. In *IEEE Globecom 2006*, pages 1–6. IEEE, 2006.

[26] Vivienne Sze, Yu-Hsin Chen, Joel Emer, Amr Suleiman, and Zhengdong Zhang. Hardware for machine learning: Challenges and opportunities. In *2017 IEEE Custom Integrated Circuits Conference (CICC)*, pages 1–8. IEEE, 2017.

[27] Milad Taghavi and Mahsa Shoaran. Hardware complexity analysis of deep neural networks and decision tree ensembles for real-time neural data classification. In *2019 9th International IEEE/EMBS Conference on Neural Engineering (NER)*, pages 407–410. IEEE, 2019.

[28] Ren-Hung Hwang, Min-Chun Peng, Chien-Wei Huang, Po-Ching Lin, and Van-Linh Nguyen. An unsupervised deep learning model for early network traffic anomaly detection. *IEEE Access*, 8:30387–30399, 2020.

[29] Arthur L Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of research and development*, 3(3):210–229, 1959.

[30] Tom M Mitchell et al. Machine learning. 1997.

[31] Tin Kam Ho. Random decision forests. In *Proceedings of 3rd international conference on document analysis and recognition*, volume 1, pages 278–282. IEEE, 1995.

[32] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[33] J Ross Quinlan. *C4. 5: programs for machine learning*. Elsevier, 2014.

[34] Rifkie Primartha and Bayu Adhi Tama. Anomaly detection using random forest: A performance revisited. In *2017 International conference on data and software engineering (ICoDSE)*, pages 1–6. IEEE, 2017.

[35] Nigel Williams, Sebastian Zander, and Grenville Armitage. A preliminary performance comparison of five machine learning algorithms for practical ip traffic flow classification. *ACM SIGCOMM Computer Communication Review*, 36(5):5–16, 2006.

[36] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp*, 1:108–116, 2018.